Intent-Based Architecture — Configurator Layer

Architecture Synthesis Plan: Addendum

> **Problem Statement:** The current runtime pipeline has hardcoded coupling between Intent JSON field names, Block category dispatch logic, Resource Sizing formulas, and Compliance constraint application. Any change to Intent JSON, Block JSON, or Compliance JSON requires code changes, testing, and redeployment.

> **Solution:** Introduce a **Configurator Layer** — a meta-configuration system that makes all field mappings, sizing formulas, and policy rules data-driven. The runtime engine becomes a generic evaluator that never needs code changes when JSONs evolve.

> **North Star Goal:** Zero code changes when Intent JSON, Block JSON, or Compliance JSON evolve. All logic lives in configuration. The runtime engine is generic.

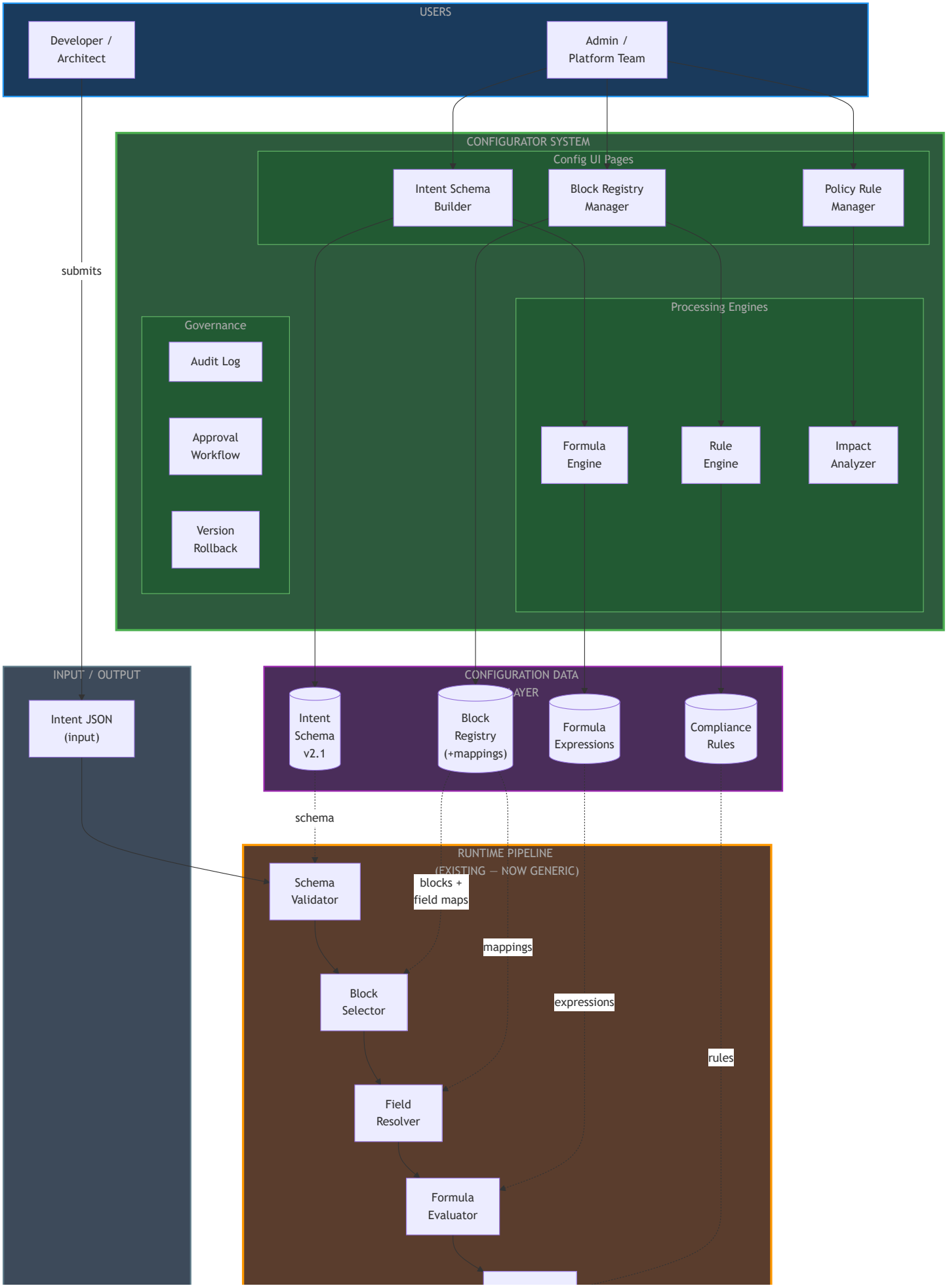
Table of Contents

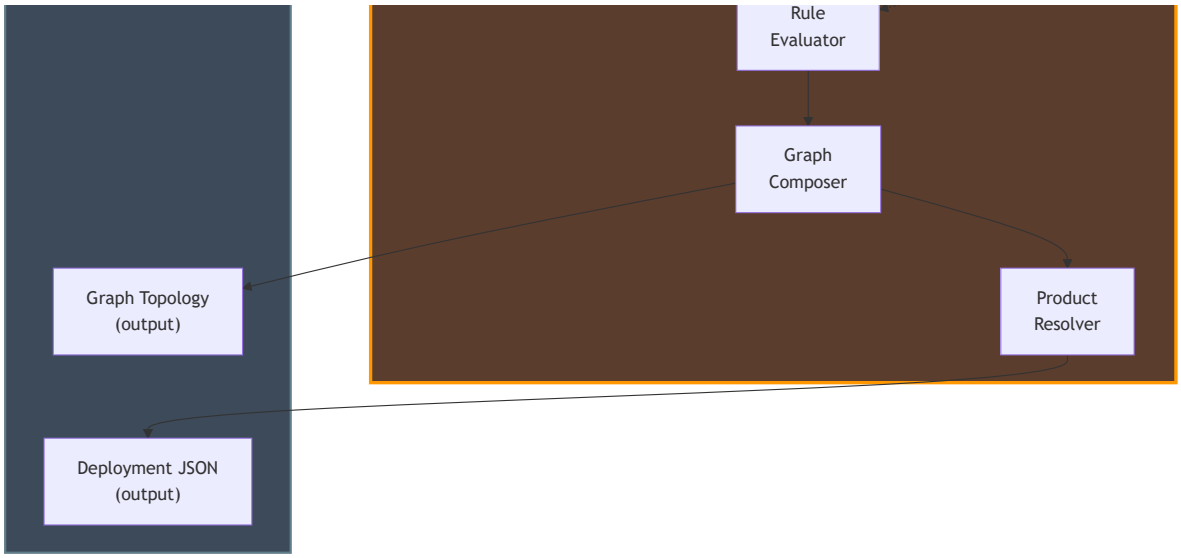
1. Master Architecture Overview

This is the bird's-eye view of the full system after the Configurator Layer is integrated. Two user personas interact with the system:

- **Admin / Platform Team**: Configures intent schemas, block registries, formulas, and policy rules via UI pages
- **Developer / Architect**: Submits Intent JSON through the runtime pipeline

The key principle: **Config Stores sit between the Configurator UI and the Runtime Pipeline**. The runtime reads everything from config — never from hardcoded logic.

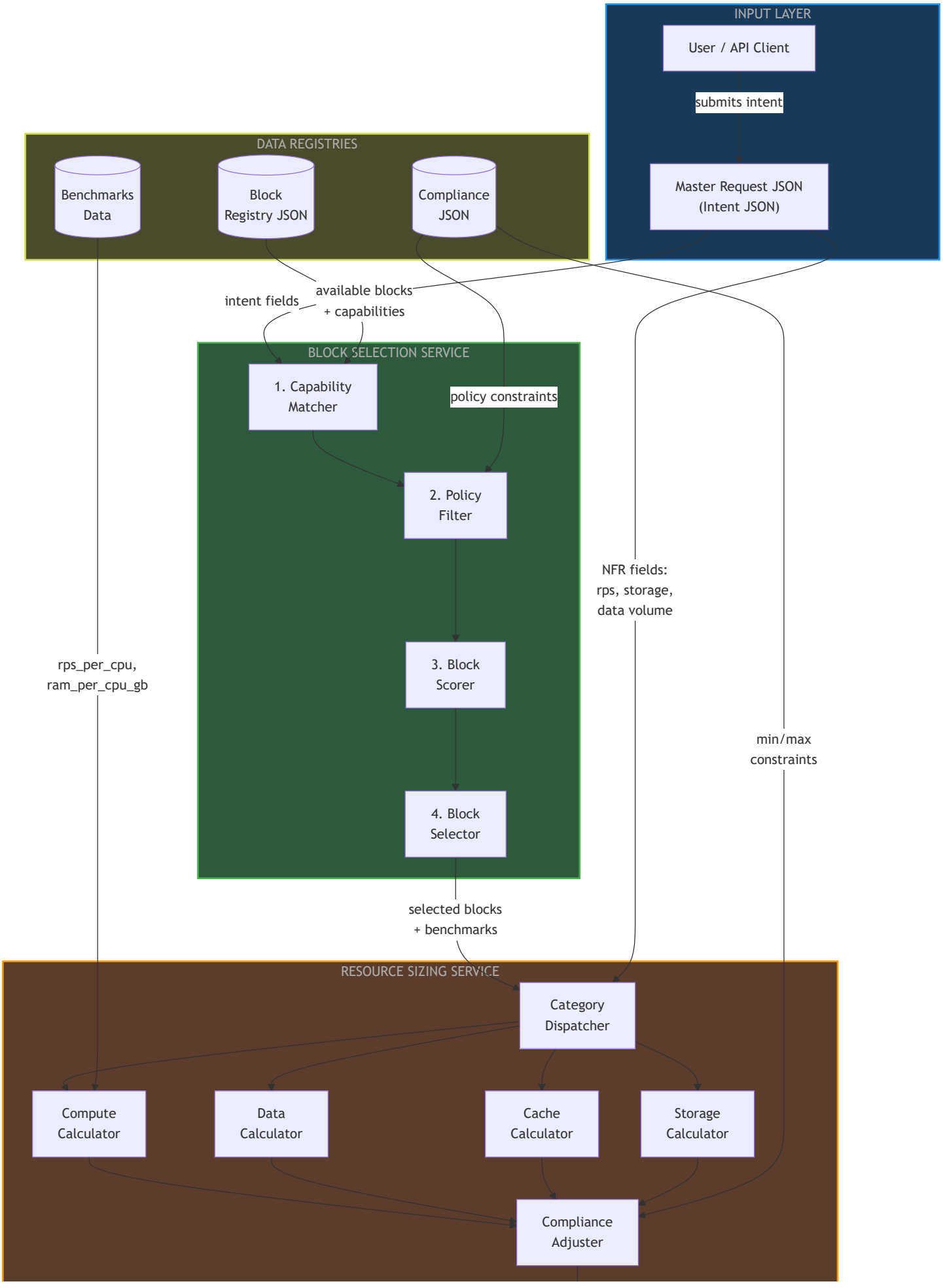

USERS
Developer / Architect
Admin / Platform Team
CONFIGURATOR SYSTEM
Config UI Pages
Intent Schema Builder
Block Registry Manager
Policy Rule Manager
submits
Governance
Audit Log
Approval Workflow
Version Rollback
Processing Engines
Formula Engine
Rule Engine
Impact Analyzer
INPUT / OUTPUT
Intent JSON (input)
CONFIGURATION DATA
AYER
Intent Schema v2.1
Block Registry (+mappings)
Formula Expressions
Compliance Rules
schema
RUNTIME PIPELINE
(EXISTING — NOW GENERIC)
Schema Validator
blocks + field maps
mappings
Block Selector
expressions
rules
Field Resolver
Formula Evaluator

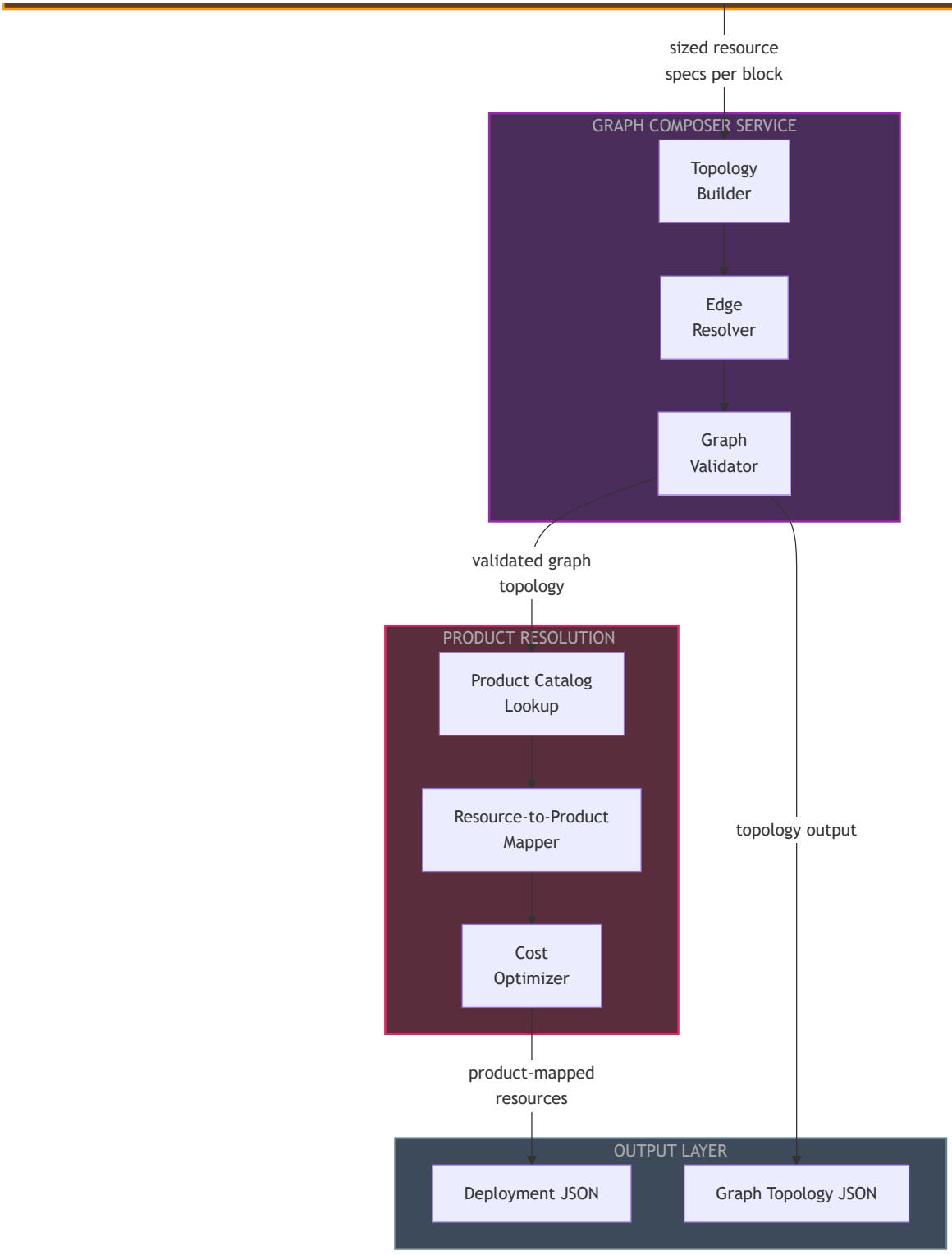


2. Existing Runtime Pipeline (Current)

This diagram represents the **current architecture** as documented in the Architecture Synthesis Plan. It shows the end-to-end flow from Intent JSON submission through Block Selection, Resource Sizing, Graph Composition, and Product Resolution to the final Deployment JSON output.

Key concern: Note how the Resource Sizing Service uses a Category Dispatcher with hardcoded if/elif branches, and each calculator reads specific intent fields by name. This is the rigid coupling that the Configurator Layer eliminates.


INPUT LAYER
User / API Client
submits intent
Master Request JSON
(Intent JSON)
DATA REGISTRIES
Benchmarks
Data
Block
Registry JSON
Compliance
JSON
intent fields
available blocks
+ capabilities
BLOCK SELECTION SERVICE
1. Capability
Matcher
policy constraints
2. Policy
Filter
3. Block
Scorer
4. Block
Selector
NFR fields:
rps, storage,
data volume
rps_per_cpu,
ram_per_cpu_gb
min/max
constraints
selected blocks
+ benchmarks
RESOURCE SIZING SERVICE
Category
Dispatcher
Compute
Calculator
Data
Calculator
Cache
Calculator
Storage
Calculator
Compliance
Adjuster



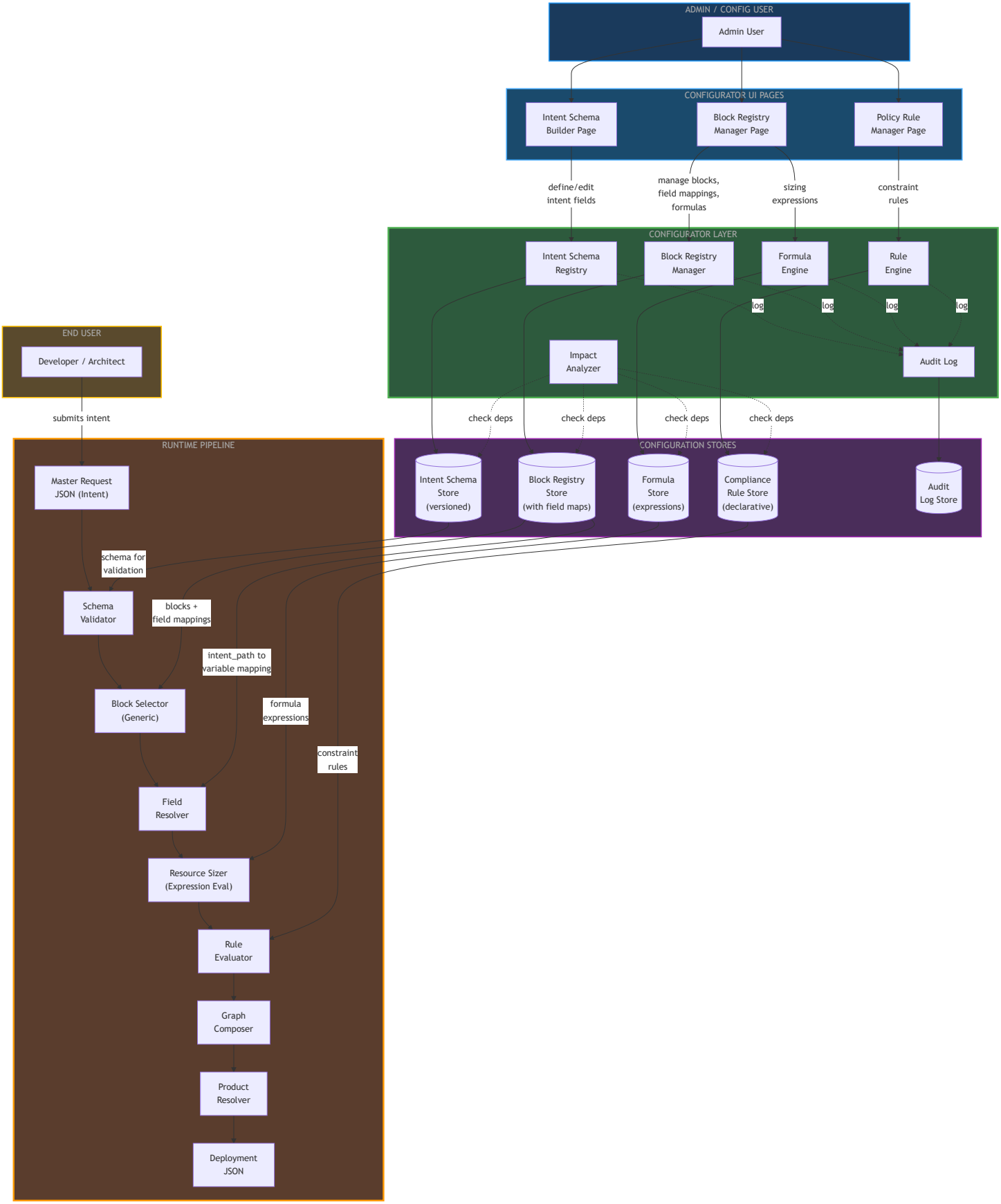
3. Complete System with Configurator Layer

This diagram shows **both flows** operating together:

- **Admin Flow** (left): Admin configures schemas, blocks, formulas, and rules through UI pages. Changes go through the Configurator Layer into Configuration Stores with audit logging.

- **Runtime Flow** (right): Developer submits intent. The runtime pipeline reads all logic from Configuration Stores — schema validation, block selection, field resolution, formula evaluation, and rule application are all config-driven.

The **Impact Analyzer** (dashed lines) continuously validates cross-store dependencies.


ADMIN / CONFIG USER
Admin User
CONFIGURATOR UI PAGES
Intent Schema Builder Page
Block Registry Manager Page
Policy Rule Manager Page
define/edit intent fields
manage blocks, field mappings, formulas
sizing expressions
constraint rules
CONFIGURATOR LAYER
Intent Schema Registry
Block Registry Manager
Formula Engine
Rule Engine
log
log
log
log
Impact Analyzer
Audit Log
END USER
Developer / Architect
submits intent
check deps
check deps
check deps
check deps
CONFIGURATION STORES
Intent Schema Store (versioned)
Block Registry Store (with field maps)
Formula Store (expressions)
Compliance Rule Store (declarative)
Audit Log Store
RUNTIME PIPELINE
Master Request JSON (Intent)
schema for validation
Schema Validator
blocks + field mappings
intent_path to variable mapping
Block Selector (Generic)
formula expressions
constraint rules
Field Resolver
Resource Sizer (Expression Eval)
Rule Evaluator
Graph Composer
Product Resolver
Deployment JSON

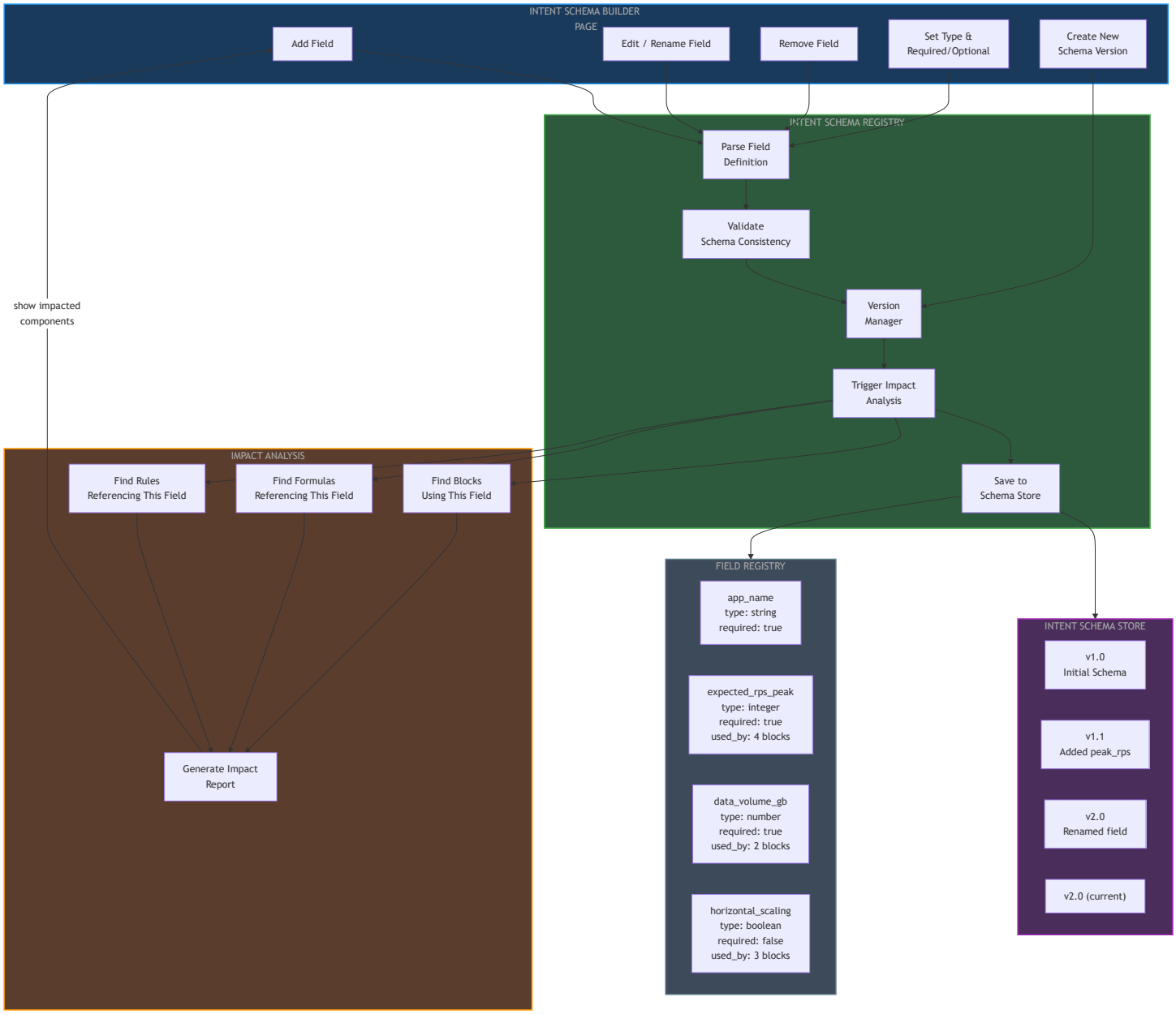
4. Intent Schema Registry — Internal Flow

The Intent Schema Registry is the **single source of truth** for what fields exist in the Intent JSON, their types, and which blocks/formulas consume them.

When a field is added, edited, renamed, or removed:

1. The change is parsed and validated for schema consistency
2. A new schema version is created
3. **Impact Analysis is triggered automatically** — scanning all blocks, formulas, and rules that reference the changed field
4. The admin sees an impact report before the change is saved

The **Field Registry** tracks per-field metadata including `used_by` count, showing how many blocks depend on each field.



5. Block Registry Manager — Field Mapping Flow

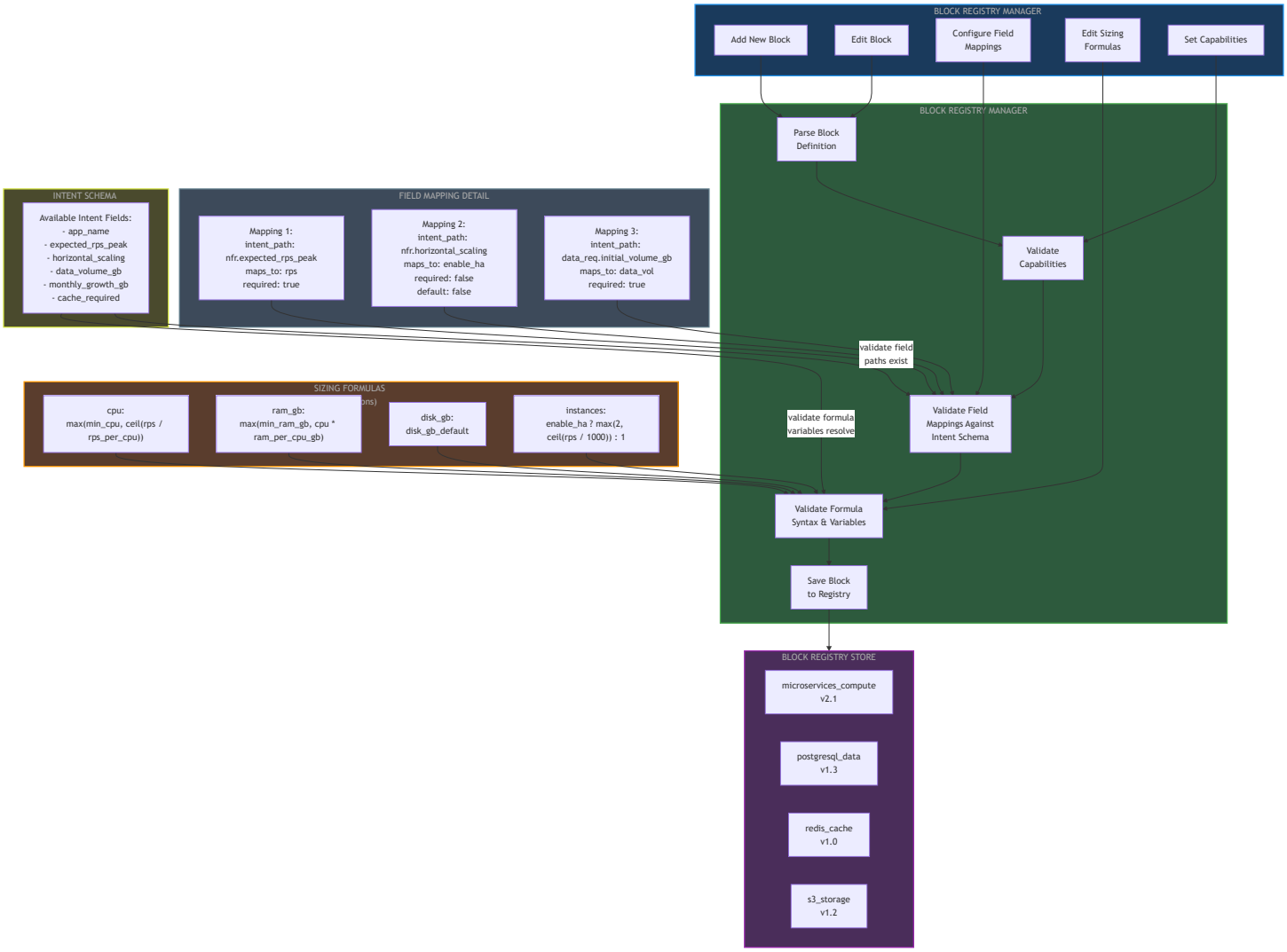
The Block Registry Manager is the **core configurator page**. Each block now explicitly declares:

- **Capabilities** it provides (same as before)
- **Field Mappings** — which intent fields it consumes, mapped to local variable names
- **Sizing Formulas** — stored as expression strings, not hardcoded Python

Validation is cross-referenced against the Intent Schema. If a block mapping references `nfr.expected_rps_peak`, the system validates that this field actually exists in the current Intent

Schema. If a formula uses variable `rps`, the system validates that a field mapping produces this variable.

This is the **key innovation**: blocks become self-describing. The runtime engine doesn't need to know about specific blocks — it just reads their declared mappings and evaluates their declared formulas.



6. Formula Engine & Rule Engine — Runtime Evaluation

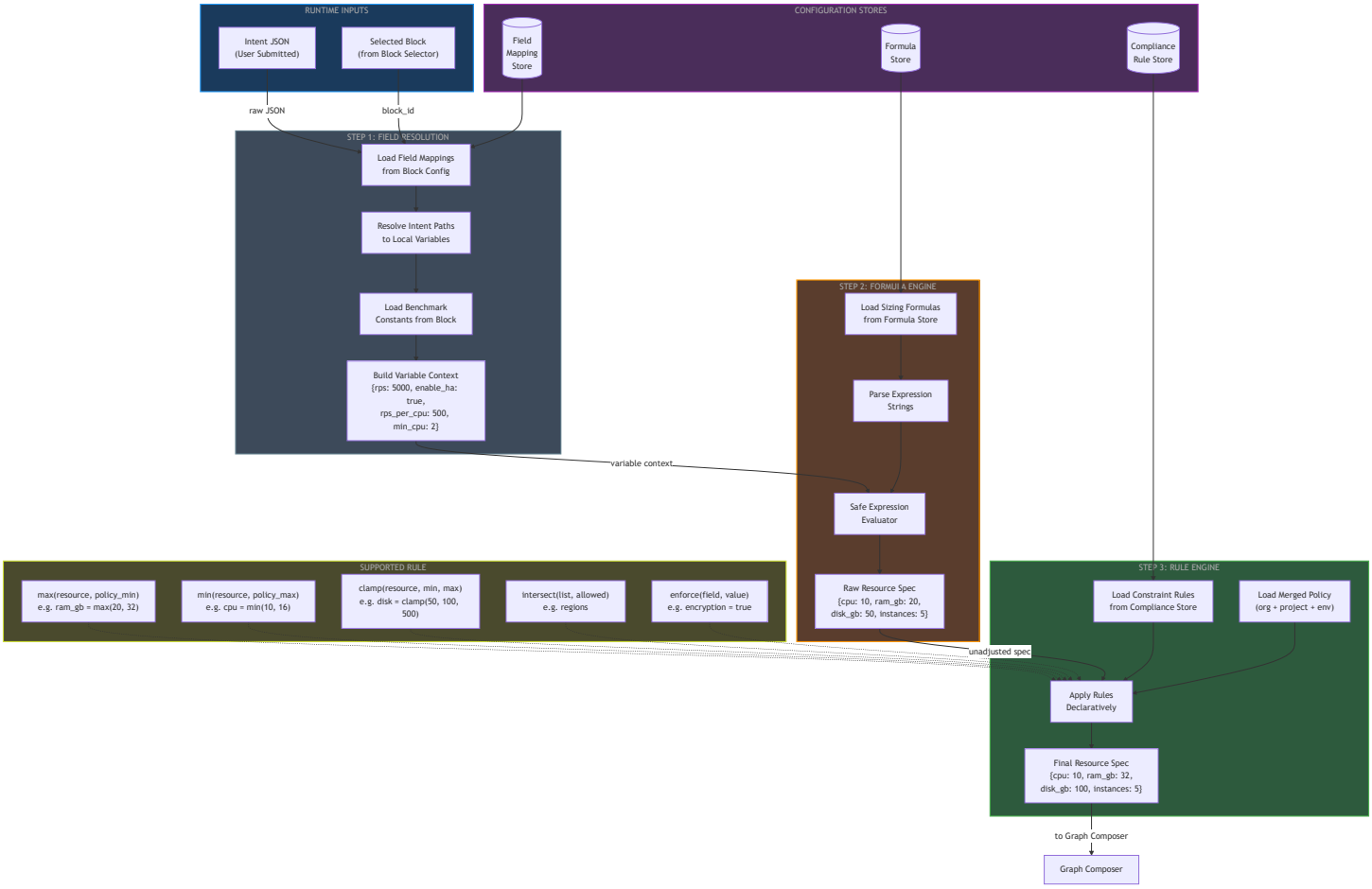
This diagram shows the **three-step runtime evaluation** that replaces the hardcoded Category Dispatcher:

Step 1 — Field Resolution: Load the block's field mappings, resolve intent JSON paths to local variables, and combine with benchmark constants to build a complete variable context.

Step 2 — Formula Engine: Load sizing formulas (stored as expression strings like `"max(min_cpu, ceil(rps / rps_per_cpu))"`), parse and evaluate them safely against the variable context to produce a raw resource spec.

Step 3 — Rule Engine: Load compliance constraint rules (declarative, not hardcoded), load the merged policy (org + project + env), and apply rules to produce the final adjusted resource spec.

Five supported rule operations: `max`, `min`, `clamp`, `intersect`, `enforce` — all declared in config, not code.



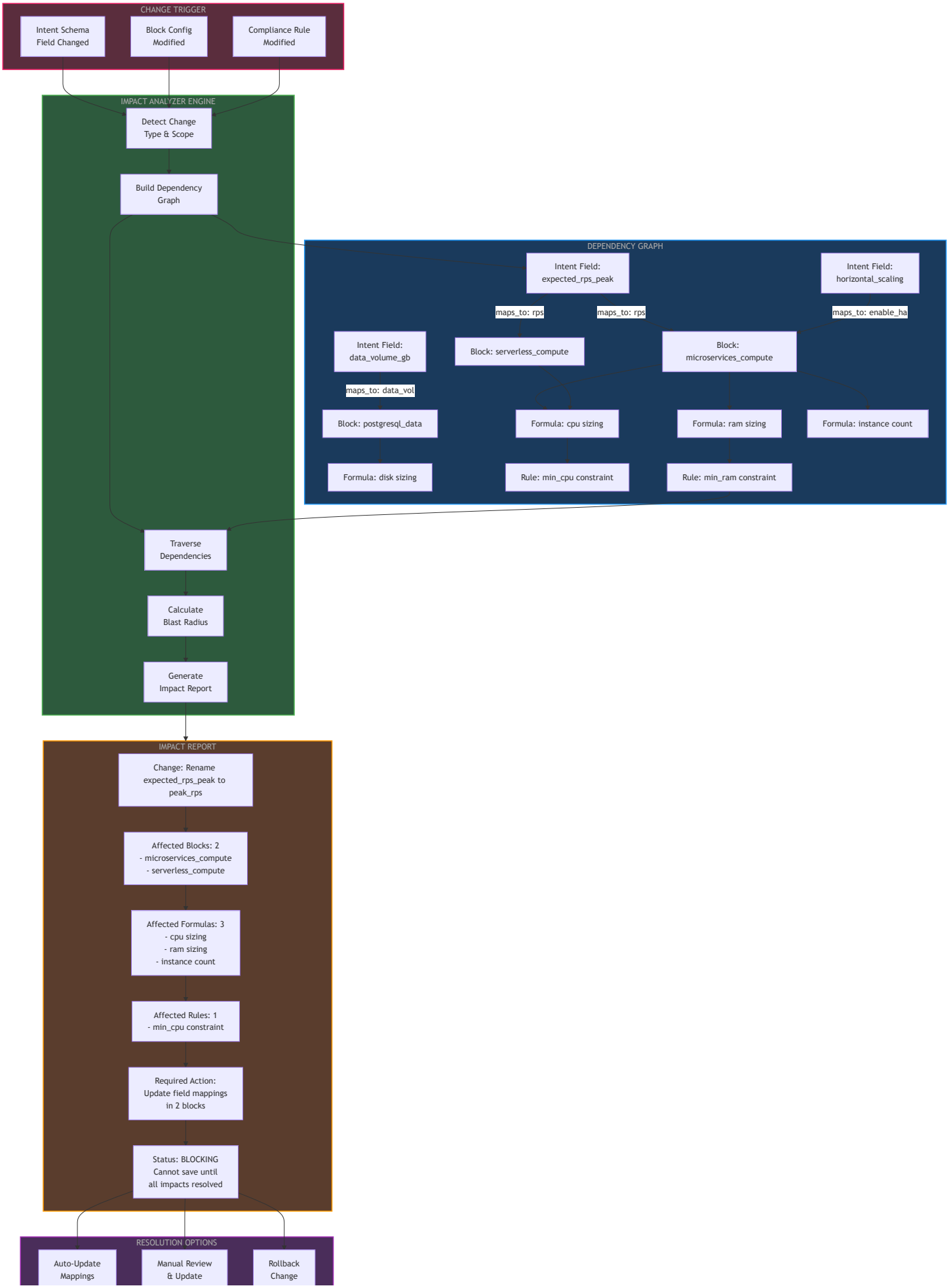
7. Impact Analyzer — Dependency Graph & Blast Radius

The Impact Analyzer is the **safety net** that prevents breaking changes. When any configuration entity changes (Intent Schema field, Block config, or Compliance rule), the analyzer:

1. **Detects** the change type and scope

2. **Builds a dependency graph** mapping: `Intent Field → Blocks → Formulas → Rules`
3. **Traverses** the graph to find all impacted downstream components
4. **Calculates blast radius** — how many components are affected
5. **Generates an impact report** with specific actions required

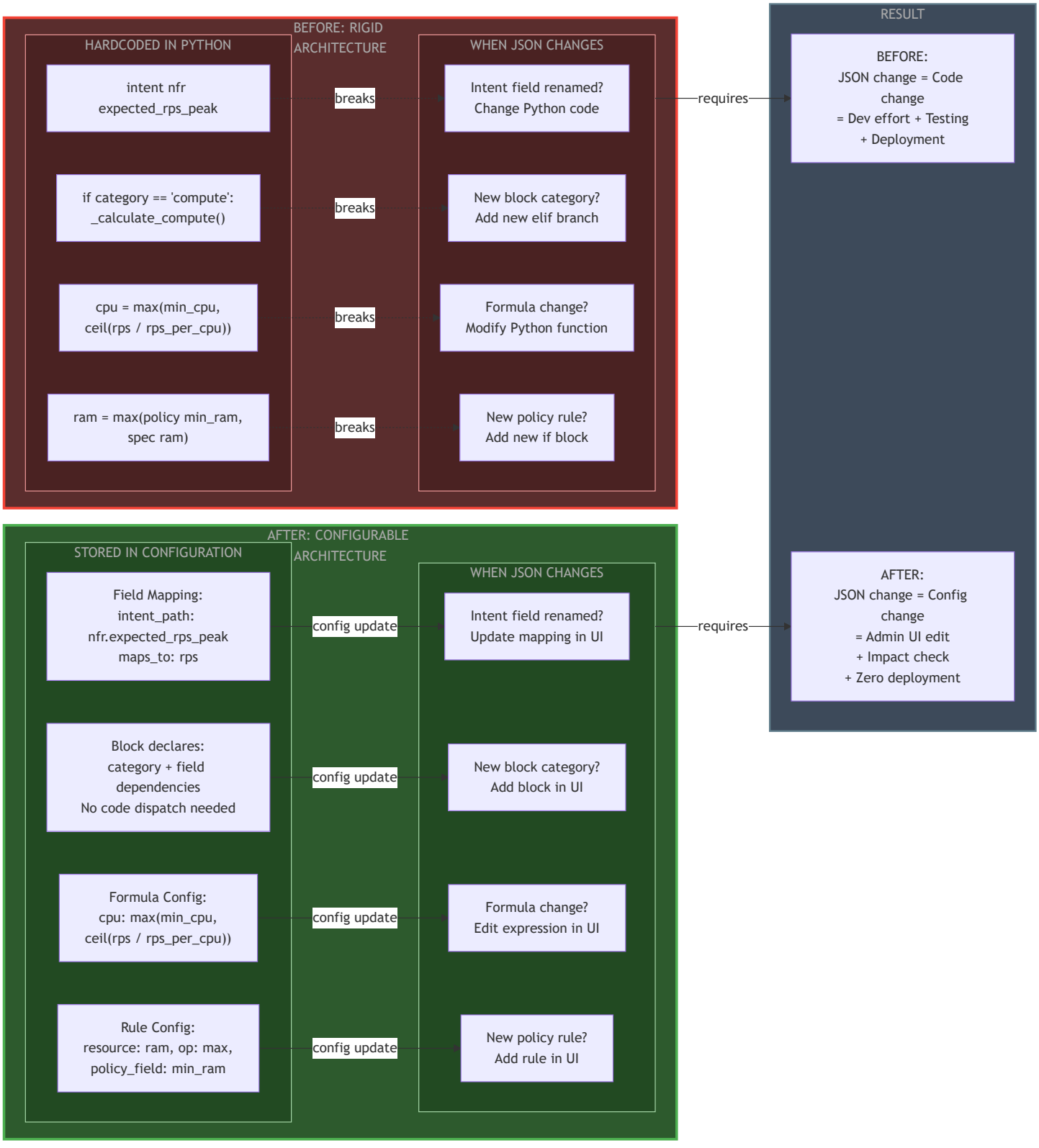
The change **cannot be saved** until all impacts are resolved. Three resolution options: Auto-Update (propagate rename), Manual Review, or Rollback.


CHANGE TRIGGER
Intent Schema Field Changed
Block Config Modified
Compliance Rule Modified
IMPACT ANALYZER ENGINE
Detect Change Type & Scope
Build Dependency Graph
DEPENDENCY GRAPH
Intent Field: expected_rps_peak
Intent Field: horizontal_scaling
maps_to: rps
maps_to: rps
maps_to: enable_ha
Intent Field: data_volume_gb
Block: serverless_compute
Block: microservices_compute
maps_to: data_vol
Block: postgresql_data
Formula: cpu sizing
Formula: ram sizing
Formula: instance count
Formula: disk sizing
Rule: min_cpu constraint
Rule: min_ram constraint
Traverse Dependencies
Calculate Blast Radius
Generate Impact Report
IMPACT REPORT
Change: Rename expected_rps_peak to peak_rps
Affected Blocks: 2
- microservices_compute
- serverless_compute
Affected Formulas: 3
- cpu sizing
- ram sizing
- instance count
Affected Rules: 1
- min_cpu constraint
Required Action: Update field mappings in 2 blocks
Status: BLOCKING
Cannot save until all impacts resolved
RESOLUTION OPTIONS
Auto-Update Mappings
Manual Review & Update
Rollback Change

8. Before vs After Configurator — Rigid vs Flexible

Side-by-side comparison showing how the same four concerns (field access, category dispatch, sizing formulas, policy rules) are handled in the current rigid architecture versus the proposed configurable architecture.

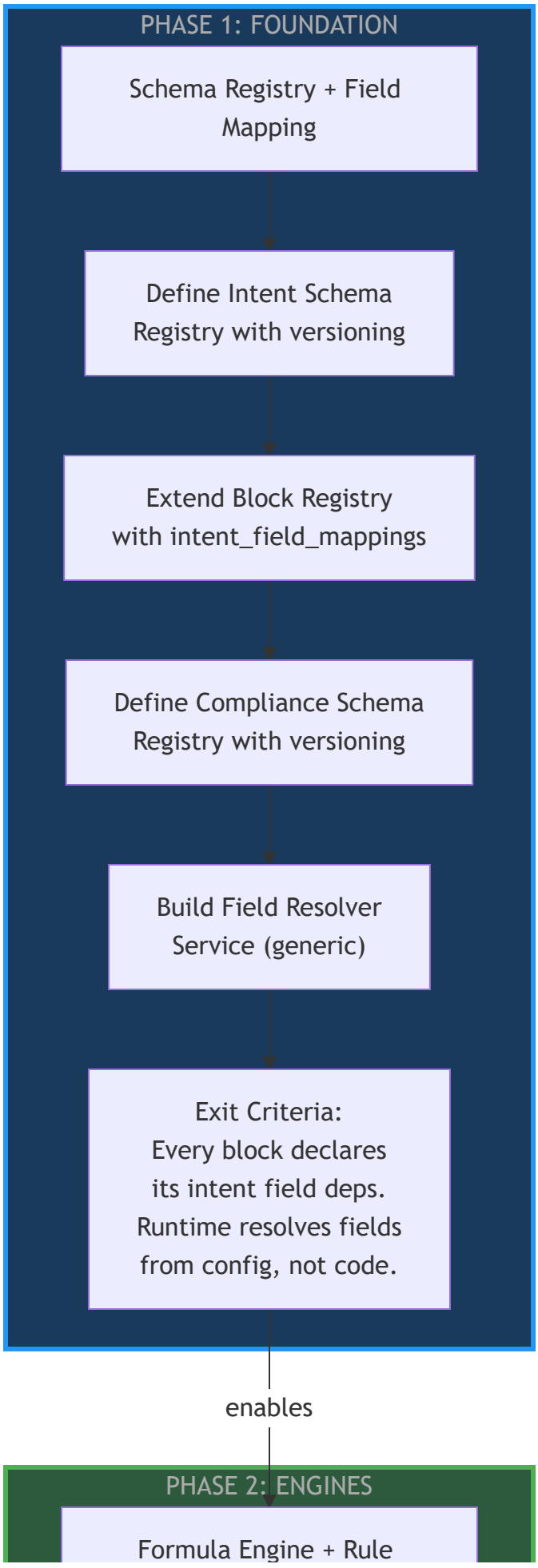
BEFORE: Every JSON change requires Python code change → dev effort → testing → deployment.
AFTER: Every JSON change requires config update via UI → impact check → zero deployment.

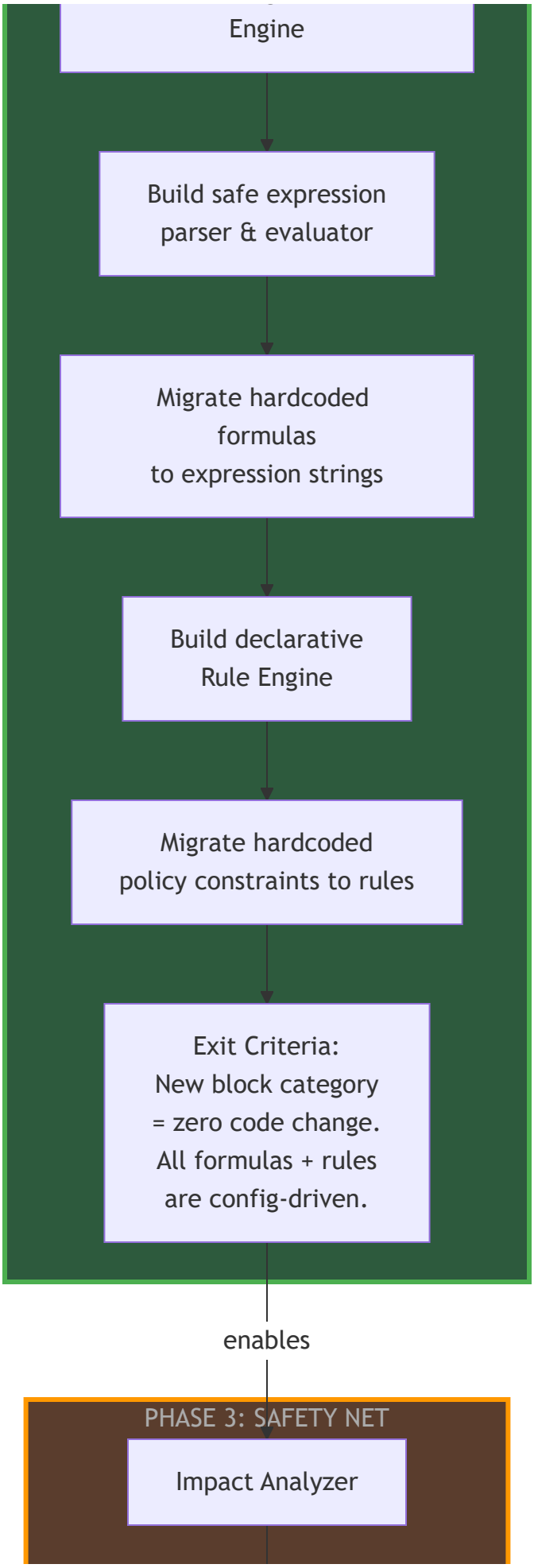


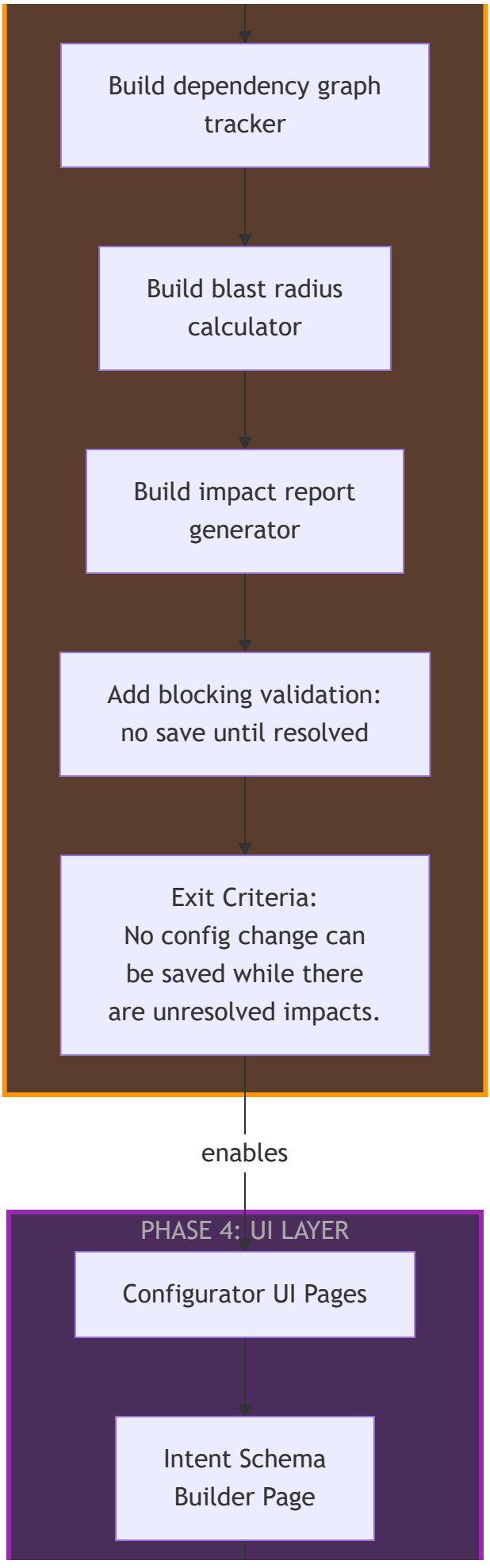
9. Phase-wise Implementation Sequence

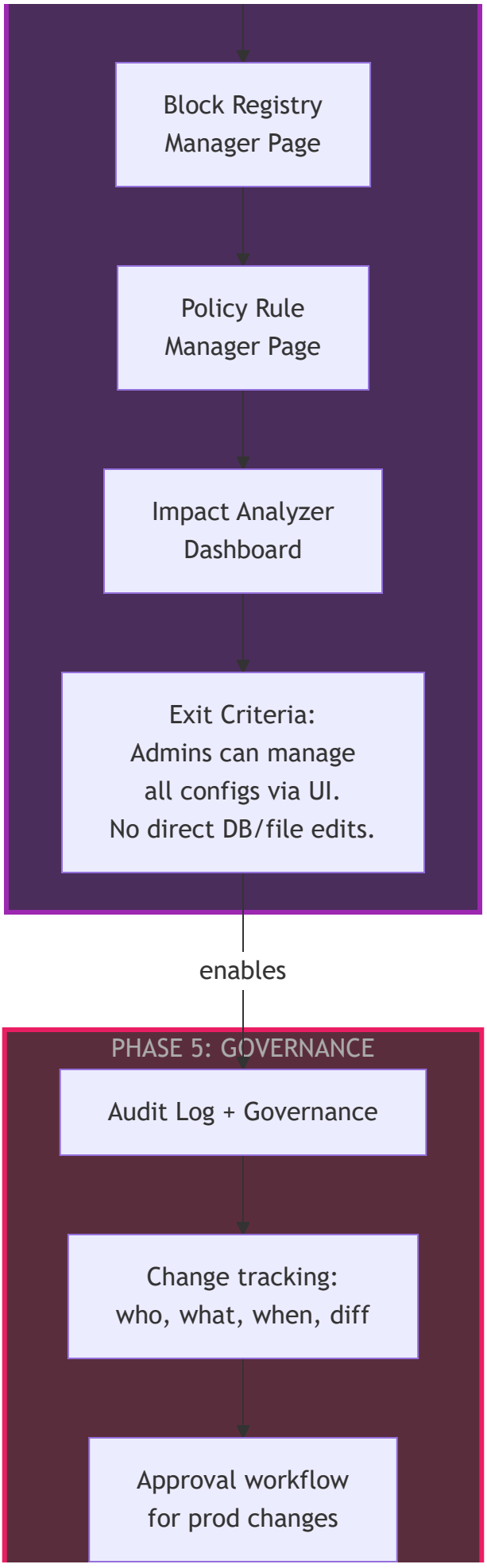
Five phases, each with clear exit criteria. Each phase enables the next. No phase can be skipped.

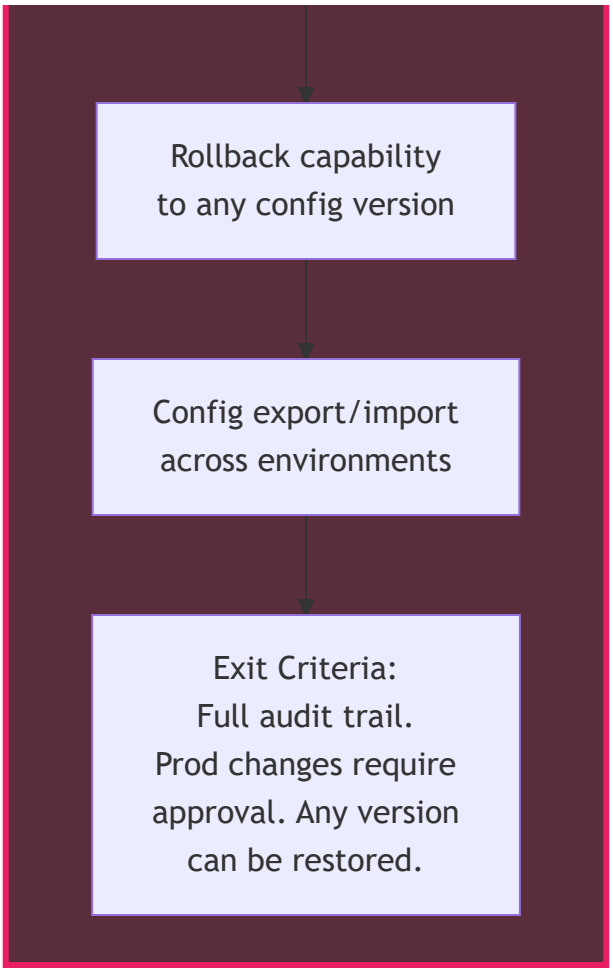
- **Phase 1 (Foundation):** Schema Registry + Field Mappings — blocks declare intent field dependencies
- **Phase 2 (Engines):** Formula Engine + Rule Engine — expression-based, not hardcoded
- **Phase 3 (Safety Net):** Impact Analyzer — dependency graph + blast radius + blocking validation
- **Phase 4 (UI Layer):** Three Configurator UI pages + Impact Dashboard
- **Phase 5 (Governance):** Audit log, approval workflow, rollback, config export/import


PHASE 1: FOUNDATION
Schema Registry + Field Mapping
Define Intent Schema Registry with versioning
Extend Block Registry with intent_field_mappings
Define Compliance Schema Registry with versioning
Build Field Resolver Service (generic)
Exit Criteria: Every block declares its intent field deps. Runtime resolves fields from config, not code.
enables
PHASE 2: ENGINES
Formula Engine + Rule


Engine
Build safe expression parser & evaluator
Migrate hardcoded formulas to expression strings
Build declarative Rule Engine
Migrate hardcoded policy constraints to rules
Exit Criteria: New block category = zero code change. All formulas + rules are config-driven.
enables
PHASE 3: SAFETY NET
Impact Analyzer


Build dependency graph tracker
Build blast radius calculator
Build impact report generator
Add blocking validation: no save until resolved
Exit Criteria: No config change can be saved while there are unresolved impacts.
enables
PHASE 4: UI LAYER
Configurator UI Pages
Intent Schema Builder Page


Block Registry Manager Page
Policy Rule Manager Page
Impact Analyzer Dashboard
Exit Criteria: Admins can manage all configs via UI. No direct DB/file edits.
enables
PHASE 5: GOVERNANCE
Audit Log + Governance
Change tracking: who, what, when, diff
Approval workflow for prod changes

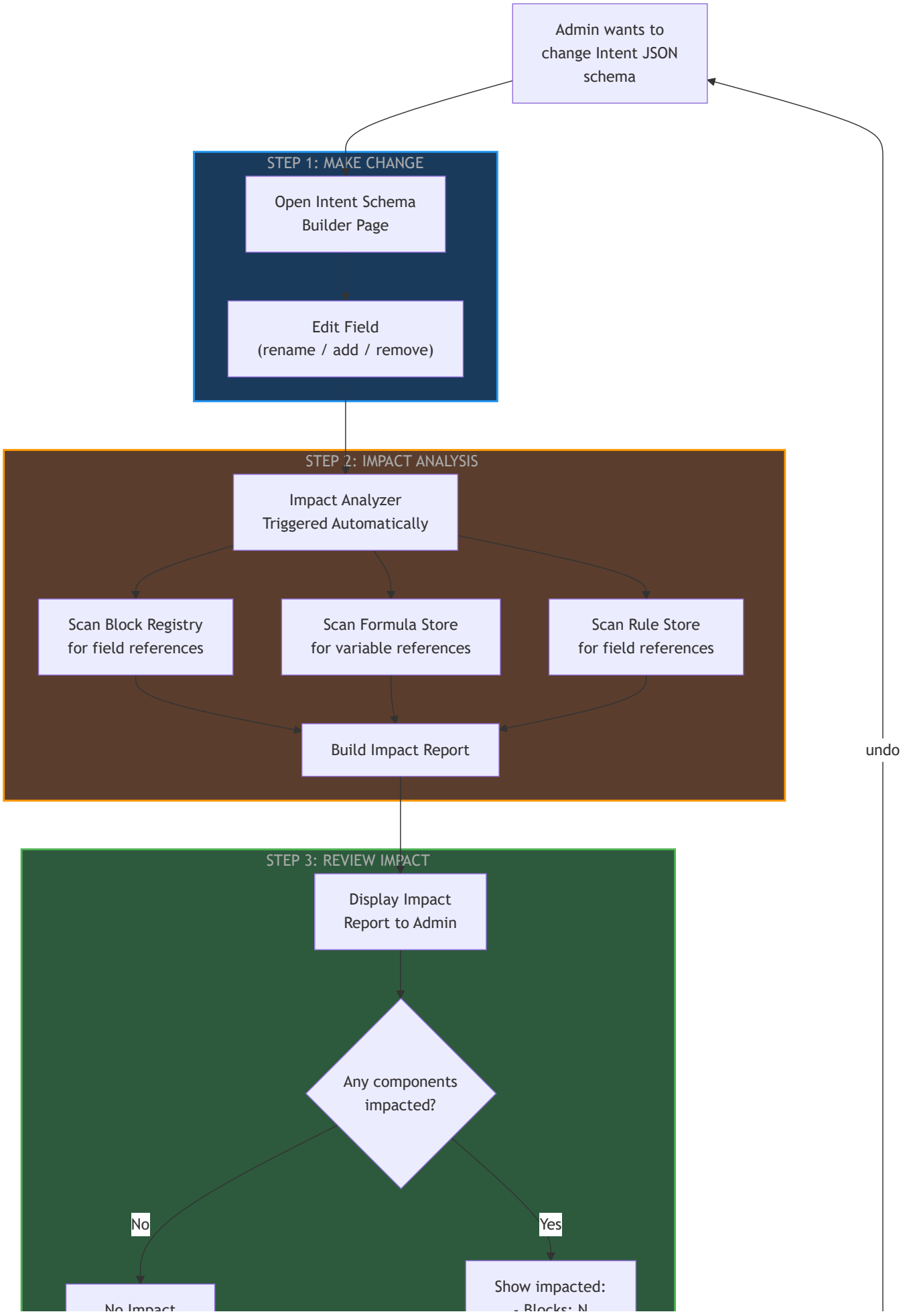


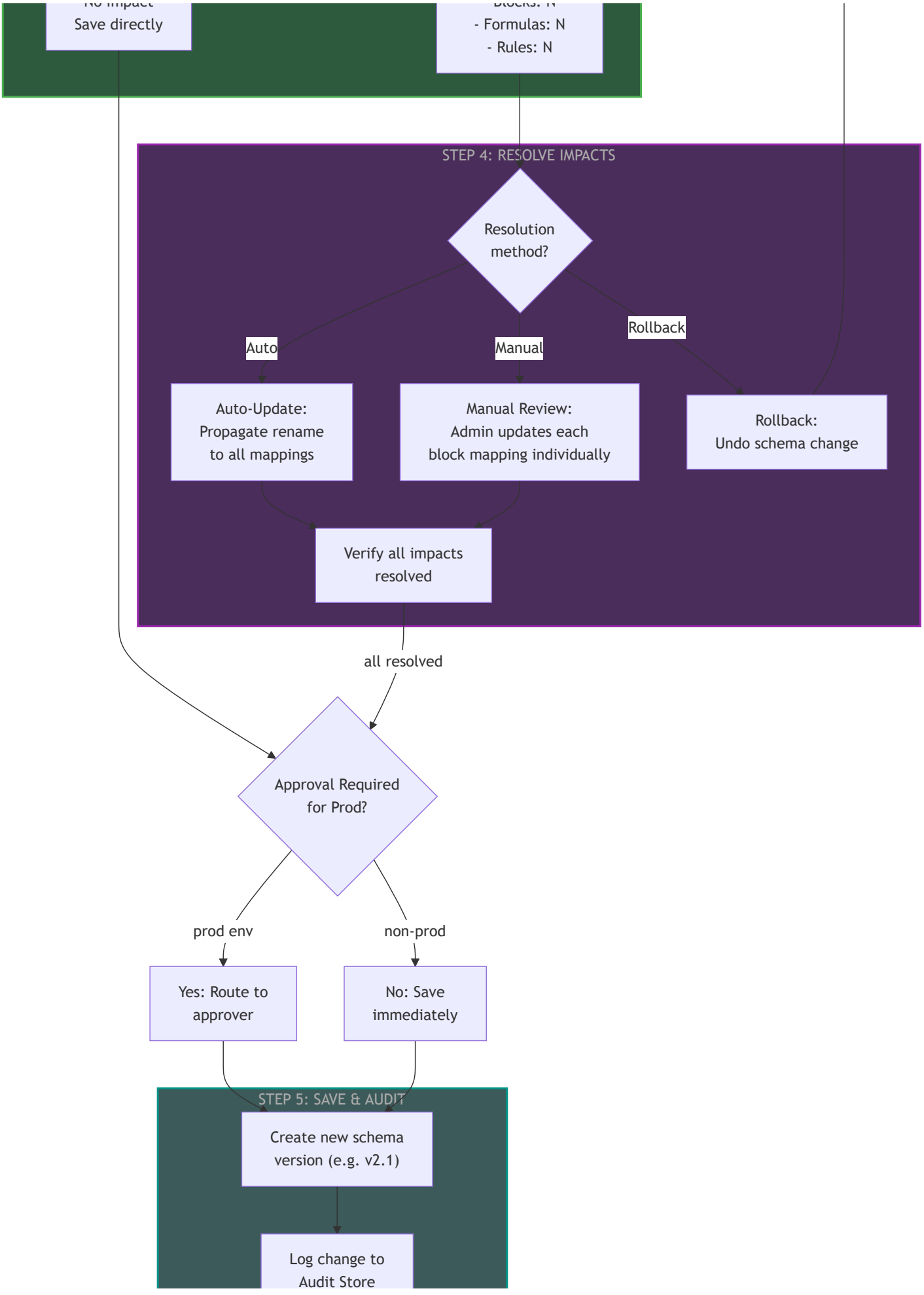
10. JSON Schema Change Workflow

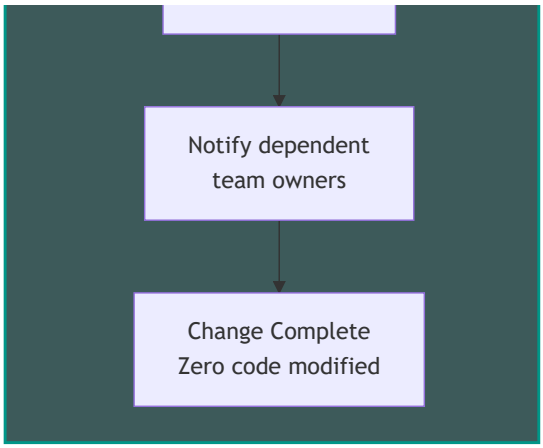
End-to-end workflow showing exactly what happens when an admin wants to change the Intent JSON schema. This is the **zero-code-change workflow** in action.

Key decision points:

- **Impact check is automatic** — triggered the moment a field is edited
- **Blocking validation** — change cannot be saved if impacts exist
- **Three resolution paths** — Auto-Update, Manual Review, or Rollback
- **Prod requires approval** — non-prod environments can save immediately
- **Audit trail** — every change is logged with before/after snapshot


Admin wants to change Intent JSON schema
STEP 1: MAKE CHANGE
Open Intent Schema Builder Page
Edit Field (rename / add / remove)
STEP 2: IMPACT ANALYSIS
Impact Analyzer Triggered Automatically
Scan Block Registry for field references
Scan Formula Store for variable references
Scan Rule Store for field references
Build Impact Report
STEP 3: REVIEW IMPACT
Display Impact Report to Admin
Any components impacted?
No
Yes
Show impacted:
undo


Save directly
- Formulas: N
- Rules: N
STEP 4: RESOLVE IMPACTS
Resolution method?
Auto
Manual
Rollback
Auto-Update: Propagate rename to all mappings
Manual Review: Admin updates each block mapping individually
Rollback: Undo schema change
Verify all impacts resolved
all resolved
Approval Required for Prod?
prod env
non-prod
Yes: Route to approver
No: Save immediately
STEP 5: SAVE & AUDIT
Create new schema version (e.g. v2.1)
Log change to Audit Store



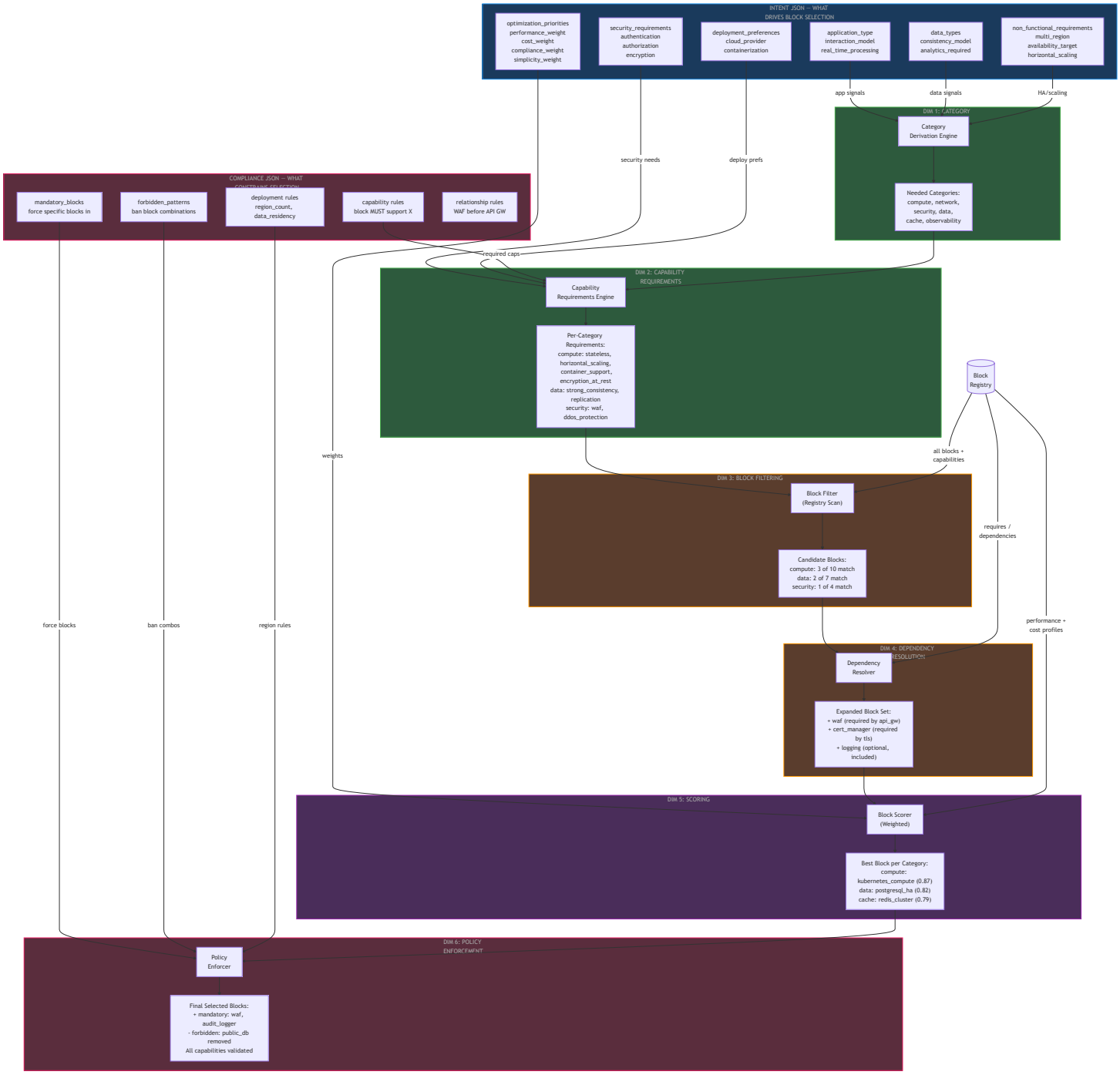
11. Intent to Block Selection — Complete Mapping (6 Dimensions)

Block selection is **NOT** just capability matching. It is a **6-dimensional process** where different sections of the Intent JSON and Compliance JSON drive different selection stages. This section makes the full mapping intuitive and traceable.

The 6 Dimensions of Block Selection

Dimension	Input Source	What It Does	Example
1. Category Derivation	Intent: `application_type`, `data_types`, `real_time_processing`, `audit_logging`	Determines WHICH block categories are needed	`application_type: web_app` → need compute, network, security categories
2. Capability Requirements	Intent: `security_requirements`, `deployment_preferences` + Policy: `capability` rules	Determines WHAT capabilities each block must have	`encryption.at_rest: true` → compute block must have `data_encryption_at_rest`
3. Block Filtering	Block Registry: `capabilities_provided`	Filters blocks in each category to those with ALL required capabilities	10 compute blocks → 3 match all required capabilities

Dimension	Input Source	What It Does	Example
4. Dependency Resolution	Block Registry: `requires`, `optional_dependencies`	Selected blocks pull in additional required blocks	`api_gateway` requires `waf` → waf block auto-included
5. Scoring & Ranking	Intent: `optimization_priorities` (performance, cost, compliance, simplicity weights)	Ranks filtered blocks to pick the best in each category	cost_weight: 0.6 → cheaper block wins over faster one
6. Policy Enforcement	Policy: `mandatory_blocks`, `forbidden_patterns`, `capability` rules	Overrides: force blocks in, ban combinations, enforce capabilities	Policy mandates `waf` block regardless of intent



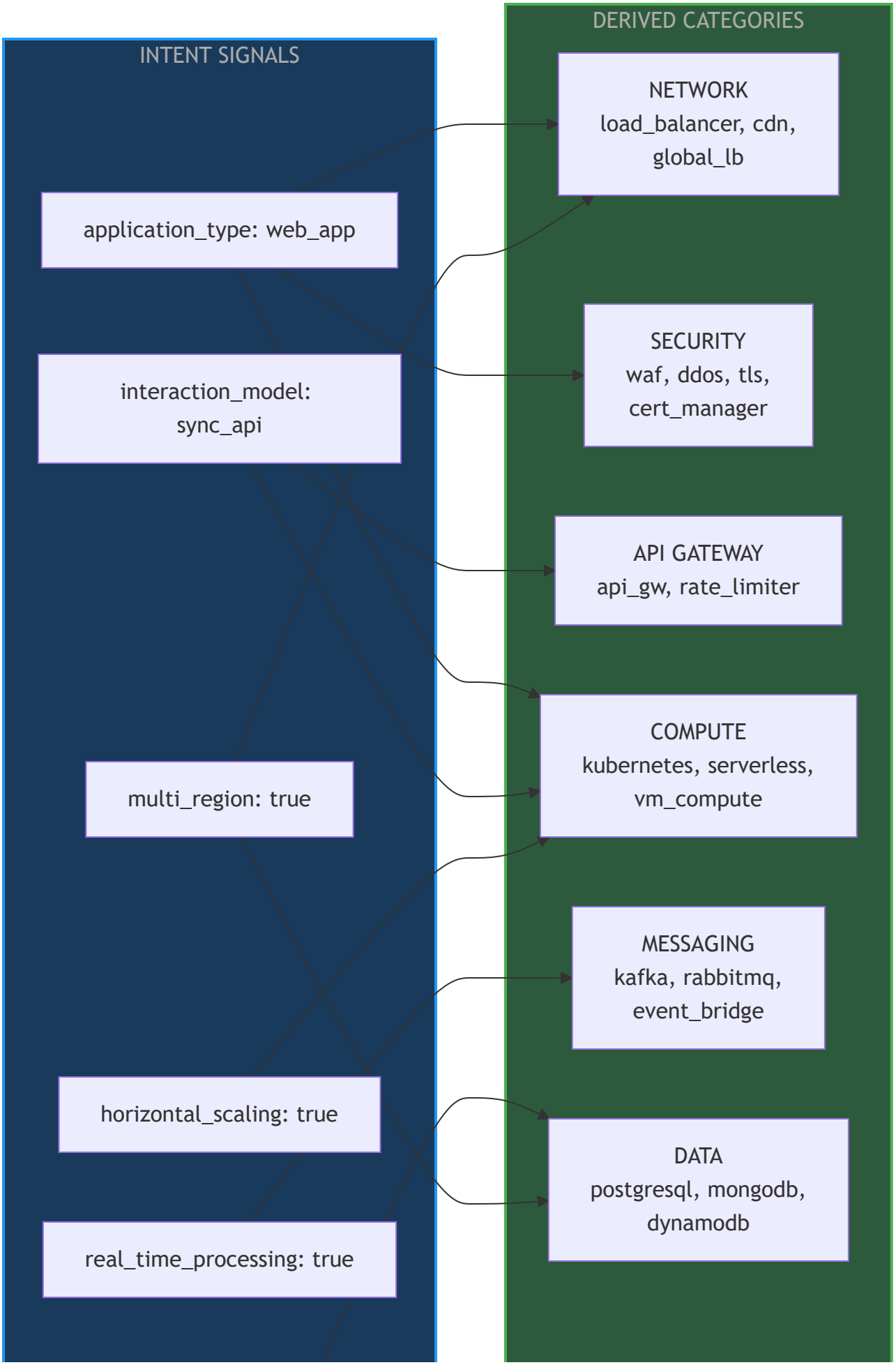
Intent Field → Category Derivation: Detailed Mapping

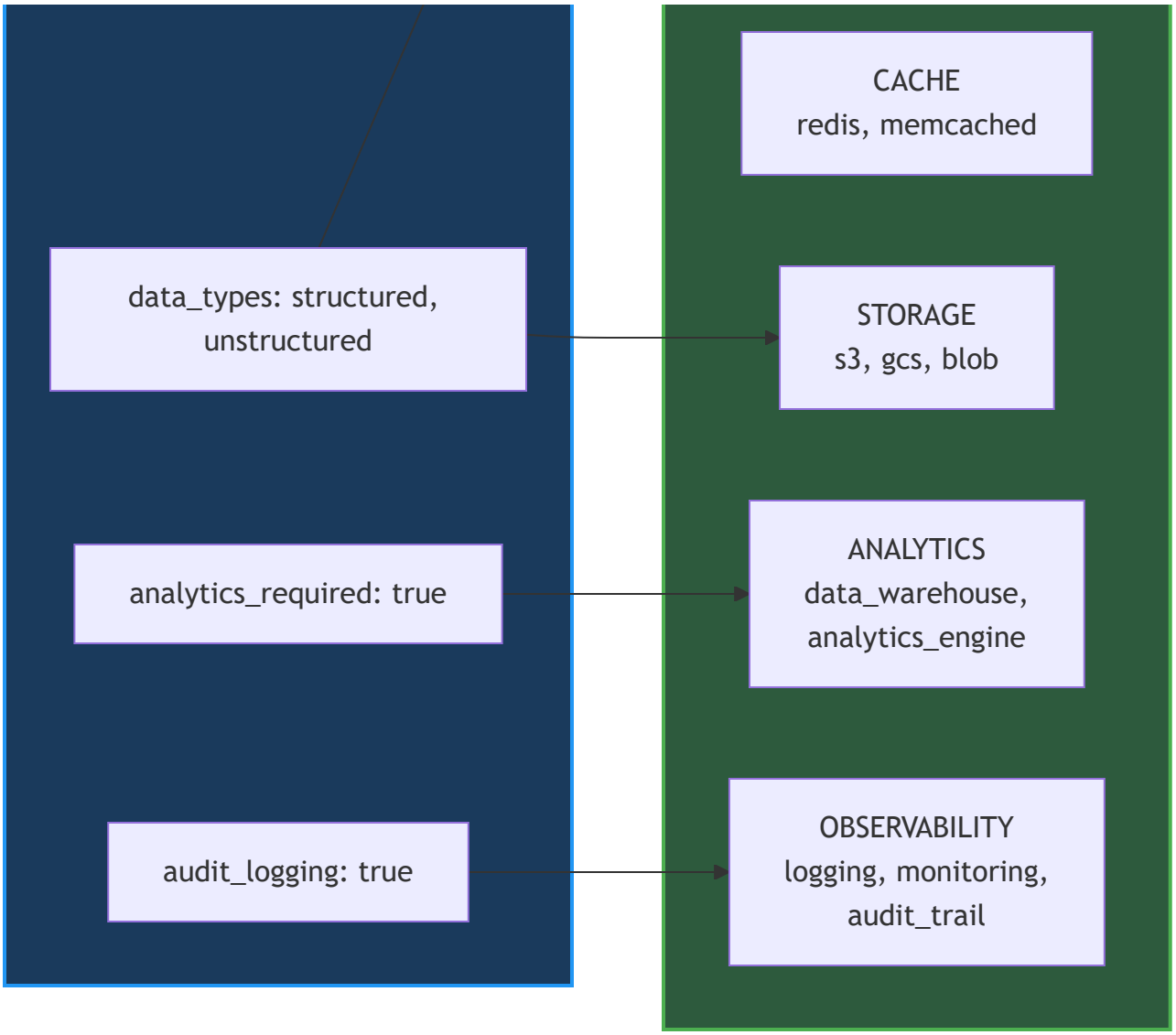
This table shows exactly which intent fields trigger which block categories to be included:

Intent Field	Value / Condition	Category Derived	Reasoning
`application_type`	`web_app` , `api_service`	compute, network, security	Web apps need servers, networking,

Intent Field	Value / Condition	Category Derived	Reasoning
			and protection
`application_type`	`ml_pipeline`	compute, data, storage, ml_ops	ML needs GPU compute, training data, model storage
`interaction_model`	`synchronous_api`	compute, network, api_gateway	API needs request handling + gateway
`interaction_model`	`event_driven`	messaging, compute	Events need message broker + processors
`real_time_processing`	`true`	messaging, streaming	Need stream processors (Kafka, etc.)
`data_types` includes	`structured`	data (SQL)	Relational database needed
`data_types` includes	`unstructured`	data (NoSQL), storage	Document DB + object storage
`data_types` includes	`time_series`	data (time-series)	Specialized time-series DB
`analytics_required`	`true`	analytics, data_warehouse	Analytics engine + warehouse

Intent Field	Value / Condition	Category Derived	Reasoning
`audit_logging_required`	`true`	observability	Logging + audit trail blocks
`multi_region_required`	`true`	network (global LB), data (replication)	Global load balancer + data replication
`horizontal_scaling_required`	`true`	compute (auto-scaling)	Blocks must support scaling
Policy: `mandatory_blocks`	`["waf", "audit_logger"]`	security, observability	Forced by policy regardless of intent


INTENT SIGNALS
application_type: web_app
interaction_model: sync_api
multi_region: true
horizontal_scaling: true
real_time_processing: true
DERIVED CATEGORIES
NETWORK
load_balancer, cdn, global_lb
SECURITY
waf, ddos, tls, cert_manager
API GATEWAY
api_gw, rate_limiter
COMPUTE
kubernetes, serverless, vm_compute
MESSAGING
kafka, rabbitmq, event_bridge
DATA
postgresql, mongodb, dynamodb

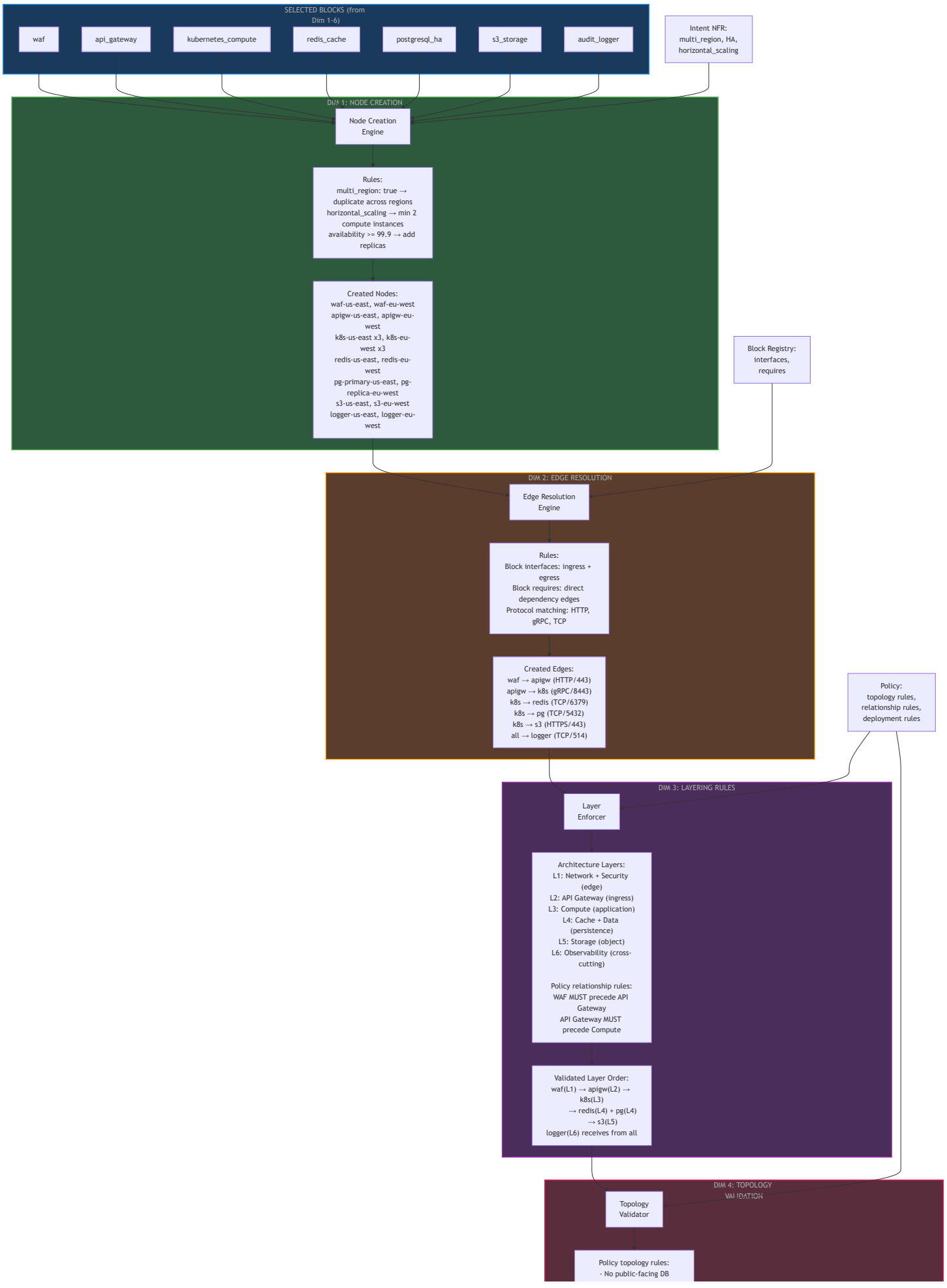


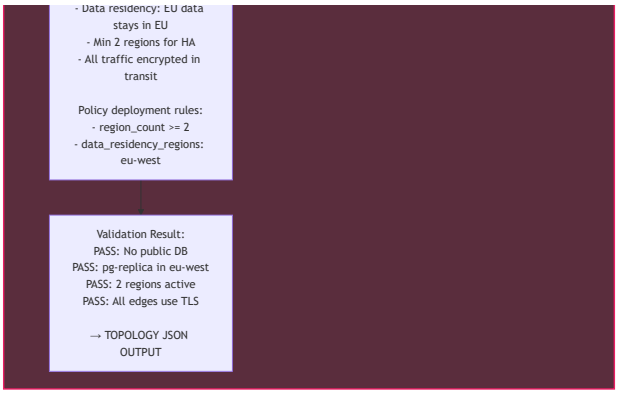
12. Selected Blocks to Topology JSON — Complete Mapping (4 Dimensions)

Once blocks are selected, the **Graph Composer** builds the topology. This is also NOT a simple "connect everything" — it's a **4-dimensional process**:

Dimension	Input Source	What It Does
1. Node Creation	Selected blocks + Intent: `multi_region`, `horizontal_scaling`, `availability_target`	Determines HOW MANY instances of each block and in which regions
2. Edge Resolution	Block Registry: `interfaces` (ingress/egress) + Block: `requires`	Determines WHICH blocks connect to which, using which

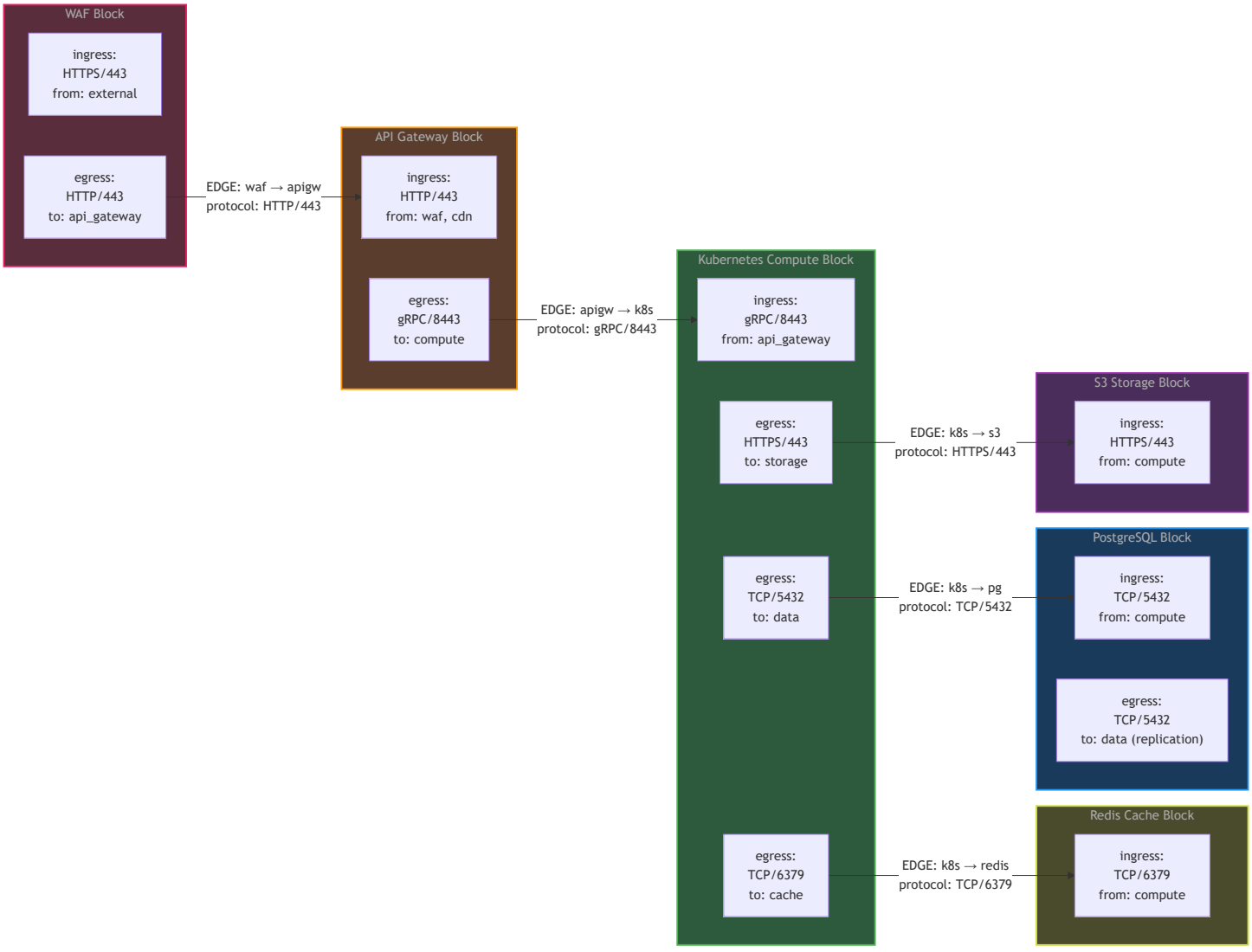
Dimension	Input Source	What It Does
		protocol
3. Layering Rules	Architecture conventions + Policy: `relationship` rules	Enforces ordering: network → security → api_gw → compute → data
4. Topology Validation	Policy: `topology` rules + `deployment` rules	Validates: no public DB, data residency, minimum region count


SELECTED BLOCKS (from Dim 1-6)
waf
api_gateway
kubernetes_compute
redis_cache
postgresql_ha
s3_storage
audit_logger
Intent NFR: multi_region, HA, horizontal_scaling
DIM 1: NODE CREATION
Node Creation Engine
Rules:
multi_region: true → duplicate across regions
horizontal_scaling → min 2 compute instances
availability >= 99.9 → add replicas
Created Nodes:
waf-us-east, waf-eu-west
apigw-us-east, apigw-eu-west
k8s-us-east x3, k8s-eu-west x3
redis-us-east, redis-eu-west
pg-primary-us-east, pg-replica-eu-west
s3-us-east, s3-eu-west
logger-us-east, logger-eu-west
Block Registry: interfaces, requires
DIM 2: EDGE RESOLUTION
Edge Resolution Engine
Rules:
Block interfaces: ingress + egress
Block requires: direct dependency edges
Protocol matching: HTTP, gRPC, TCP
Created Edges:
waf → apigw (HTTP/443)
apigw → k8s (gRPC/8443)
k8s → redis (TCP/6379)
k8s → pg (TCP/5432)
k8s → s3 (HTTPS/443)
all → logger (TCP/514)
Policy: topology rules, relationship rules, deployment rules
DIM 3: LAYERING RULES
Layer Enforcer
Architecture Layers:
L1: Network + Security (edge)
L2: API Gateway (ingress)
L3: Compute (application)
L4: Cache + Data (persistence)
L5: Storage (object)
L6: Observability (cross-cutting)
Policy relationship rules:
WAF MUST precede API Gateway
API Gateway MUST precede Compute
Validated Layer Order:
waf(L1) → apigw(L2) → k8s(L3)
→ redis(L4) + pg(L4)
→ s3(L5)
logger(L6) receives from all
DIM 4: TOPOLOGY VALIDATION
Topology Validator
Policy topology rules:
- No public-facing DB



How Block Interfaces Drive Edge Creation

Each block in the registry declares its **interfaces** — ingress (what it accepts) and egress (what it connects to). The Graph Composer matches egress of one block to ingress of another to create edges.



13. End-to-End Traced Example: E-Commerce Web App

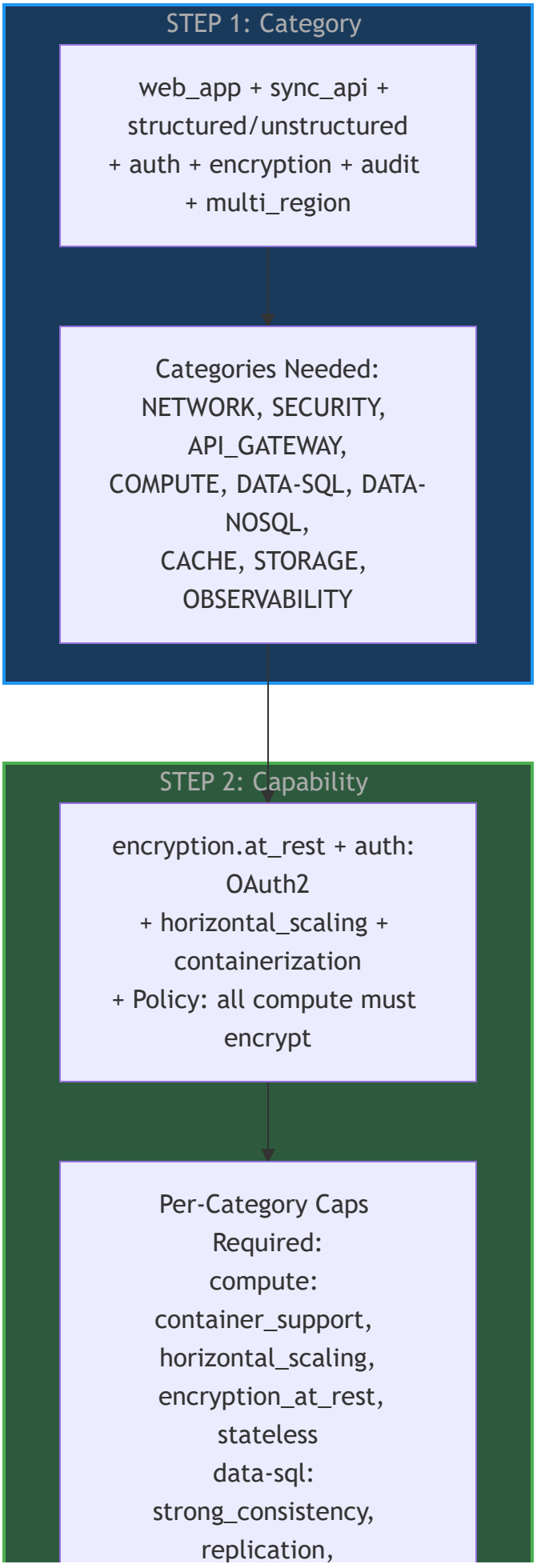
This traces a **real example** through all 10 dimensions (6 for block selection + 4 for topology) to show exactly how an intent becomes a deployment topology.

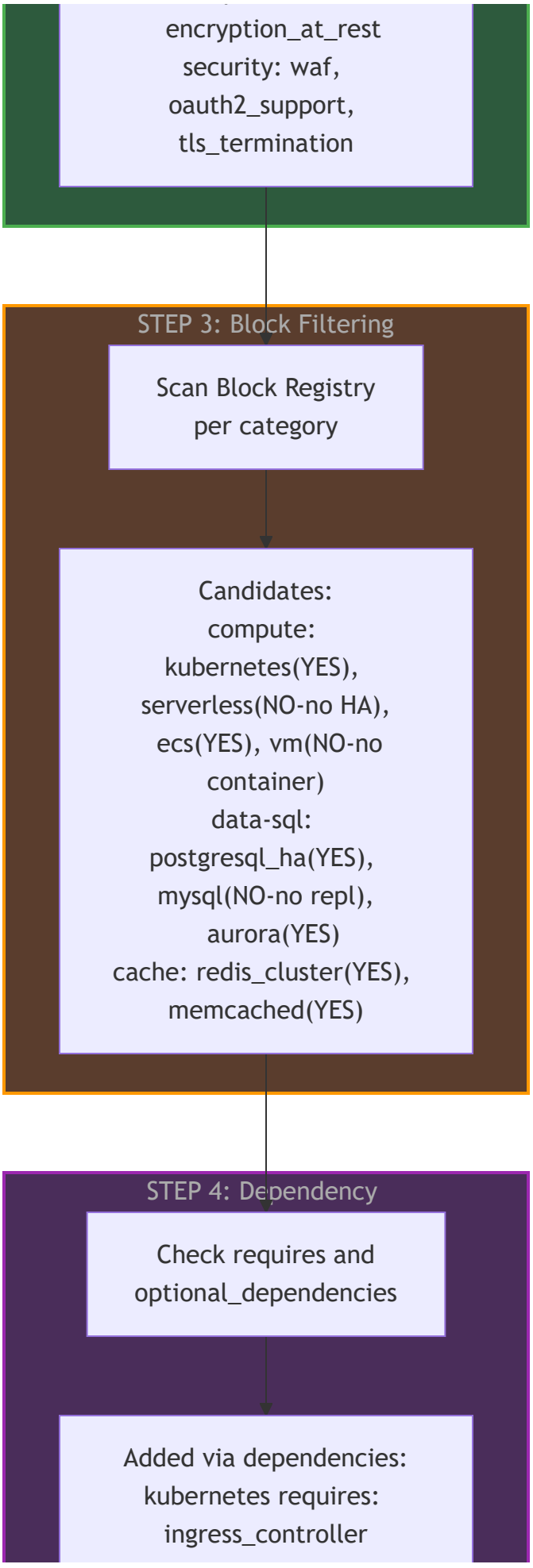
Input Intent (simplified):

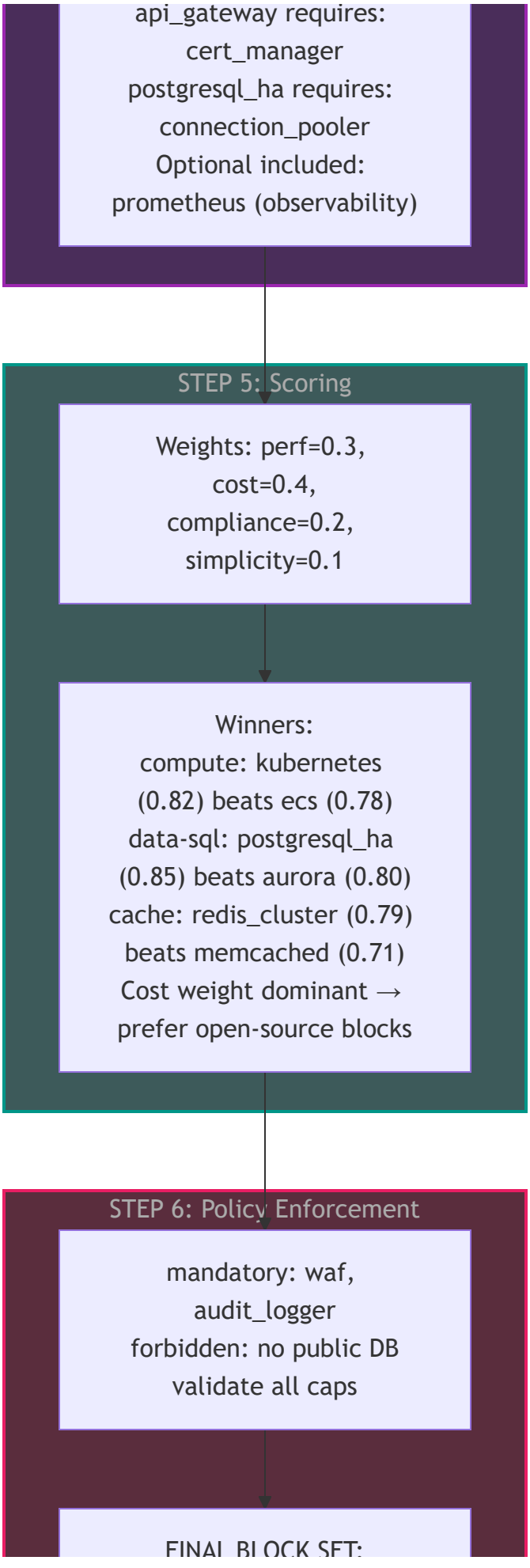
```
application_type: web_app
interaction_model: synchronous_api
data_types: [structured, unstructured]
real_time_processing: false
authentication_required: true (OAuth2)
encryption: { at_rest: true, in_transit: true }
audit_logging_required: true
expected_rps_peak: 5000
multi_region_required: true
horizontal_scaling_required: true
availability_target_percent: 99.9
optimization_priorities: { performance: 0.3, cost: 0.4, compliance: 0.2, simplicity: 0.1 }
```

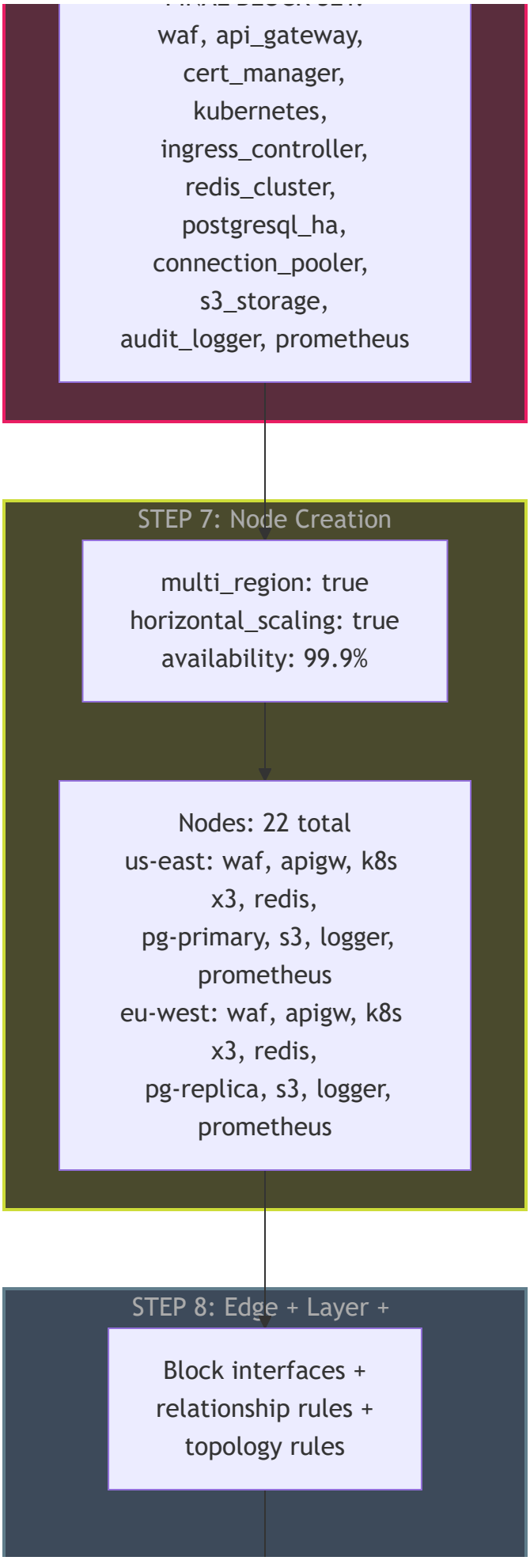
Policy (simplified):

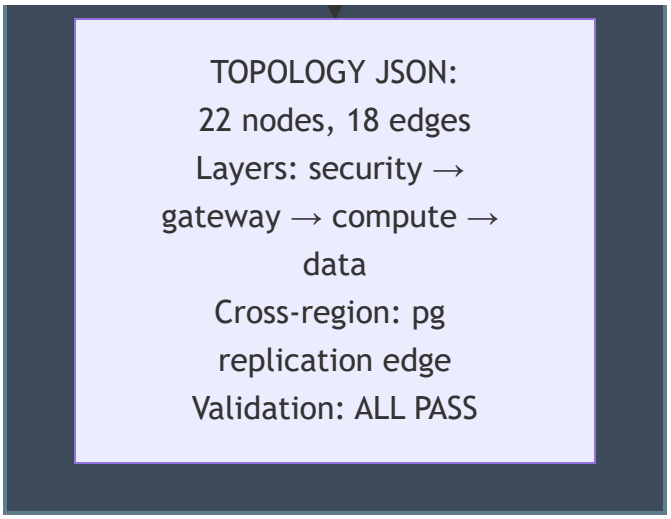
```
mandatory_blocks: [waf, audit_logger]
capability_rules: [all compute must support encryption_at_rest]
topology_rules: [no public-facing database]
relationship_rules: [waf must precede api_gateway]
deployment_rules: [region_count >= 2, data_residency: eu-west]
```


STEP 1: Category
web_app + sync_api + structured/unstructured + auth + encryption + audit + multi_region
Categories Needed: NETWORK, SECURITY, API_GATEWAY, COMPUTE, DATA-SQL, DATA-NOSQL, CACHE, STORAGE, OBSERVABILITY
STEP 2: Capability
encryption.at_rest + auth: OAuth2 + horizontal_scaling + containerization + Policy: all compute must encrypt
Per-Category Caps Required: compute: container_support, horizontal_scaling, encryption_at_rest, stateless data-sql: strong_consistency, replication,

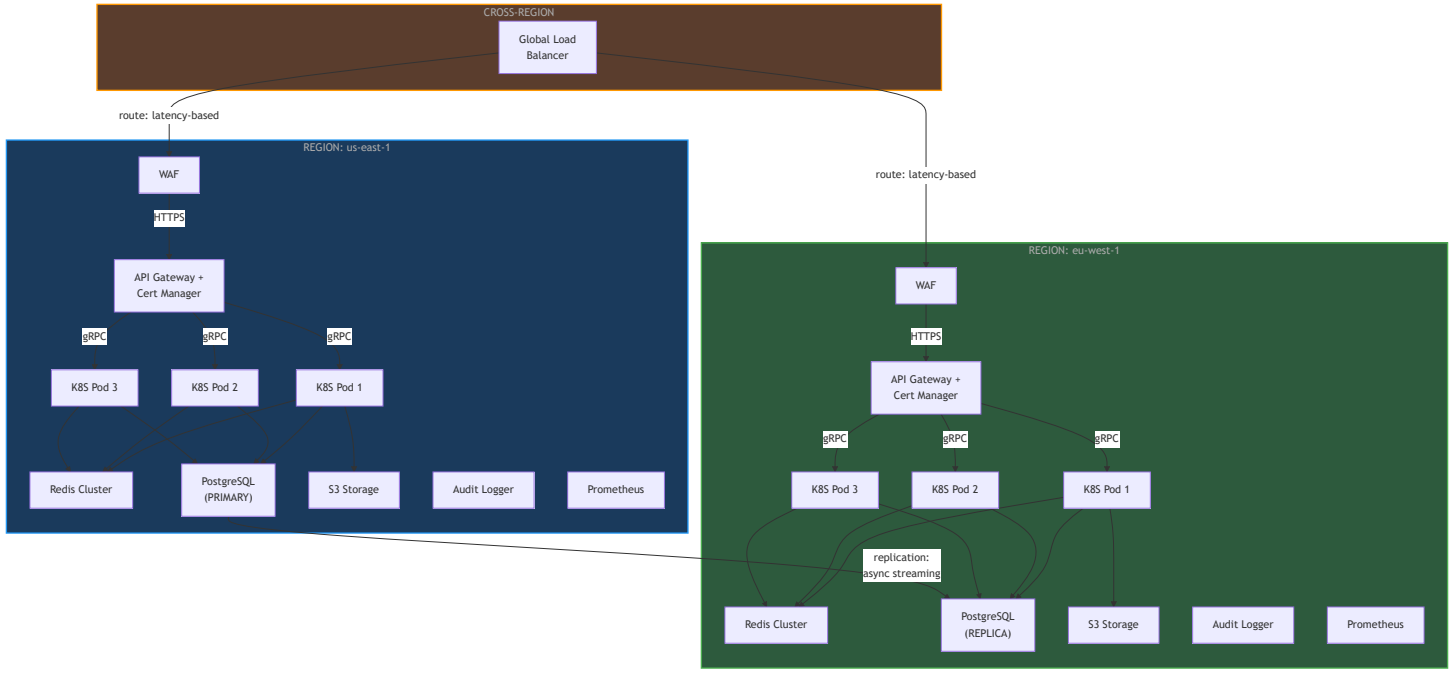

encryption_at_rest
security: waf,
oauth2_support,
tls_termination
STEP 3: Block Filtering
Scan Block Registry
per category
Candidates:
compute:
kubernetes(YES),
serverless(NO-no HA),
ecs(YES), vm(NO-no
container)
data-sql:
postgresql_ha(YES),
mysql(NO-no repl),
aurora(YES)
cache: redis_cluster(YES),
memcached(YES)
STEP 4: Dependency
Check requires and
optional_dependencies
Added via dependencies:
kubernetes requires:
ingress_controller


api_gateway requires:
cert_manager
postgresql_ha requires:
connection_pooler
Optional included:
prometheus (observability)
STEP 5: Scoring
Weights: perf=0.3,
cost=0.4,
compliance=0.2,
simplicity=0.1
Winners:
compute: kubernetes
(0.82) beats ecs (0.78)
data-sql: postgresql_ha
(0.85) beats aurora (0.80)
cache: redis_cluster (0.79)
beats memcached (0.71)
Cost weight dominant →
prefer open-source blocks
STEP 6: Policy Enforcement
mandatory: waf,
audit_logger
forbidden: no public DB
validate all caps
FINAL BLOCK SET:


waf, api_gateway,
cert_manager,
kubernetes,
ingress_controller,
redis_cluster,
postgresql_ha,
connection_pooler,
s3_storage,
audit_logger, prometheus
STEP 7: Node Creation
multi_region: true
horizontal_scaling: true
availability: 99.9%
Nodes: 22 total
us-east: waf, apigw, k8s
x3, redis,
pg-primary, s3, logger,
prometheus
eu-west: waf, apigw, k8s
x3, redis,
pg-replica, s3, logger,
prometheus
STEP 8: Edge + Layer +
Block interfaces +
relationship rules +
topology rules



Resulting Topology Visualization



> **Gap identified:** The above topology is a **logical view** — it shows application blocks but hides the actual infrastructure underneath. When the block is `kubernetes_compute`, we're not deploying "3 pods in air" — there are VMs, control planes, networking, and storage volumes underneath. The next section addresses this.

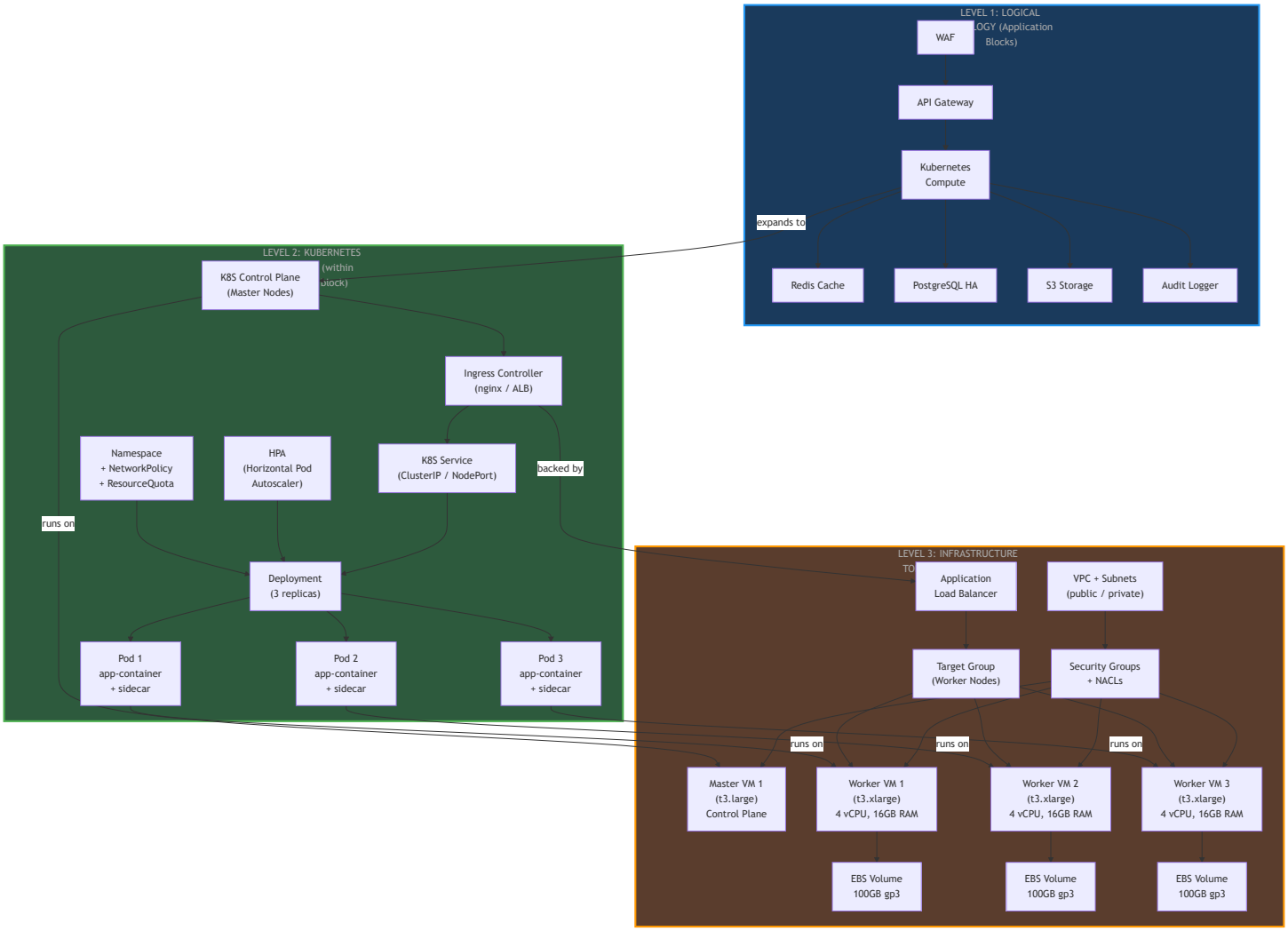
13A. Topology Depth Levels — Logical vs Infrastructure vs K8S

The topology JSON must capture **three depth levels**, not just the flat logical view. Each block in the logical topology **expands into infrastructure components** during Product Resolution.

Why This Matters

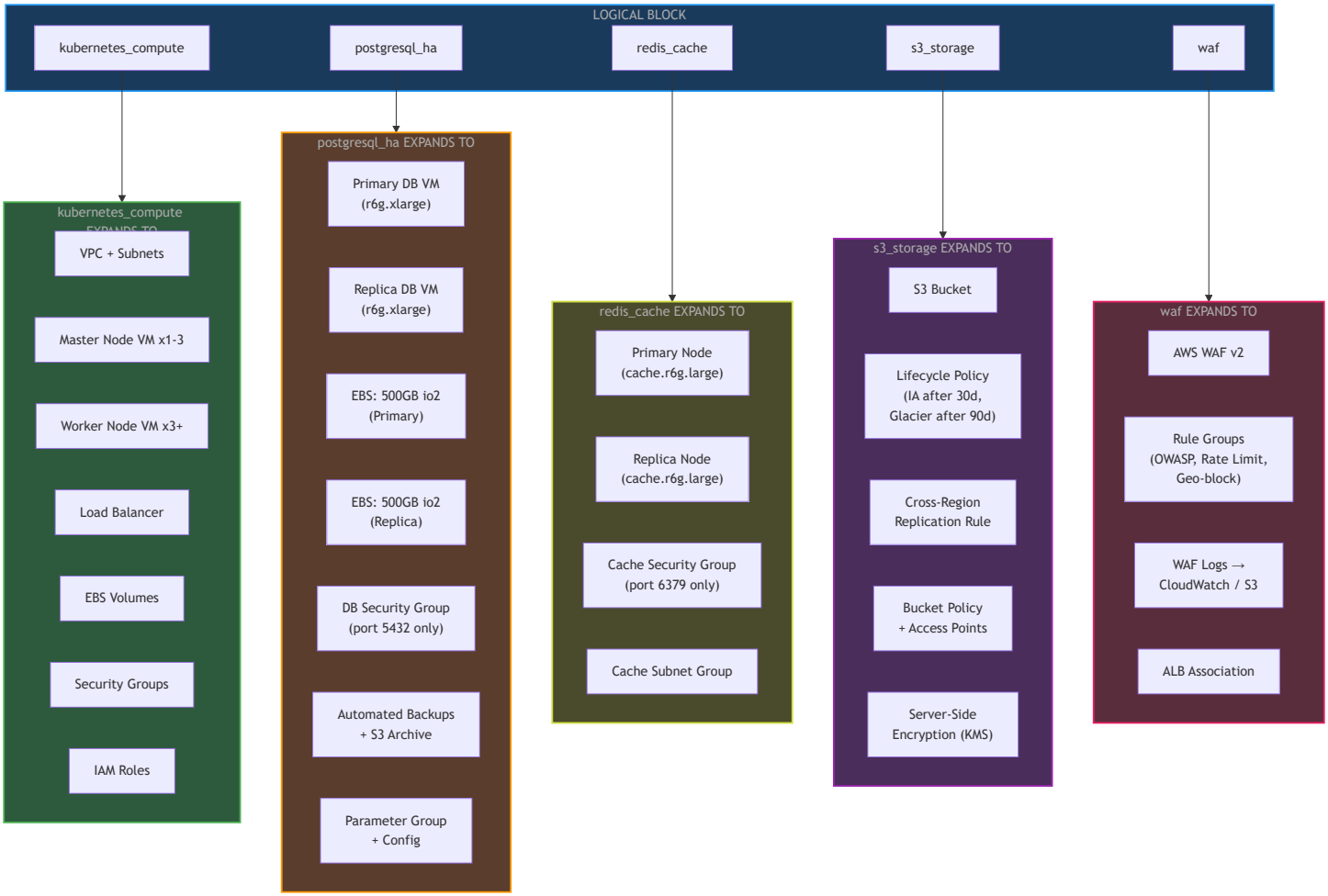
What we say	What we actually provision	What we pay for
"K8S Pod 1, Pod 2, Pod 3"	3 Worker Node VMs + 1-3 Master Node VMs + Ingress Controller + CoreDNS + CNI Plugin	VM instance hours (e.g., 3x t3.xlarge)
"PostgreSQL HA"	1 Primary VM + 1 Replica VM + EBS Volumes + Security Groups + Backup Storage	VM + storage + IOPS
"Redis Cluster"	2-3 Cache Node VMs or Managed Service instances + Replication	Node hours or ElastiCache pricing
"WAF"	Managed WAF Service + Rule Sets + Logging Integration	Request-based pricing
"S3 Storage"	Bucket + Lifecycle Policies + Cross-Region Replication + Access Policies	Storage GB + requests

The Three Depth Levels

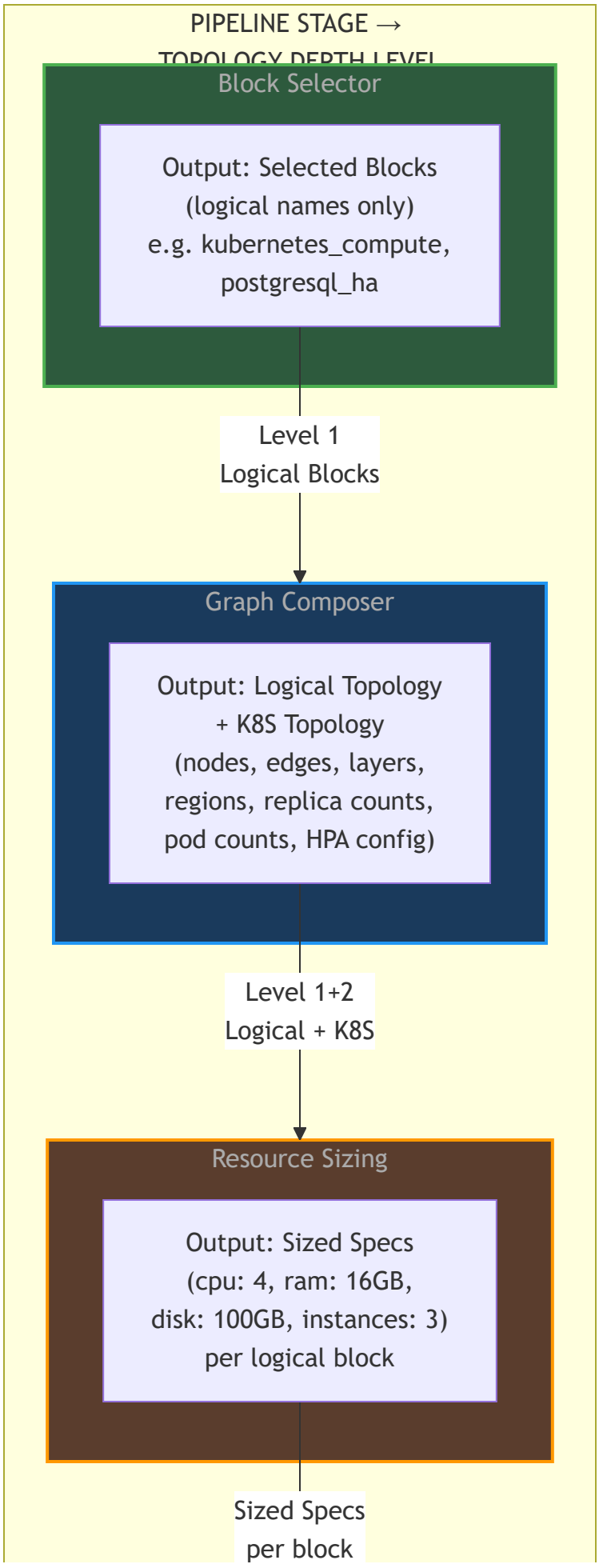


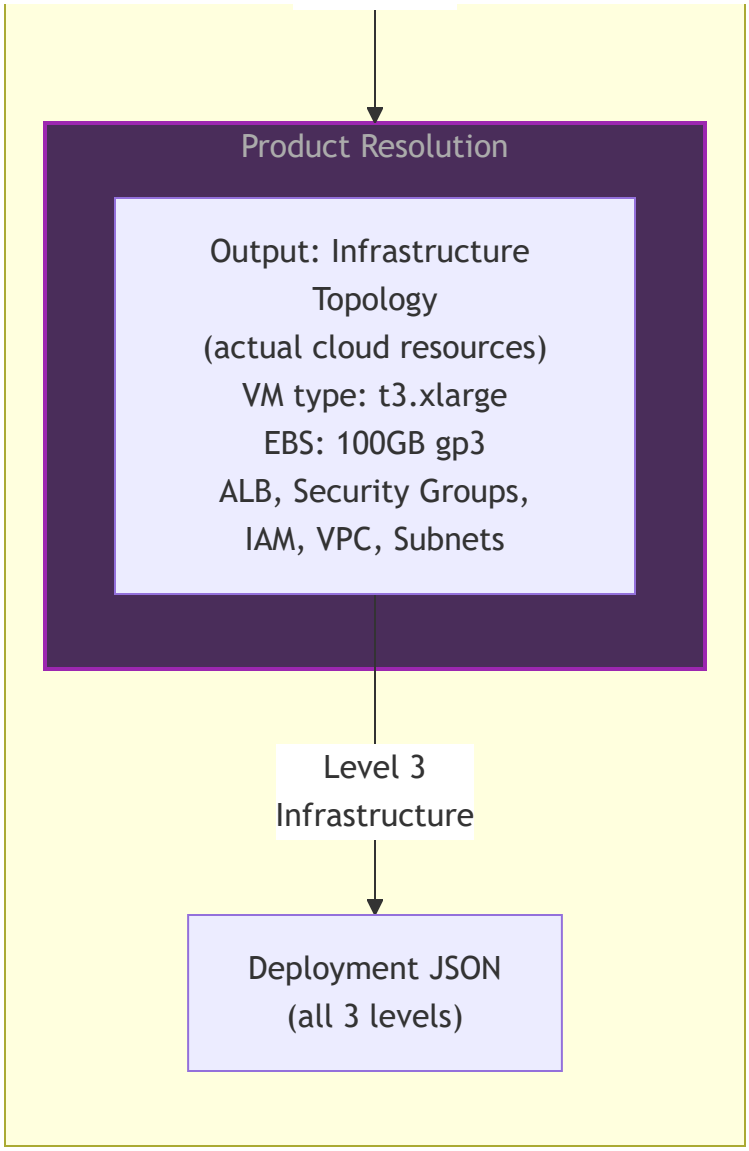
How Each Logical Block Expands to Infrastructure

The same depth principle applies to **every** block, not just compute:


LOGICAL BLOCK
kubernetes_compute
postgresql_ha
redis_cache
s3_storage
waf
kubernetes_compute
VPC + Subnets
Master Node VM x1-3
Worker Node VM x3+
Load Balancer
EBS Volumes
Security Groups
IAM Roles
postgresql_ha EXPANDS TO
Primary DB VM (r6g.xlarge)
Replica DB VM (r6g.xlarge)
EBS: 500GB io2 (Primary)
EBS: 500GB io2 (Replica)
DB Security Group (port 5432 only)
Automated Backups + S3 Archive
Parameter Group + Config
redis_cache EXPANDS TO
Primary Node (cache.r6g.large)
Replica Node (cache.r6g.large)
Cache Security Group (port 6379 only)
Cache Subnet Group
s3_storage EXPANDS TO
S3 Bucket
Lifecycle Policy (IA after 30d, Glacier after 90d)
Cross-Region Replication Rule
Bucket Policy + Access Points
Server-Side Encryption (KMS)
waf EXPANDS TO
AWS WAF v2
Rule Groups (OWASP, Rate Limit, Geo-block)
WAF Logs → CloudWatch / S3
ALB Association

Where Each Depth Level Lives in the Pipeline


PIPELINE STAGE →
Block Selector
Output: Selected Blocks
(logical names only)
e.g. kubernetes_compute,
postgresql_ha
Level 1
Logical Blocks
Graph Composer
Output: Logical Topology
+ K8S Topology
(nodes, edges, layers,
regions, replica counts,
pod counts, HPA config)
Level 1+2
Logical + K8S
Resource Sizing
Output: Sized Specs
(cpu: 4, ram: 16GB,
disk: 100GB, instances: 3)
per logical block
Sized Specs
per block



Block Expansion Registry: The Missing Piece

Each block in the registry needs an **expansion template** that defines what infrastructure components it creates. This is how Product Resolution knows what to provision.

Block	Expansion Template	Infrastructure Components
`kubernetes_compute`	`k8s_cluster_template`	VPC, Subnets (public/private), Master VMs, Worker VMs, ALB, Target Groups, EBS Volumes, Security Groups, IAM Roles, Auto Scaling Group
`postgresql_ha`	`rds_ha_template` or `vm_pg_template`	Primary VM + Replica VM, EBS Volumes (io2), Security Group (5432), Parameter Group, Backup Config, Replication Config

Block	Expansion Template	Infrastructure Components
`redis_cache`	`elasticache_template` or `vm_redis_template`	Primary Node + Replica, Cache Subnet Group, Security Group (6379), Parameter Group
`s3_storage`	`s3_bucket_template`	Bucket, Lifecycle Policy, Replication Rule, Bucket Policy, KMS Encryption Key
`waf`	`waf_v2_template`	WAF WebACL, Rule Groups, Logging Config, ALB Association
`api_gateway`	`apigw_template`	API Gateway, Stage, Usage Plan, Custom Domain, Certificate (ACM)
`audit_logger`	`logging_template`	CloudWatch Log Group, Log Stream, Metric Filters, S3 Archive Bucket, Kinesis Firehose

This expansion template is **configurable** — same block can expand differently depending on:

- Cloud provider (AWS vs GCP vs Azure)
- Managed vs self-managed preference (RDS vs VM-based PostgreSQL)
- Cost tier (production templates vs dev templates)

13B. Per-Microservice Block Instances — Service-Level Repetition

The Gap: Block Type vs Block Instance

The pipeline currently selects one `microservice_compute` block and sizes it for the whole application. In a microservices architecture the same block **type** must be instantiated multiple times — once per service — each independently sized, capability-checked, and policy-evaluated.

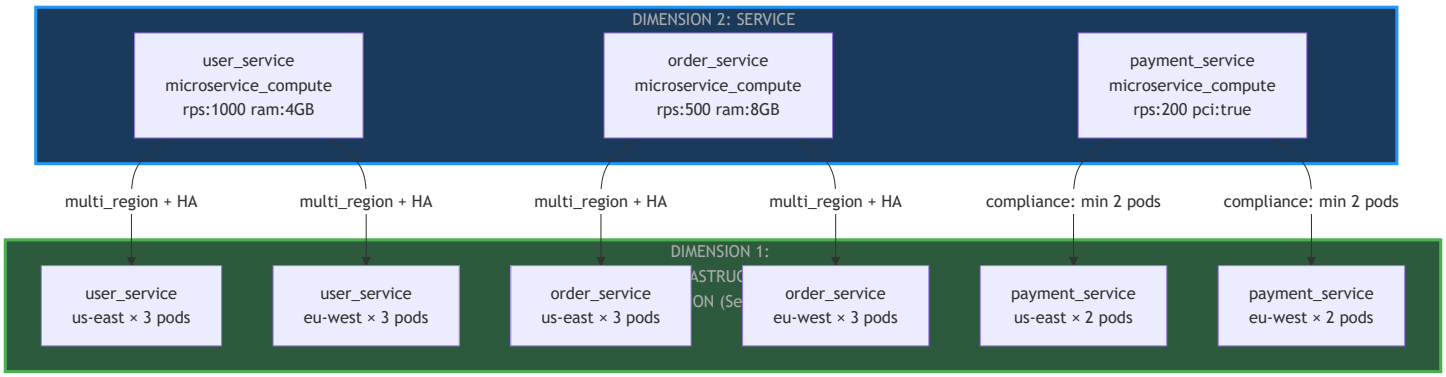
Concept	What It Is	Current Status
Block Type	`microservice_compute` — the definition in the Block Registry	Exists

Concept	What It Is	Current Status
Block Instance	`user_service` , `order_service` , `payment_service` — runtime instantiations of that type	Missing — this section adds it

Two Dimensions of Repetition (Combined View)

These two dimensions are orthogonal — they stack on top of each other:

Dimension	What Repeats	Driven By	In Plan?
Infrastructure Repetition	Same service duplicated across regions / HA pods	`multi_region` , `availability_target` , `horizontal_scaling`	Yes — Section 12, 13A
Service Repetition	Same block type instantiated once per microservice	`services[]` array in Intent JSON	Added here — Section 13B



Intent JSON Extended for Services

The Intent JSON gains a top-level `services` array. Each entry describes one microservice as an independently parametrised block instance:

```
{
  "application_type": "microservices",
  "shared_nfr": {
    "multi_region_required": true,
    "availability_target_percent": 99.9,
    "horizontal_scaling_required": true
  },
  "services": [
    {
      "name": "user_service",
      "block_type": "microservice_compute",
      "rps": 1000,
      "ram_hint_gb": 4,
      "stateless": true
    },
    {
      "name": "order_service",
      "block_type": "microservice_compute",
      "rps": 500,
      "ram_hint_gb": 8,
      "stateless": false
    },
    {
      "name": "payment_service",
      "block_type": "microservice_compute",
      "rps": 200,
      "ram_hint_gb": 4,
      "extra_capabilities": ["pci_compliance", "vault_integration"]
    }
  ]
}
```

`shared_nfr` applies to all instances. Each service can override or extend with its own fields.

Pipeline Processes Each Instance Independently

For every entry in `services[]`, the full pipeline runs as a separate cycle:

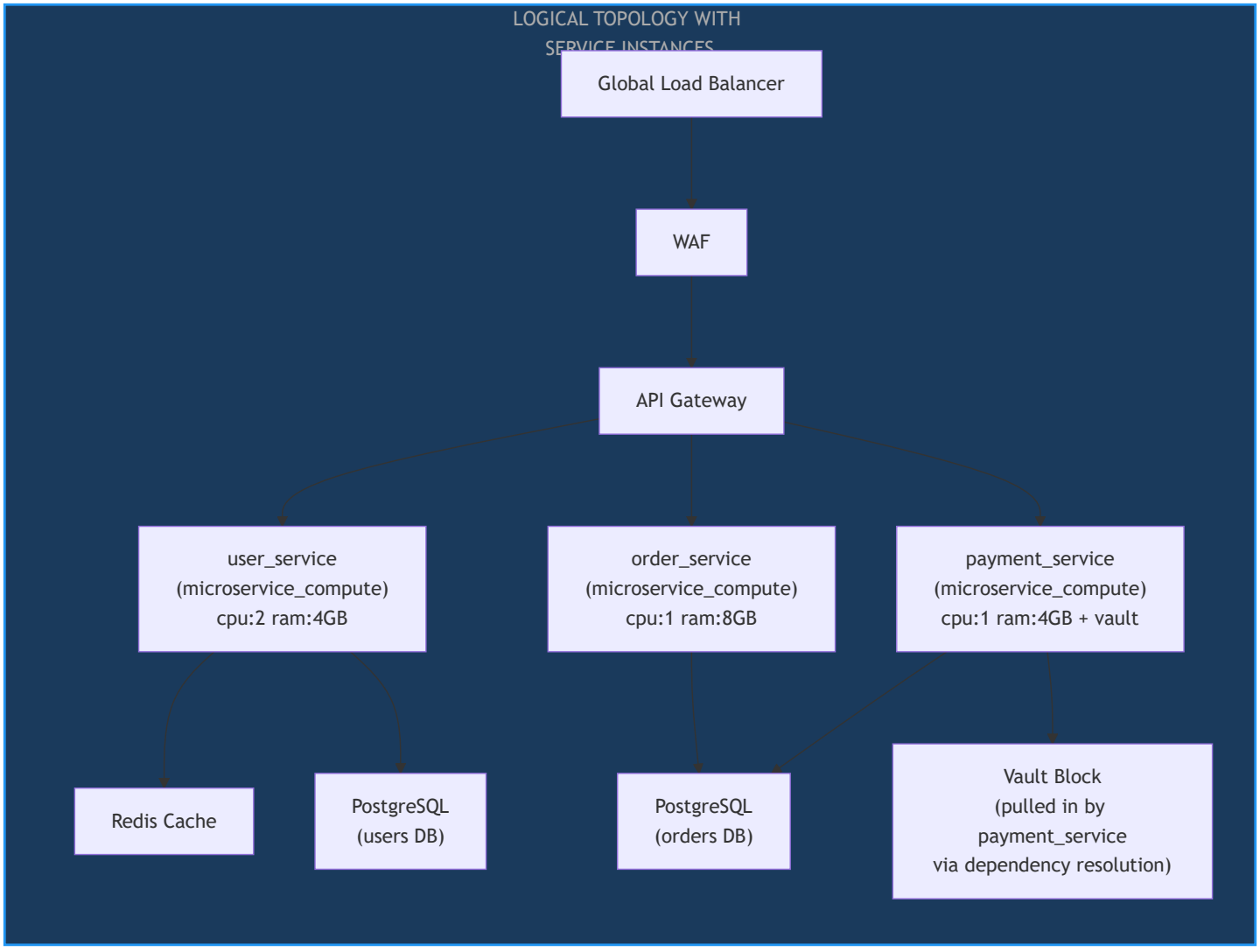

INTENT: services[]
user_service
block_type:
microservice_compute
order_service
block_type:
microservice_compute
payment_service
block_type:
microservice_compute
extra_caps: pci, vault
PIPELINE RUNS ONCE PER
1. Block Type Lookup
Find microservice_compute
in Block Registry
2. Capability Check
Merge base caps + instance
extra_caps
3. Field Resolution
Resolve shared_nfr + per-
service fields
to variable context
4. Formula Engine
Size independently
cpu, ram, instances per
service
5. Rule Engine
Apply compliance per
instance
(payment gets PCI rules)
6. Register BlockInstance
in Topology Graph
BLOCK INSTANCES CREATED
user_service
cpu:2 ram:4GB instances:1
order_service
cpu:1 ram:8GB instances:1
payment_service
cpu:1 ram:4GB instances:2

Topology: N Service Nodes, Not One Compute Node

Before (current plan): One compute node handles all traffic.

```
apigw → kubernetes_compute
```

After (with Block Instances): Each service is a distinct node in the topology graph.



Key points:

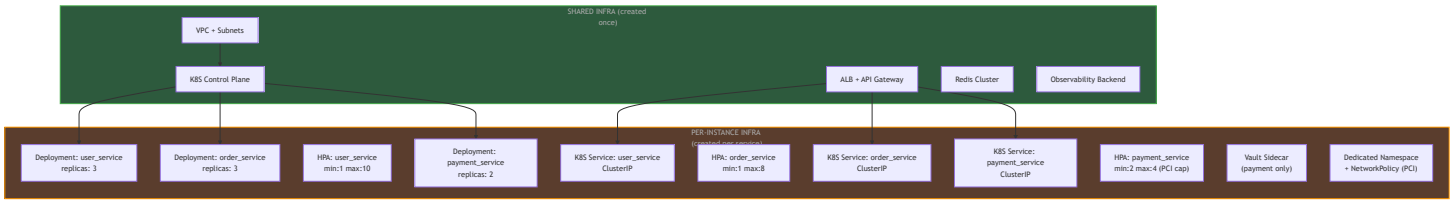
- `vault_block` was **not in the original intent** — it was pulled in automatically via dependency resolution because `payment_service` declared `vault_integration` as an extra capability
- Each service node has its **own sizing spec** — order_service uses more RAM because it holds order state

- Shared infrastructure (`Redis` , `API Gateway` , `WAF`) is created **once** and referenced by multiple instances — not duplicated

Shared vs Per-Instance Infrastructure

Not everything repeats per service. The Block Expansion Registry must distinguish:

Infrastructure Component	Shared or Per-Instance	Rule
VPC, Subnets	**Shared** — created once	All services in the same cluster share networking
K8S Control Plane	**Shared**	One cluster per region, all services run on it
ALB / API Gateway	**Shared**	One ingress point, routes to each service
K8S Deployment	**Per-Instance**	Each service gets its own `Deployment` object
K8S Service (ClusterIP)	**Per-Instance**	Each service gets its own internal DNS
HPA (Horizontal Pod Autoscaler)	**Per-Instance**	Each service scales independently
Security Group (service port)	**Per-Instance**	Each service has its own port and access rules
Vault / HSM block	**Per-Instance**	Only pulled in by services that need it
Observability (logging, metrics)	**Shared + Per-Instance**	Shared backend, per-instance log streams / dashboards



How This Combines With Both Repetition Dimensions

The full picture when all three layers are applied:

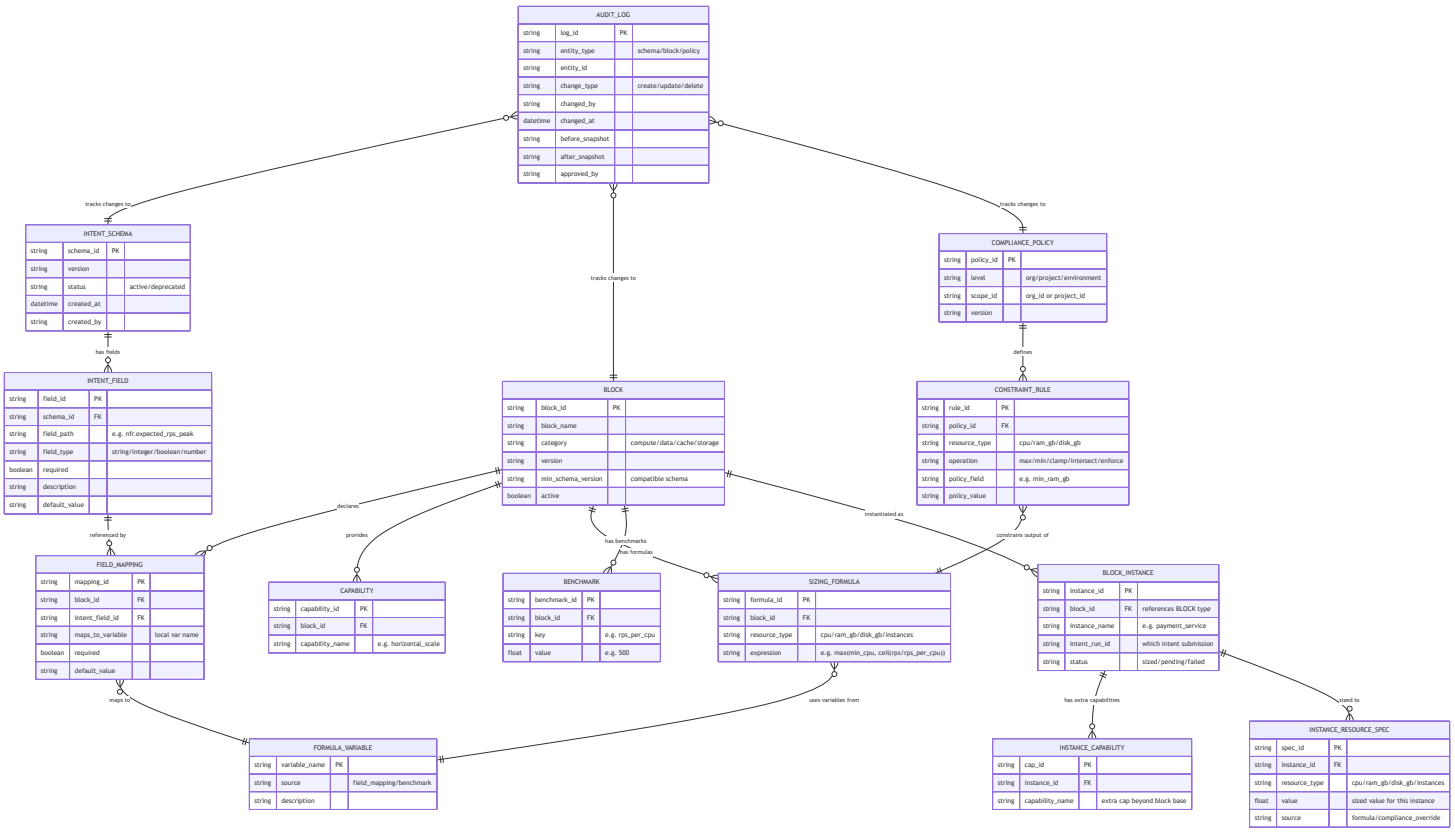
```
Intent: 3 services, multi_region: true, availability: 99.9%

Service Repetition (Section 13B):
  user_service    → microservice_compute → cpu:2 ram:4GB
  order_service   → microservice_compute → cpu:1 ram:8GB
  payment_service → microservice_compute → cpu:1 ram:4GB + vault

Infrastructure Repetition (Section 13A) applied per instance:
  user_service    → 3 pods × 2 regions = 6 pods
  order_service   → 3 pods × 2 regions = 6 pods
  payment_service → 2 pods × 2 regions = 4 pods (PCI min=2 cap)

Block Expansion (Section 13A) shared vs per-instance:
  Shared:         1 VPC, 1 K8S control plane, 1 ALB, 1 Redis cluster
  Per-instance:   3 Deployments, 3 Services, 3 HPAs, 3 Security Groups
  payment only:   Vault sidecar, dedicated Namespace + NetworkPolicy
```

14. Data Model Relationships — ER Diagram

Complete entity-relationship diagram showing all configuration entities, their attributes, and relationships. This is the **database schema** backing the Configurator.

Key relationships:

- `INTENT_SCHEMA` has many `INTENT_FIELD` s
- Each `BLOCK` declares many `FIELD_MAPPING` s, each referencing an `INTENT_FIELD`
- Each `BLOCK` has many `SIZING_FORMULA` s that use `FORMULA_VARIABLE` s
- `FIELD_MAPPING` s produce `FORMULA_VARIABLE` s that formulas consume
- `COMPLIANCE_POLICY` defines `CONSTRAINT_RULE` s that constrain formula outputs
- `BLOCK` (type) has many `BLOCK_INSTANCE` s — one per microservice declared in the intent
- `BLOCK_INSTANCE` carries its own sized resource spec and extra capabilities, resolved at runtime
- `AUDIT_LOG` tracks changes to all entities



15. Gaps Identified in Current Architecture

#	Gap	Severity	Addressed By
1	**No JSON Schema Versioning** — No version field on any JSON type, no backward compatibility strategy	High	Phase 1: Schema Registry with versioning
2	**Hardcoded Field Coupling** — Resource Sizing directly reads `intent["nfr"]["expected_rps_peak"]` , Block Selection reads specific intent fields by name	Critical	Phase 1: Field Mapping declarations
3	**No Formula Engine** — Sizing formulas are Python functions, not configurable expressions	Critical	Phase 2: Expression-based Formula Engine
4	**Blocks Don't Declare Intent Field Dependencies** — No way to validate at	High	Phase 1: `intent_field_mappings` on

#	Gap	Severity	Addressed By
	config time whether intent satisfies a block's requirements		each block
5	**No Admin / Configuration UI** — No pages for managing blocks, policies, or intent templates	High	Phase 4: Three Configurator UI pages
6	**No Conflict Resolution for Block-Intent Mismatch** — What happens when intent asks for something no block provides?	Medium	Phase 3: Impact Analyzer + validation
7	**No Feedback Loop** — Benchmarks are static, no mechanism to learn from actual deployment resource usage	Medium	Future: Feedback loop from monitoring
8	**No Intent Templates** — Every user fills intent from scratch, leading to inconsistencies	Low	Future: Template library in Intent Schema Builder
9	**No Block Instance Model** — Pipeline treats each block type as a singleton; no concept of multiple instances of the same block type for different microservices	High	Section 13B: `BLOCK_INSTANCE` entity + `services[]` array in Intent JSON
10	**Fat Block Expansion Templates** — Current expansion templates bundle all infra into one monolithic template per block, preventing reuse and forcing unnecessary provisioning	High	Section 13B: Expansion templates decomposed into small infrastructure primitive blocks; shared infra (VPC, ALB, Control Plane) created once and referenced by all instances

16. Configurator Component Summary

Component	Purpose	Prevents Code Change When
Intent Schema Registry	Defines intent JSON structure with versioning	Intent fields added / renamed / removed
Block Field Mapping	Blocks declare which intent fields they consume	Intent schema evolves
Formula Engine	Evaluates sizing formulas from expression strings	Sizing logic changes
Rule Engine	Evaluates constraint rules from declarative config	New policy types added
Impact Analyzer	Shows blast radius of any config change	Before any config change is saved
Audit Log	Tracks all config changes with who/what/when/diff	Compliance / governance requirements
Approval Workflow	Requires second approver for prod changes	Governance / change management
Version Rollback	Restores any previous config version	Recovery from bad changes

Runtime Engine Transformation

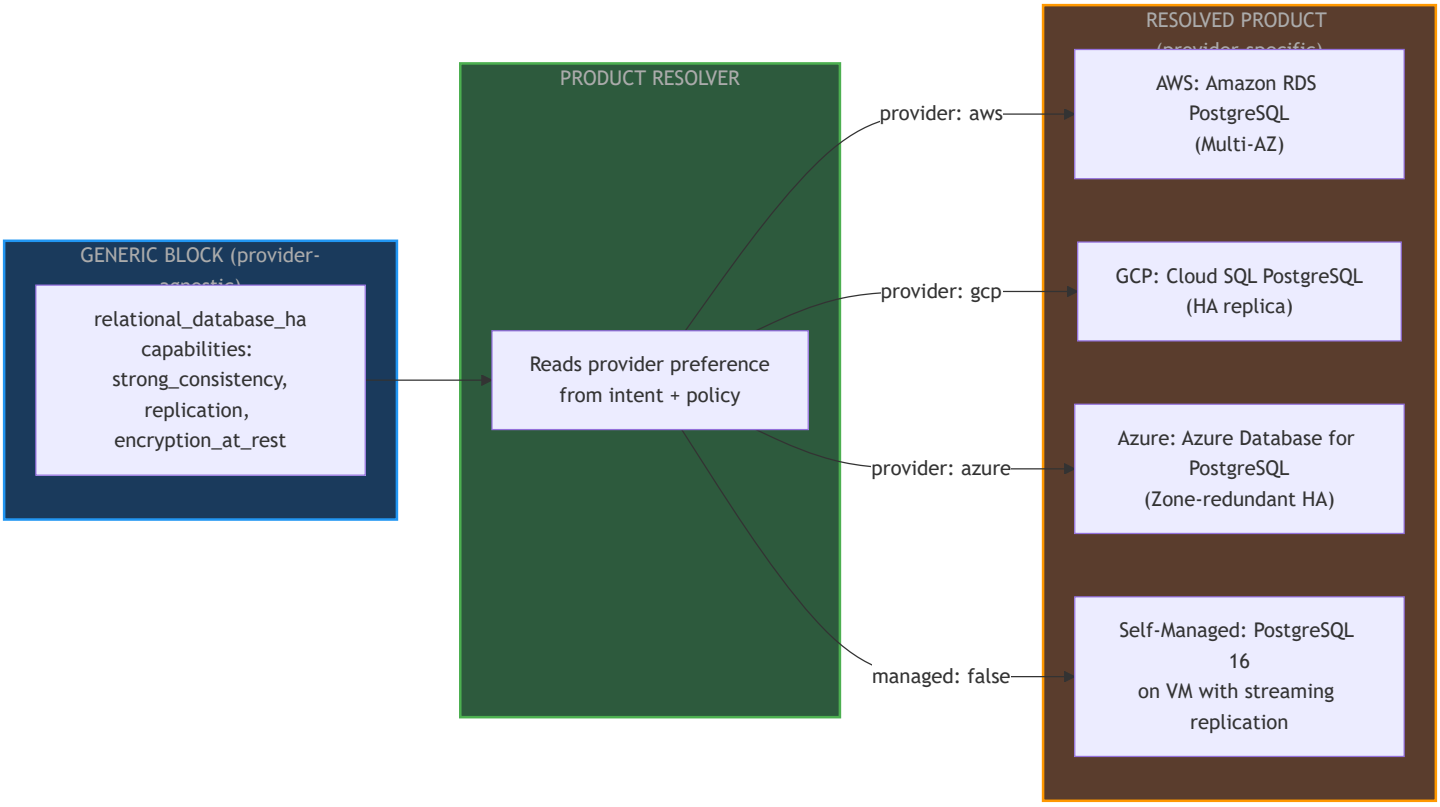
Concern	Today (Rigid)	After Configurator (Flexible)
Field Access	`intent["nfr"]["expected_rps_peak"]`	`resolve_field(block.mappings, intent)`
Category Dispatch	`if category == "compute": _calc()`	`evaluate_formula(block.formulas, vars)`
Sizing Formula	Hardcoded Python arithmetic	Expression string evaluated at runtime
Policy Constraint	Hardcoded per resource type	Declarative rule applied generically

17. Generic Block Catalog — Full Registry

The Principle: Generic Block vs Product

A block is **never** a specific product. It is a generic capability description. Product resolution happens as the final pipeline stage, mapping each generic block to a real product based on:

- Cloud provider preference (AWS / GCP / Azure / self-managed)
- Managed vs self-managed preference
- Cost tier (dev / staging / production)
- Policy constraints (data residency, vendor lock-in rules)

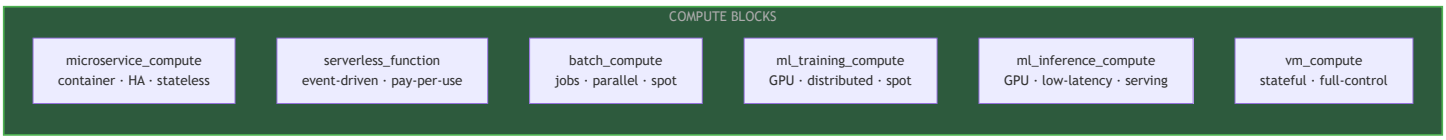


The runtime pipeline never references AWS, GCP, or Azure by name until the **Product Resolver** stage. All prior stages (Block Selection, Sizing, Compliance, Graph Composition) work purely on generic blocks.

Category 1 — COMPUTE

Block ID	Capabilities Provided	Used When
`microservice_compute`	`container_support` , `horizontal_scaling` , `stateless` , `auto_scaling` , `health_check` , `kubernetes_native` , `statefulset_support` , `gpu_support` , `custom_scheduling`	Any containerised service — capability profile at instance level drives K8s vs managed-container vs GPU variant
`serverless_function`	`event_driven` , `auto_scaling` , `stateless` , `pay_per_request`	Lightweight event handlers, short-lived tasks
`batch_compute`	`job_scheduling` , `parallel_execution` , `spot_instance_support`	Nightly jobs, data processing pipelines
`ml_training_compute`	`gpu_support` , `distributed_training` , `high_memory` , `spot_instance_support`	ML model training workloads
`ml_inference_compute`	`gpu_support` , `low_latency` , `auto_scaling` , `model_serving`	Real-time ML inference endpoints
`vm_compute`	`persistent_state` , `custom_os` , `full_control`	Legacy apps, stateful workloads needing full VM

> **Note on `microservice_compute` capabilities:** This block declares a **superset** of capabilities it CAN support. Each block instance (per microservice) carries only the **active capability subset** it actually needs — derived from intent signals. Product Resolution uses this active subset as part of the lookup key, not the full superset. See "Capability-Profile-Driven Product Resolution" subsection below.



Category 2 — NETWORKING

Block ID	Capabilities Provided	Used When
`global_load_balancer`	`anycast_routing`, `geo_routing`, `multi_region_failover`	Multi-region apps, latency-based routing
`regional_load_balancer`	`l7_routing`, `ssl_termination`, `health_check`, `sticky_sessions`	Single-region L7 load balancing
`cdn`	`edge_caching`, `static_asset_delivery`, `geo_distribution`	Static assets, media, global latency reduction
`dns`	`domain_resolution`, `health_check_routing`, `geo_dns`	Custom domain management
`vpc_network`	`network_isolation`, `private_subnet`, `public_subnet`, `nat_support`	Every deployment — networking foundation
`nat_gateway`	`outbound_internet`, `private_subnet_egress`	Private subnets needing internet access
`vpn_gateway`	`site_to_site_vpn`, `on_premise_connectivity`	Hybrid cloud / on-prem connectivity
`private_endpoint`	`private_connectivity`, `no_public_internet`	Connecting services without public exposure

Category 3 — SECURITY

Block ID	Capabilities Provided	Used When
`web_application_firewall`	`owasp_protection`, `rate_limiting`, `geo_blocking`, `bot_protection`	Any public-facing HTTP endpoint
`ddos_protection`	`volumetric_attack_mitigation`, `layer3_layer4_protection`	High-traffic or high-risk public services
`tls_termination`	`ssl_offloading`, `certificate_management`, `https_enforcement`	All external-facing services

Block ID	Capabilities Provided	Used When
`certificate_manager`	`tls_cert_lifecycle` , `auto_renewal` , `wildcard_support`	Services needing TLS certs
`secret_manager`	`secrets_storage` , `rotation` , `access_control` , `audit_trail`	Any service needing API keys, passwords, tokens
`key_management_service`	`encryption_key_management` , `hsm_backed` , `key_rotation` , `envelope_encryption`	Encryption at rest requirements, PCI/HIPAA
`identity_aware_proxy`	`zero_trust_access` , `identity_verification` , `context_aware_access`	Internal tools, admin panels without VPN

Category 4 — API & INGRESS

Block ID	Capabilities Provided	Used When
`api_gateway`	`request_routing` , `rate_limiting` , `auth_integration` , `api_versioning` , `throttling`	Any API needing managed ingress
`ingress_controller`	`kubernetes_ingress` , `path_routing` , `tls_termination` , `canary_routing`	K8s-based deployments
`service_mesh`	`mtls` , `circuit_breaker` , `traffic_shaping` , `service_discovery` , `east_west_traffic`	Microservices with complex inter-service communication
`graphql_gateway`	`schema_federation` , `query_batching` , `subscriptions`	GraphQL APIs
`grpc_gateway`	`grpc_transcoding` , `http2_support` , `proto_validation`	gRPC-based services

Category 5 — DATA

Block ID	Capabilities Provided	Used When
`relational_database`	`sql`, `acid_transactions`, `structured_data`	Standard OLTP, single AZ acceptable
`relational_database_ha`	`sql`, `acid_transactions`, `replication`, `automatic_failover`, `encryption_at_rest`	Production OLTP requiring HA
`document_database`	`json_documents`, `flexible_schema`, `horizontal_scaling`, `secondary_indexes`	Unstructured / semi-structured data
`key_value_database`	`single_digit_ms_latency`, `horizontal_scaling`, `ttl_support`	Session storage, counters, user preferences
`time_series_database`	`time_indexed`, `retention_policies`, `aggregation_queries`, `high_write_throughput`	Metrics, IoT data, event logs
`graph_database`	`relationship_traversal`, `graph_queries`, `connected_data`	Social graphs, recommendation engines, fraud detection
`wide_column_database`	`massive_scale`, `multi_region_replication`, `high_write_throughput`, `eventual_consistency`	Very large datasets, multi-region writes
`search_engine`	`full_text_search`, `faceted_search`, `relevance_scoring`, `geospatial_search`	Product search, log search, autocomplete
`data_warehouse`	`olap`, `columnar_storage`, `analytical_queries`, `large_dataset_scans`	Business intelligence, analytics

Category 6 — CACHE

Block ID	Capabilities Provided	Used When
`in_memory_cache`	`sub_ms_latency`, `ttl_support`, `eviction_policies`	Session cache, DB query cache, rate limiting counters
`distributed_cache_cluster`	`horizontal_scaling`, `replication`, `pub_sub`, `lua_scripting`, `persistence`	High-availability cache with persistence requirements
`cdn_cache`	`edge_caching`, `cache_invalidation`, `vary_headers`	HTTP response caching at the edge

Category 7 — STORAGE

Block ID	Capabilities Provided	Used When
`object_storage`	`unlimited_capacity`, `versioning`, `lifecycle_policies`, `cross_region_replication`, `presigned_urls`	File uploads, media, backups, static assets
`block_storage`	`persistent_disk`, `low_latency_io`, `snapshot_support`, `encryption_at_rest`	VM disks, database data volumes
`file_storage`	`nfs_protocol`, `shared_access`, `posix_compliant`	Shared file systems, legacy app storage
`archive_storage`	`cold_storage`, `low_cost`, `retrieval_delay`	Long-term backup, compliance archival

Category 8 — MESSAGING & STREAMING

Block ID	Capabilities Provided	Used When
`message_queue`	`guaranteed_delivery`, `dead_letter_queue`, `at_least_once`, `visibility_timeout`	Async task processing, decoupled services

Block ID	Capabilities Provided	Used When
`event_streaming`	`ordered_messages` , `consumer_groups` , `replay` , `high_throughput` , `retention`	Event sourcing, real-time data pipelines, audit streams
`pub_sub_broker`	`fan_out` , `topic_based_routing` , `push_and_pull` , `filtering`	Notification systems, multi-subscriber events
`event_bus`	`event_routing` , `pattern_matching` , `cross_service_events` , `schema_registry`	Microservice event choreography

Category 9 — OBSERVABILITY

Block ID	Capabilities Provided	Used When
`log_aggregator`	`centralised_logging` , `structured_logs` , `log_search` , `retention_policies`	All deployments — always included
`metrics_collector`	`time_series_metrics` , `dashboards` , `alerting` , `scraping`	Service health monitoring
`distributed_tracing`	`trace_correlation` , `latency_breakdown` , `service_dependency_map`	Microservices — diagnosing inter-service latency
`audit_trail`	`immutable_logs` , `who_what_when` , `compliance_ready` , `tamper_detection`	Compliance requirements (SOC2, HIPAA, PCI)
`alerting`	`threshold_alerts` , `anomaly_detection` , `pagerduty_integration` , `escalation_policies`	On-call and incident management
`uptime_monitor`	`synthetic_checks` , `external_probe` , `sla_reporting`	SLA tracking, user-facing availability

Category 10 — IDENTITY & AUTH

Block ID	Capabilities Provided	Used When
`identity_provider`	`oauth2` , `oidc` , `saml` , `mfa` , `sso` , `user_management`	Any app needing user authentication
`api_auth_service`	`jwt_validation` , `api_key_management` , `token_introspection`	API-to-API auth, machine identity
`rbac_engine`	`role_based_access` , `policy_enforcement` , `fine_grained_permissions`	Apps with complex permission models
`service_account_manager`	`machine_identity` , `short_lived_credentials` , `workload_identity`	Service-to-service auth in K8s / cloud

Category 11 — ML / AI

Block ID	Capabilities Provided	Used When
`ml_feature_store`	`feature_serving` , `training_serving_skew_prevention` , `feature_versioning`	ML apps needing consistent feature pipelines
`ml_experiment_tracker`	`run_tracking` , `metric_logging` , `artifact_storage` , `model_registry`	ML teams tracking experiments
`model_registry`	`model_versioning` , `staging_production_promotion` , `rollback`	Production ML model lifecycle management
`vector_database`	`embedding_storage` , `similarity_search` , `ann_indexing`	RAG pipelines, semantic search, LLM memory

Category 12 — INFRASTRUCTURE PRIMITIVES

These are the **small building blocks used by Block Expansion** (Section 13A/13B). They are not selected by Block Selection — they are provisioned as part of expanding logical blocks into real infrastructure.

Block ID	Capabilities Provided	Shared or Per-Instance
`vpc_network`	`network_isolation`, `subnet_management`, `route_tables`	Shared — one per deployment
`subnet`	`ip_allocation`, `availability_zone_placement`	Shared — created once per AZ
`security_group`	`inbound_outbound_rules`, `port_based_access`	Per-instance — each service/block gets its own
`iam_role`	`cloud_api_permissions`, `least_privilege`, `service_account`	Per-instance — each block instance gets its own
`virtual_machine`	`compute_instance`, `custom_os`, `ssh_access`	Per-instance or shared (K8s nodes shared)
`auto_scaling_group`	`scale_out_in`, `health_replacement`, `launch_template`	Per-instance — per service HPA/ASG
`persistent_volume`	`block_storage_attachment`, `snapshot`, `encryption`	Per-instance — each DB / stateful block gets its own
`namespace_isolation`	`kubernetes_namespace`, `network_policy`, `resource_quota`	Per-instance — one per microservice in K8s

Capability-Profile-Driven Product Resolution

For most blocks, the lookup key is simply `(block_id + provider + tier)` — one product per provider. But for blocks that declare a superset of capabilities (like `microservice_compute`), the lookup key includes the **active capability profile** of the block instance, resolving to different products depending on what capabilities are activated.

Lookup key structure:

```
(block_id, active_capability_profile, cloud_provider, managed_preference, tier)
  → product_name + product_config_params
```

`microservice_compute` capability-profile mapping:

Active Capability Profile	AWS	GCP	Azure	Self-Managed
`container_support` only (no k8s)	ECS Fargate	Cloud Run	Azure Container Apps	Docker Compose / Nomad
`container_support` + `auto_scaling`	ECS Fargate + ASG	Cloud Run	Azure Container Apps	Nomad + autoscaler
`container_support` + `kubernetes_native`	EKS (EC2 node groups)	GKE Standard	AKS	K8s on VMs (kubeadm)
`container_support` + `kubernetes_native` + `statefulset_support`	EKS + EBS CSI driver	GKE + Persistent Disk	AKS + Azure Disk	K8s + Rook-Ceph
`container_support` + `gpu_support`	EKS + GPU node group (p3/g4)	GKE + GPU node pool	AKS + NC-series nodes	K8s + GPU operator
`container_support` + `kubernetes_native` + `custom_scheduling`	EKS + Karpenter	GKE Autopilot	AKS + KEDA	K8s + custom scheduler

Rule: The product resolver matches the **most specific** capability profile row — longest matching subset wins.

Product Resolution Table — Generic Block → Cloud Product

This is the mapping the **Product Resolver** uses as its lookup. For blocks without capability variants, the key is `(block_id + provider + tier)`. For `microservice_compute` the extended capability-profile

table above applies. The runtime never hardcodes this — it reads the active mapping from configuration.

Generic Block	AWS	GCP	Azure	Self-Managed
`microservice_compute`	See capability-profile table above	See capability-profile table above	See capability-profile table above	See capability-profile table above
`serverless_function`	AWS Lambda	Cloud Functions	Azure Functions	OpenFaaS / Knative
`batch_compute`	AWS Batch	Cloud Batch	Azure Batch	Kubernetes Jobs
`ml_training_compute`	SageMaker Training / EC2 P-series	Vertex AI Training / TPU	Azure ML Compute	GPU VMs + Kubeflow
`ml_inference_compute`	SageMaker Endpoints / Inf2	Vertex AI Endpoints	Azure ML Online Endpoints	Triton Inference Server
`global_load_balancer`	AWS Global Accelerator	Cloud Load Balancing (global)	Azure Front Door	Nginx + GeoDNS
`regional_load_balancer`	AWS ALB	Cloud Load Balancing (regional)	Azure Application Gateway	HAProxy / Nginx
`cdn`	CloudFront	Cloud CDN	Azure CDN	Varnish / Nginx
`vpc_network`	AWS VPC	VPC	Azure Virtual Network	Linux networking
`web_application_firewall`	AWS WAF v2	Cloud Armor	Azure WAF	ModSecurity
`ddos_protection`	AWS Shield Advanced	Cloud Armor Managed Protection	Azure DDoS Protection	Cloudflare

Generic Block	AWS	GCP	Azure	Self-Managed
`tls_termination`	AWS ACM + ALB	Managed SSL (GCP)	Azure App Gateway TLS	Certbot + Nginx
`certificate_manager`	AWS ACM	Google-managed SSL	Azure App Service Certs	cert-manager (K8s)
`secret_manager`	AWS Secrets Manager	Secret Manager	Azure Key Vault (Secrets)	HashiCorp Vault
`key_management_service`	AWS KMS	Cloud KMS	Azure Key Vault (Keys)	HashiCorp Vault
`api_gateway`	AWS API Gateway	Apigee / Cloud Endpoints	Azure API Management	Kong / KrakenD
`ingress_controller`	AWS Load Balancer Controller	GKE Ingress	AKS Ingress	Nginx / Traefik
`service_mesh`	AWS App Mesh	Anthos Service Mesh	Azure Service Mesh (Istio)	Istio / Linkerd
`relational_database`	RDS (single AZ)	Cloud SQL (single)	Azure DB for PostgreSQL (single)	PostgreSQL VM
`relational_database_ha`	RDS Multi-AZ	Cloud SQL HA	Azure DB for PostgreSQL HA	PostgreSQL + Patroni
`document_database`	Amazon DocumentDB	Firestore / MongoDB Atlas	Azure Cosmos DB (Mongo API)	MongoDB
`key_value_database`	Amazon DynamoDB	Firestore / Bigtable	Azure Cosmos DB	Redis / ScyllaDB

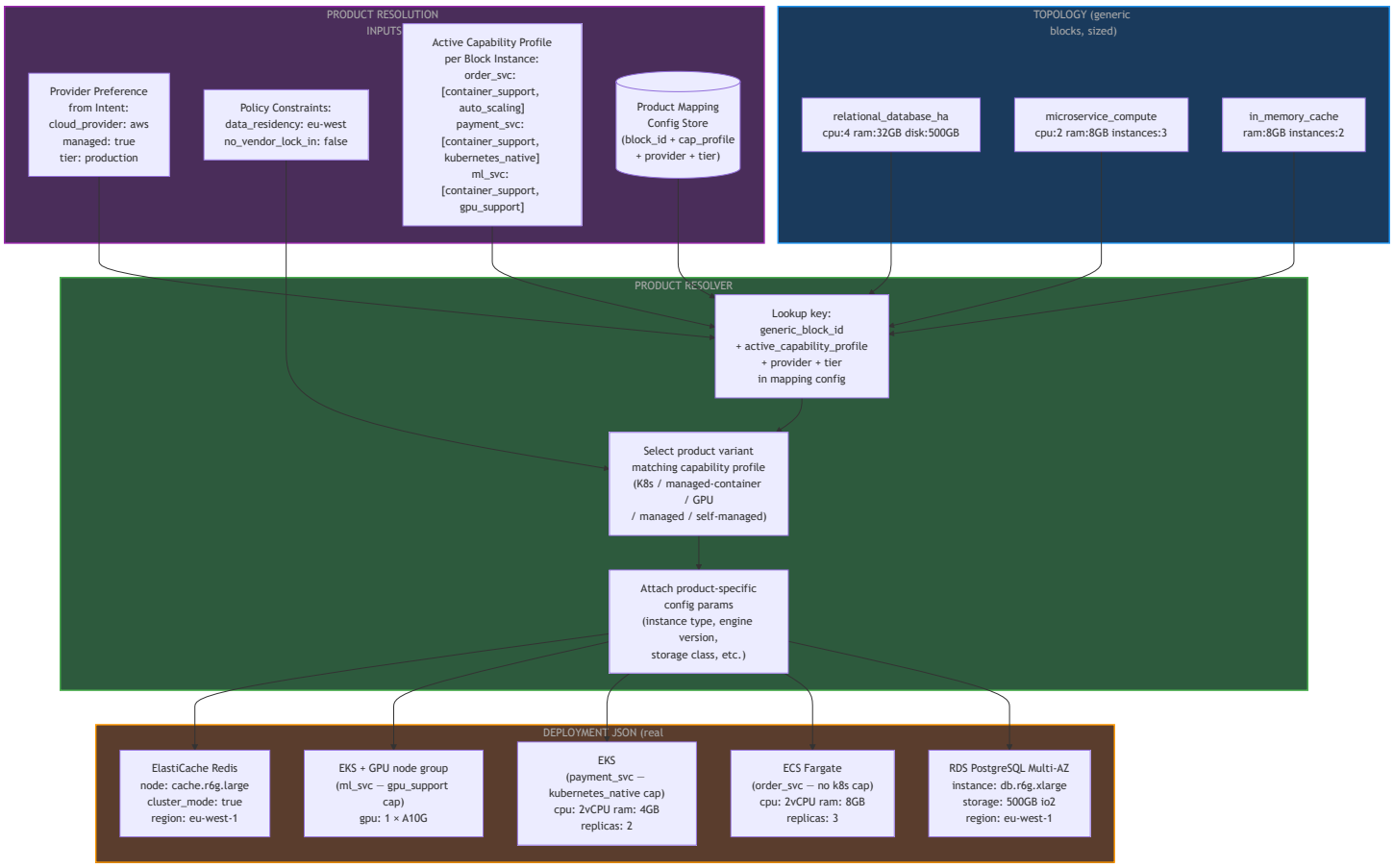
Generic Block	AWS	GCP	Azure	Self-Managed
			(Table API)	
`time_series_database`	Amazon Timestream	Cloud Bigtable	Azure Data Explorer	InfluxDB / TimescaleDB
`graph_database`	Amazon Neptune	—	Azure Cosmos DB (Gremlin)	Neo4j
`wide_column_database`	Amazon Keyspaces	Bigtable	Azure Cosmos DB (Cassandra API)	Apache Cassandra
`search_engine`	Amazon OpenSearch	Vertex AI Search / Elasticsearch	Azure AI Search	Elasticsearch / Typesense
`data_warehouse`	Amazon Redshift	BigQuery	Azure Synapse Analytics	ClickHouse / Trino
`in_memory_cache`	Amazon ElastiCache (Redis)	Memorystore (Redis)	Azure Cache for Redis	Redis
`distributed_cache_cluster`	ElastiCache Cluster Mode	Memorystore Cluster	Azure Cache for Redis Enterprise	Redis Cluster
`object_storage`	Amazon S3	Google Cloud Storage	Azure Blob Storage	MinIO
`block_storage`	Amazon EBS	Persistent Disk	Azure Managed Disk	Ceph RBD / LVM
`file_storage`	Amazon EFS	Filestore	Azure Files	NFS / GlusterFS

Generic Block	AWS	GCP	Azure	Self-Managed
`archive_storage`	S3 Glacier	Cloud Storage Archive	Azure Archive Storage	Tape / MinIO cold tier
`message_queue`	Amazon SQS	Cloud Tasks	Azure Service Bus (Queue)	RabbitMQ
`event_streaming`	Amazon Kinesis / MSK (Kafka)	Pub/Sub	Azure Event Hubs	Apache Kafka
`pub_sub_broker`	Amazon SNS	Cloud Pub/Sub	Azure Service Bus (Topic)	RabbitMQ Exchanges
`event_bus`	Amazon EventBridge	Eventarc	Azure Event Grid	Apache Kafka + routing
`log_aggregator`	CloudWatch Logs	Cloud Logging	Azure Monitor Logs	ELK / Loki
`metrics_collector`	CloudWatch Metrics	Cloud Monitoring	Azure Monitor Metrics	Prometheus + Grafana
`distributed_tracing`	AWS X-Ray	Cloud Trace	Azure Application Insights	Jaeger / Zipkin
`audit_trail`	AWS CloudTrail	Cloud Audit Logs	Azure Activity Log	OpenTelemetry + immutable store
`alerting`	CloudWatch Alarms	Cloud Alerting	Azure Alerts	Alertmanager
`identity_provider`	Amazon Cognito	Firebase Auth / Identity Platform	Azure AD B2C	Keycloak / Auth0

Generic Block	AWS	GCP	Azure	Self-Managed
`api_auth_service`	API Gateway Authorizer / Cognito	Apigee auth	Azure API Management auth	JWT validator sidecar
`rbac_engine`	AWS IAM + Resource Policies	IAM Conditions	Azure RBAC	Open Policy Agent (OPA)
`ml_feature_store`	Amazon SageMaker Feature Store	Vertex AI Feature Store	Azure ML Feature Store	Feast
`ml_experiment_tracker`	SageMaker Experiments	Vertex AI Experiments	Azure ML Experiments	MLflow
`model_registry`	SageMaker Model Registry	Vertex AI Model Registry	Azure ML Model Registry	MLflow Registry
`vector_database`	Amazon OpenSearch (vector)	Vertex AI Vector Search	Azure AI Search (vector)	Qdrant / Weaviate / pgvector

How Product Resolution Works in the Pipeline

Product resolution is the **last stage** — it receives the final topology (all generic blocks, sized, validated) and looks up each block's product mapping.



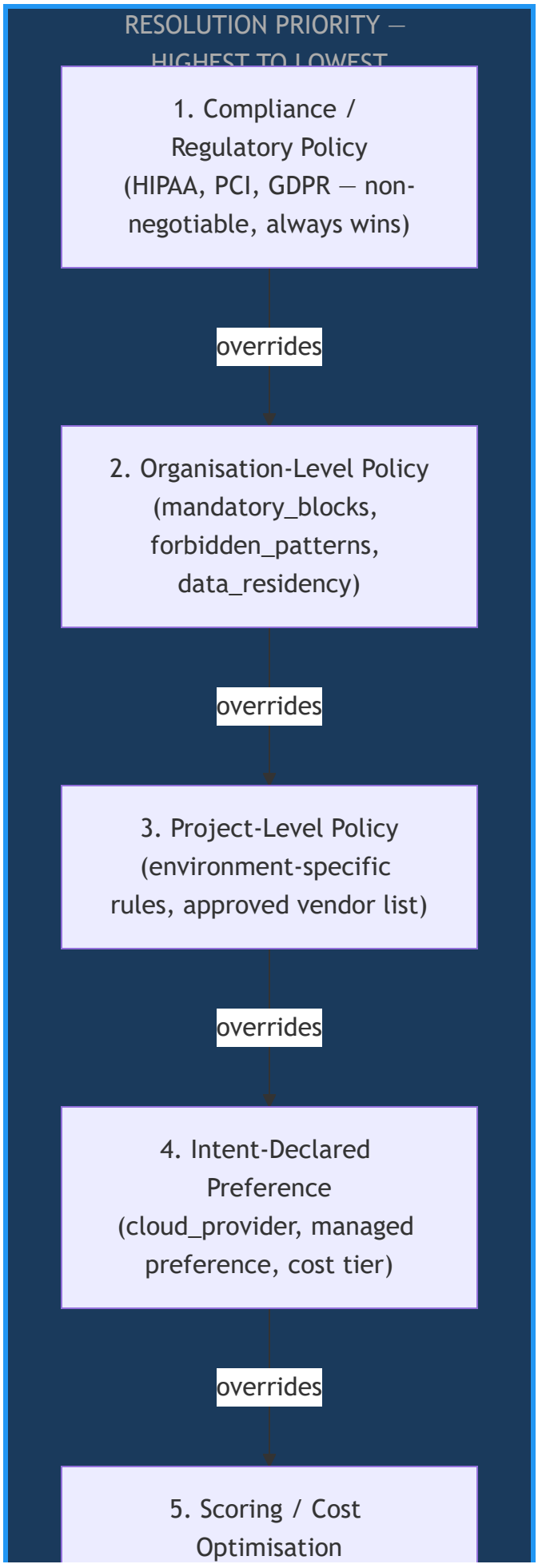
Key design principle: The Product Mapping Config Store is itself managed by the Configurator Layer. Adding support for a new cloud provider = adding new rows to the mapping config. Zero code change.

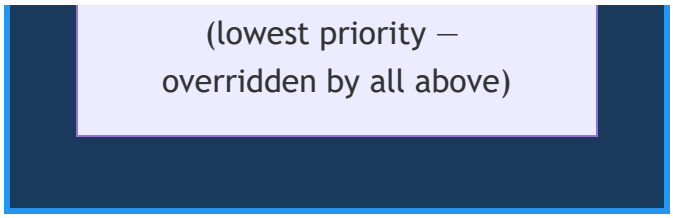
18. Collision Detection & Resolution Strategies

Four distinct collision types can arise in the pipeline. Each has a defined detection point and resolution strategy. All are data-driven — no code changes needed when new conflicts are discovered.

The Resolution Priority Hierarchy

Before diving into individual collisions, this hierarchy governs all conflict resolution. When two rules conflict, the higher-priority rule always wins:


RESOLUTION PRIORITY — HIGHEST TO LOWEST
1. Compliance / Regulatory Policy (HIPAA, PCI, GDPR — non-negotiable, always wins)
overrides
2. Organisation-Level Policy (mandatory_blocks, forbidden_patterns, data_residency)
overrides
3. Project-Level Policy (environment-specific rules, approved vendor list)
overrides
4. Intent-Declared Preference (cloud_provider, managed preference, cost tier)
overrides
5. Scoring / Cost Optimisation



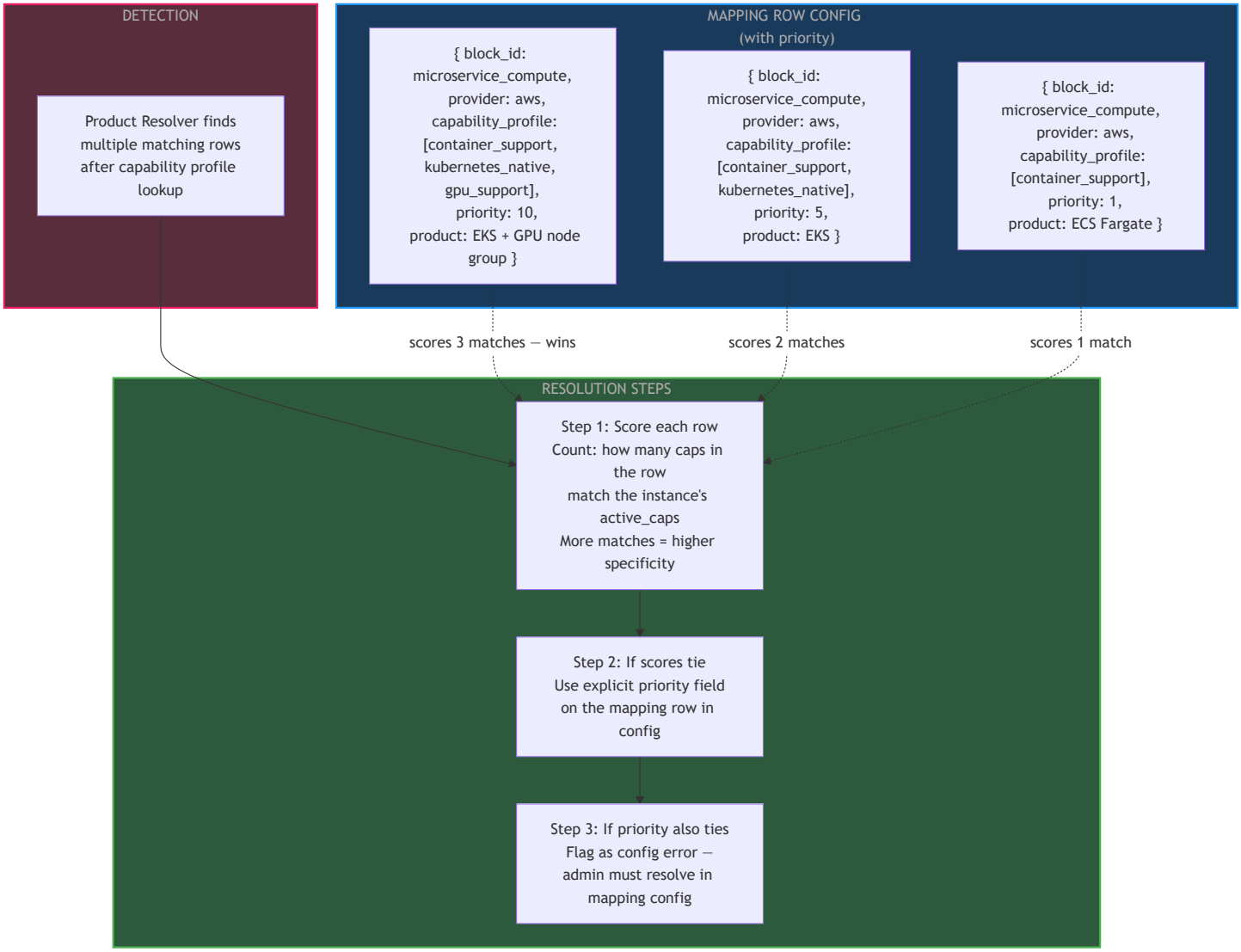
Collision 1 — Multiple Capability Profiles Match the Same Block Instance

When it happens: A block instance has an active capability set that matches more than one row in the product mapping table, with the same match length.

Example:

```
instance active_caps = [container_support, kubernetes_native, gpu_support]

Row A: [container_support, kubernetes_native]   → EKS            (2 matches)
Row B: [container_support, gpu_support]          → EKS+GPU        (2 matches)
Both rows tie on specificity score → ambiguous
```

Resolution Strategy — Specificity Score + Explicit Priority



Rule: Always define the most specific capability combination as its own row with the highest priority. The Impact Analyzer flags any gap where a reachable capability combination has no matching row.

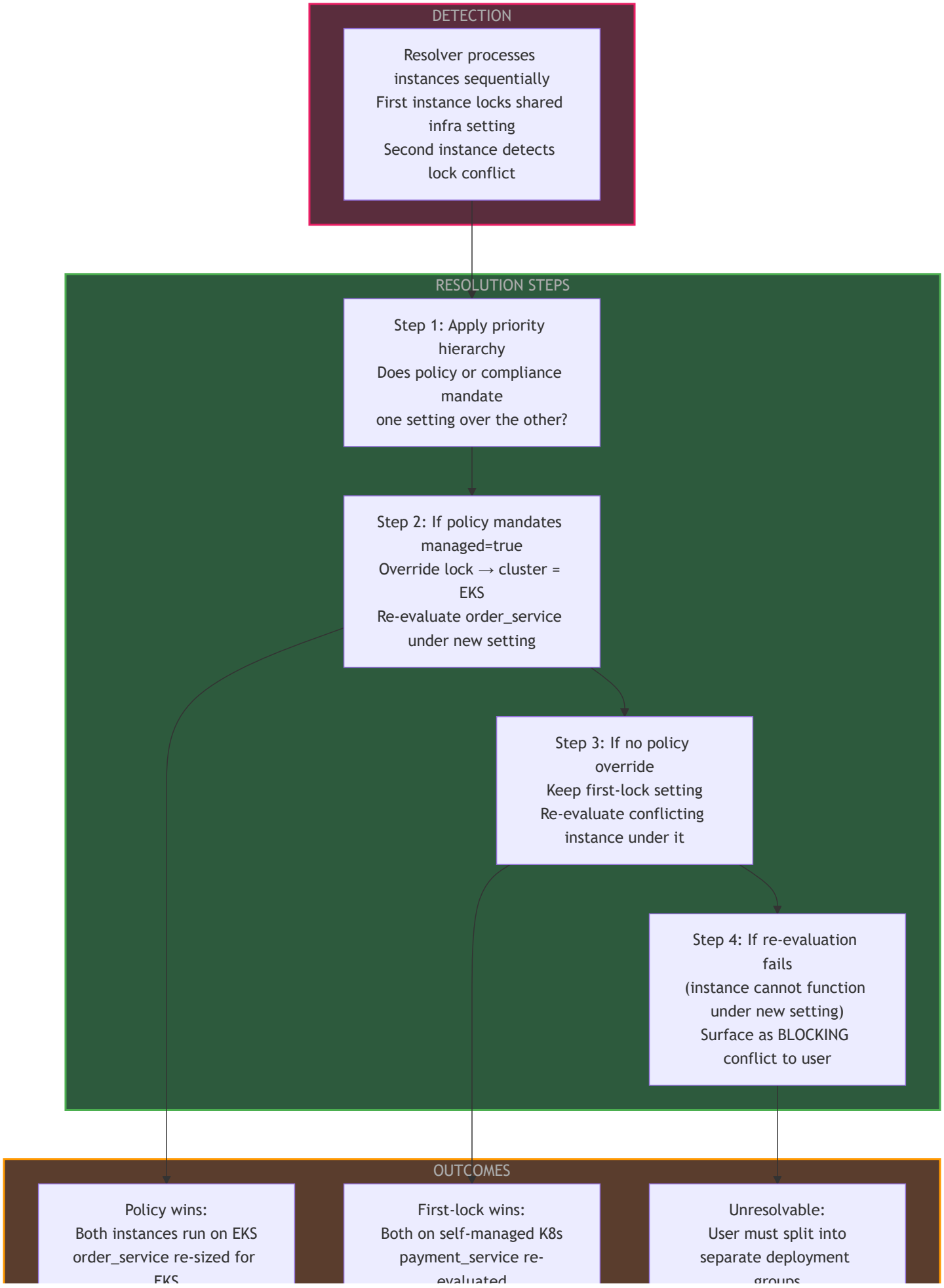
Collision 2 — Two Block Instances Conflict on Shared Infrastructure

When it happens: Two block instances (e.g., `order_service` and `payment_service`) have conflicting preferences for shared infrastructure (same VPC, same cluster).

Example:

```
order_service   → managed: false → wants self-managed K8s
payment_service → managed: true  → wants EKS
Both share one K8s cluster — cannot be both simultaneously
```

Resolution Strategy — Shared Infra Lock + Policy Hierarchy


DETECTION
Resolver processes instances sequentially
First instance locks shared infra setting
Second instance detects lock conflict
RESOLUTION STEPS
Step 1: Apply priority hierarchy
Does policy or compliance mandate one setting over the other?
Step 2: If policy mandates managed=true
Override lock → cluster = EKS
Re-evaluate order_service under new setting
Step 3: If no policy override
Keep first-lock setting
Re-evaluate conflicting instance under it
Step 4: If re-evaluation fails
(instance cannot function under new setting)
Surface as BLOCKING conflict to user
OUTCOMES
Policy wins:
Both instances run on EKS
order_service re-sized for EKS
First-lock wins:
Both on self-managed K8s
payment_service re-evaluated
Unresolvable:
User must split into separate deployment groups

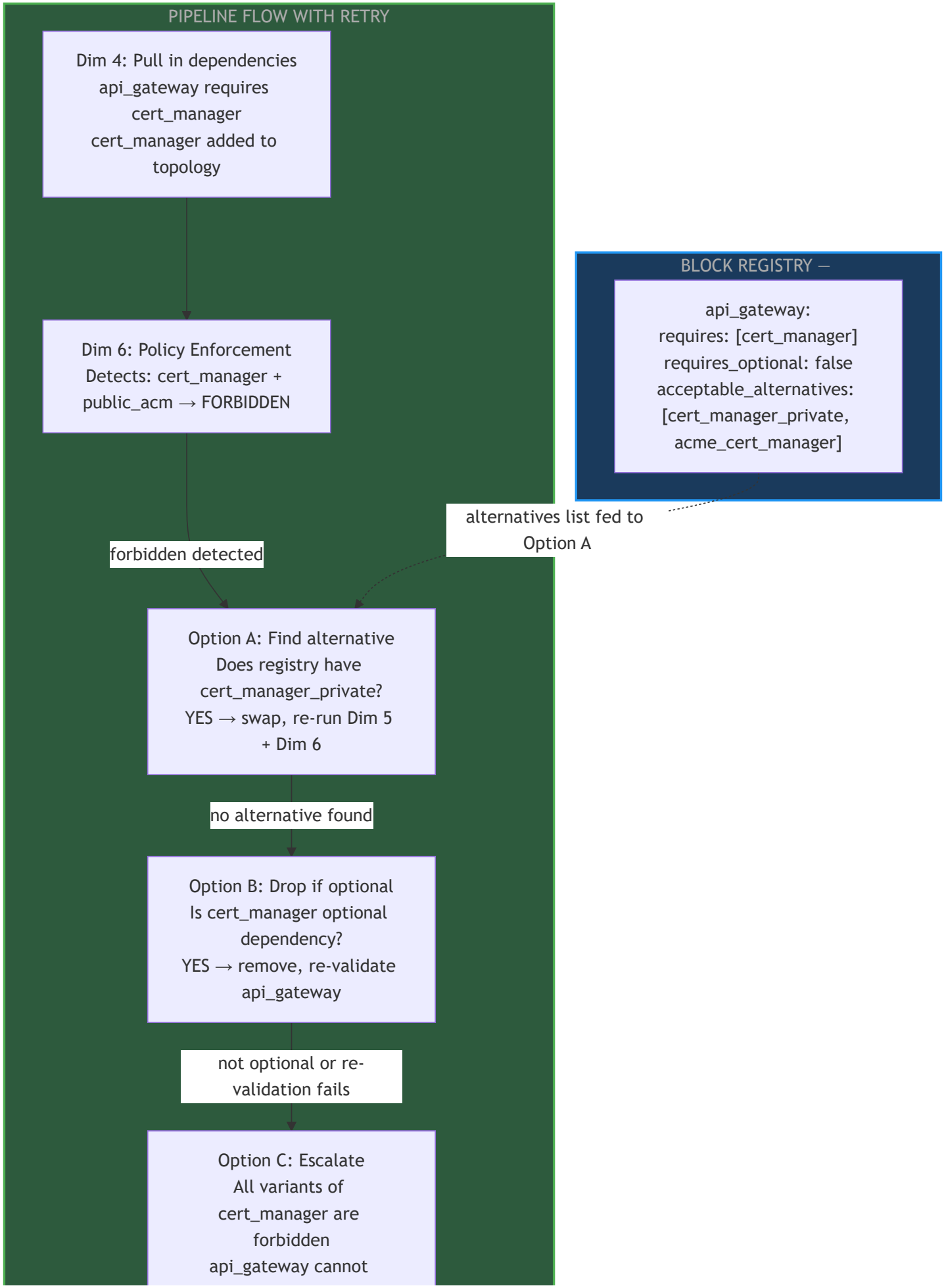
Collision 3 — Dependency Pull-In Conflicts With Forbidden Pattern

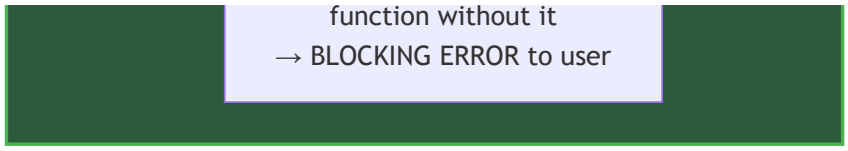
When it happens: Dimension 4 (Dependency Resolution) auto-pulls in a block. Dimension 6 (Policy Enforcement) then finds the pulled-in block violates a `forbidden_patterns` rule.

Example:

```
api_gateway (Dim 4) auto-pulls in → cert_manager
Policy: forbidden_patterns: [cert_manager + public_acm]
cert_manager resolves to ACM (public) on AWS → FORBIDDEN
```

Resolution Strategy — Post-Pull-In Re-Validation Loop


PIPELINE FLOW WITH RETRY
Dim 4: Pull in dependencies
api_gateway requires
cert_manager
cert_manager added to
topology
Dim 6: Policy Enforcement
Detects: cert_manager +
public_acm → FORBIDDEN
forbidden detected
Option A: Find alternative
Does registry have
cert_manager_private?
YES → swap, re-run Dim 5
+ Dim 6
no alternative found
Option B: Drop if optional
Is cert_manager optional
dependency?
YES → remove, re-validate
api_gateway
not optional or re-
validation fails
Option C: Escalate
All variants of
cert_manager are
forbidden
api_gateway cannot
BLOCK REGISTRY —
api_gateway:
requires: [cert_manager]
requires_optional: false
acceptable_alternatives:
[cert_manager_private,
acme_cert_manager]
alternatives list fed to
Option A



Key addition to Block Registry: Each `requires` dependency declaration must also carry an `acceptable_alternatives` list — so the retry loop has candidates to try without hardcoding alternatives in the pipeline.

Collision 4 — Mutually Exclusive Capabilities on Same Block Instance

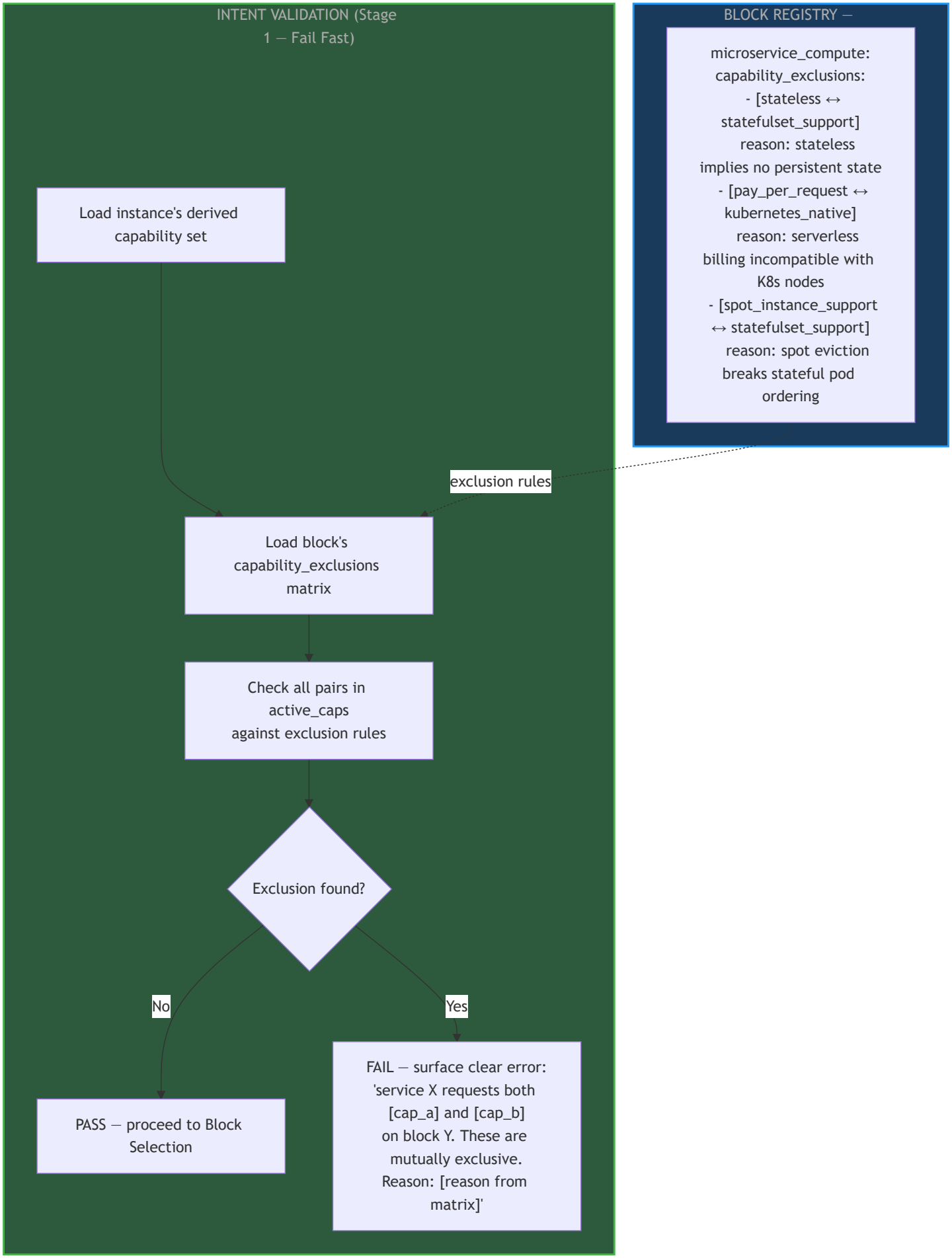
When it happens: A service's intent signals derive a capability set that contains two capabilities that are architecturally contradictory for the same block instance.

Example:

```
service declares: [stateless, statefulset_support]
stateless            = no persistent storage, free horizontal scaling
statefulset_support  = requires persistent storage, ordered pod scaling
These are contradictory on the same service
```

Resolution Strategy — Capability Compatibility Matrix at Intent Validation (Fail Fast)

This check runs at **Stage 1 — Intent Validation** — before Block Selection, before Sizing, before anything else. Cheapest place to catch it.


INTENT VALIDATION (Stage 1 — Fail Fast)
Load instance's derived capability set
Load block's capability_exclusions matrix
Check all pairs in active_caps against exclusion rules
Exclusion found?
No
Yes
PASS — proceed to Block Selection
FAIL — surface clear error: 'service X requests both [cap_a] and [cap_b] on block Y. These are mutually exclusive. Reason: [reason from matrix]'
exclusion rules
BLOCK REGISTRY —
microservice_compute:
capability_exclusions:
- [stateless ↔ statefulset_support]
reason: stateless implies no persistent state
- [pay_per_request ↔ kubernetes_native]
reason: serverless billing incompatible with K8s nodes
- [spot_instance_support ↔ statefulset_support]
reason: spot eviction breaks stateful pod ordering

Collision Summary & Detection Points

Collision	Detection Point	Resolution Strategy	Escalation If Unresolvable
1. Multiple capability profiles match	Product Resolver	Specificity score → explicit priority field on mapping row	Admin must add/fix mapping row in config
2. Instances conflict on shared infra	Shared Infra Lock during Block Instance processing	Apply priority hierarchy → re-evaluate losing instance	User must split into separate deployment groups
3. Dependency pull-in vs forbidden pattern	Dim 6 post-pull-in re-validation loop	Try alternative → drop if optional → escalate	Blocking error: no valid alternative exists
4. Mutually exclusive capabilities	Stage 1 Intent Validation (fail fast)	Reject immediately with clear message from capability_exclusions matrix	User must fix intent — no automated resolution

Required Block Registry Additions to Support Instance-Level Collision Resolution

Two new fields must be added to every block definition:

```
{
  "block_id": "microservice_compute",
  "capabilities_provided": ["container_support", "kubernetes_native", "..."],

  "capability_exclusions": [
    { "cap_a": "stateless", "cap_b": "statefulset_support",
      "reason": "stateless implies no persistent state; statefulset_support requires it" }
  ],

  "dependency_declarations": [
    {
      "block_id": "cert_manager",
      "required": true,
      "optional": false,
      "acceptable_alternatives": ["cert_manager_private", "acme_cert_manager"]
    }
  ]
}
```

Both fields are **config-driven** — managed through the Block Registry Manager UI page (Section 5). Adding a new exclusion rule or alternative = config update, zero code change.

Block-Level Collisions — Registry-Intrinsic Conflicts

The following collision types arise from **block definitions themselves**, independent of any specific intent submission. They must be detected and resolved during Block Registry validation and Dimension 3–4 processing.

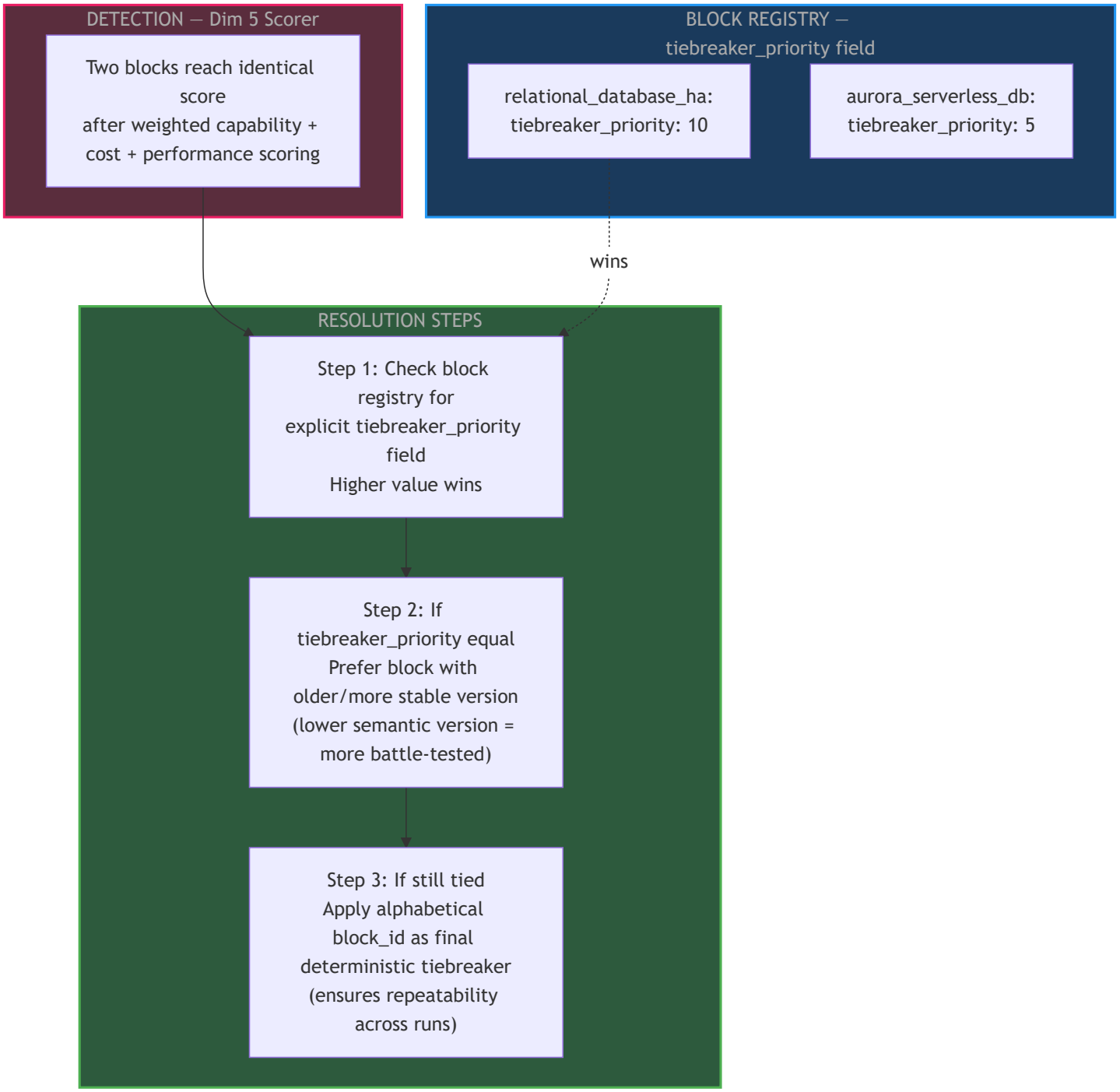
Block Collision 1 — Two Blocks Provide Identical Capabilities (Scoring Tie)

When it happens: Two blocks in the same category declare the same capability set. After Dim 3 filtering and Dim 5 scoring, their scores are equal — no deterministic winner.

Example:

```
relational_database_ha:  [sql, replication, automatic_failover]
aurora_serverless_db:    [sql, replication, automatic_failover]
After scoring with weights: both score 0.82 → tie
```

Resolution Strategy:



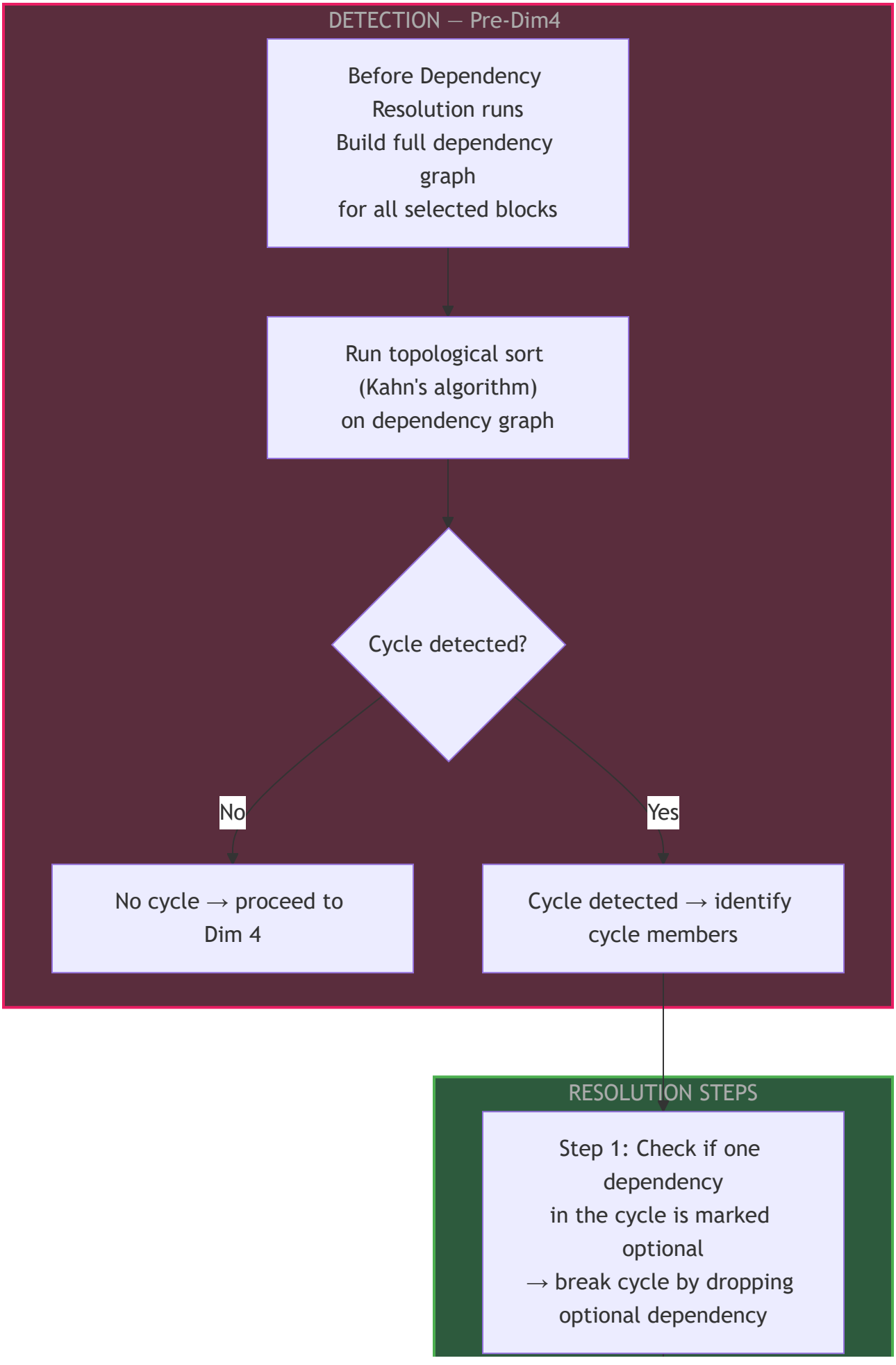
Block Collision 2 — Circular Dependency Between Blocks

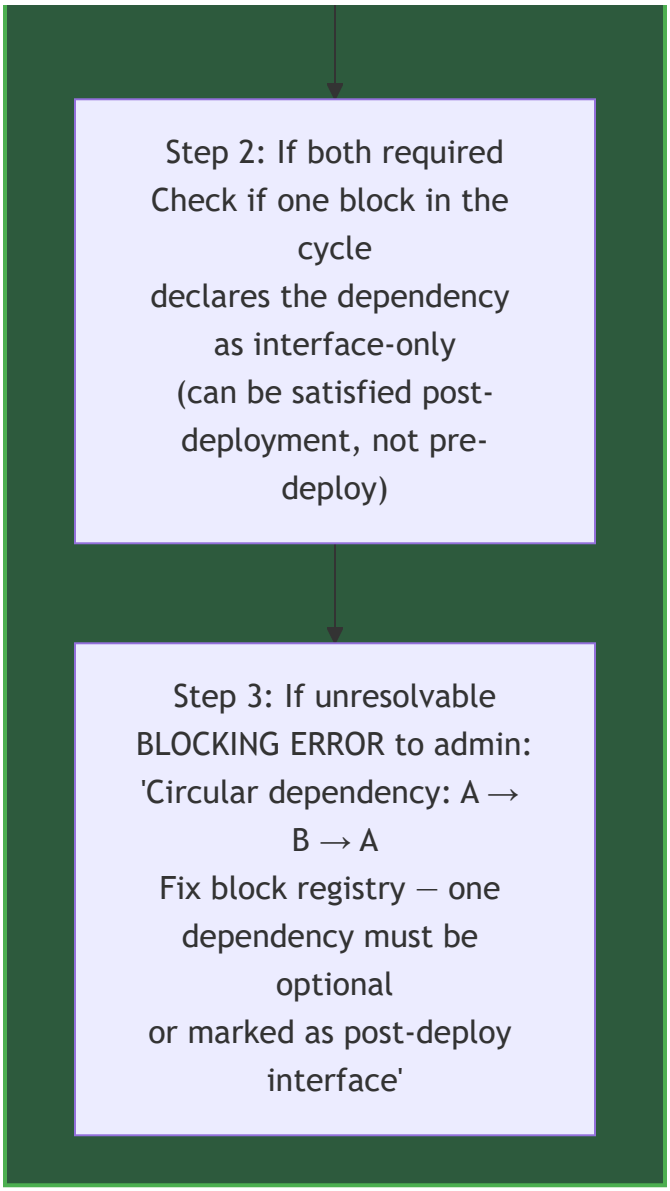
When it happens: Block A requires Block B, and Block B requires Block A (directly or transitively through a chain).

Example:

```
service_mesh  requires: api_gateway
api_gateway   requires: service_mesh   ← circular
```

Resolution Strategy — Cycle Detection Before Dim 4 Runs


DETECTION — Pre-Dim4
Before Dependency Resolution runs
Build full dependency graph
for all selected blocks
Run topological sort (Kahn's algorithm) on dependency graph
Cycle detected?
No
Yes
No cycle → proceed to Dim 4
Cycle detected → identify cycle members
RESOLUTION STEPS
Step 1: Check if one dependency in the cycle is marked optional
→ break cycle by dropping optional dependency



New block registry field: `dependency_timing: pre_deploy | post_deploy` — a `post_deploy` dependency breaks cycles because it is wired after both blocks are provisioned, not as a provisioning prerequisite.

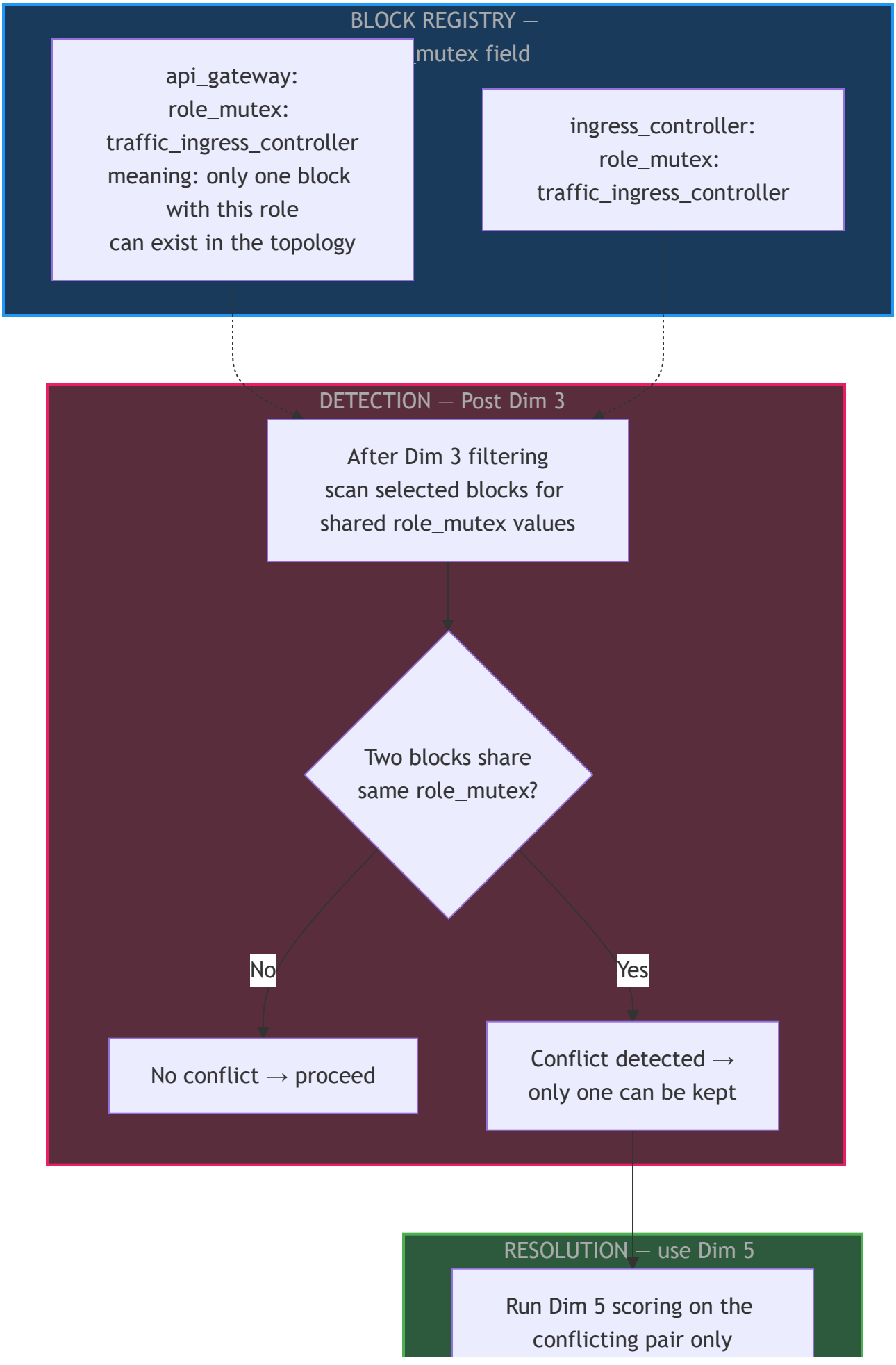
Block Collision 3 — Partial Capability Overlap (Two Blocks Compete for Same Role)

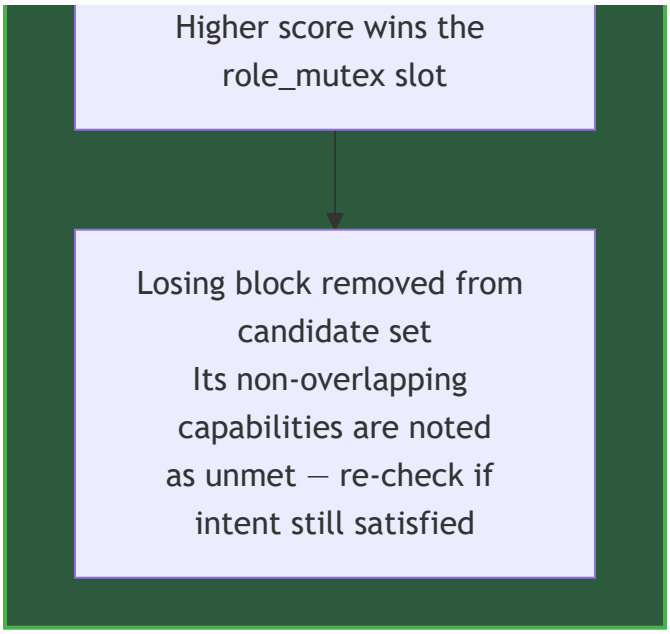
When it happens: Two different blocks both provide a capability that maps to the same architectural role (e.g., both handle incoming traffic). Selecting both would create conflicting entry points.

Example:

```
api_gateway:        capabilities: [request_routing, rate_limiting, auth_integration]
ingress_controller: capabilities: [request_routing, tls_termination, path_routing]
Both provide request_routing → both survive Dim 3 filter
Both selected → two competing traffic entry points in topology
```

Resolution Strategy — Role Mutex Declaration in Block Registry


BLOCK REGISTRY —
mutex field
api_gateway:
role_mutex:
traffic_ingress_controller
meaning: only one block
with this role
can exist in the topology
ingress_controller:
role_mutex:
traffic_ingress_controller
DETECTION — Post Dim 3
After Dim 3 filtering
scan selected blocks for
shared role_mutex values
Two blocks share
same role_mutex?
No
Yes
No conflict → proceed
Conflict detected →
only one can be kept
RESOLUTION — use Dim 5
Run Dim 5 scoring on the
conflicting pair only



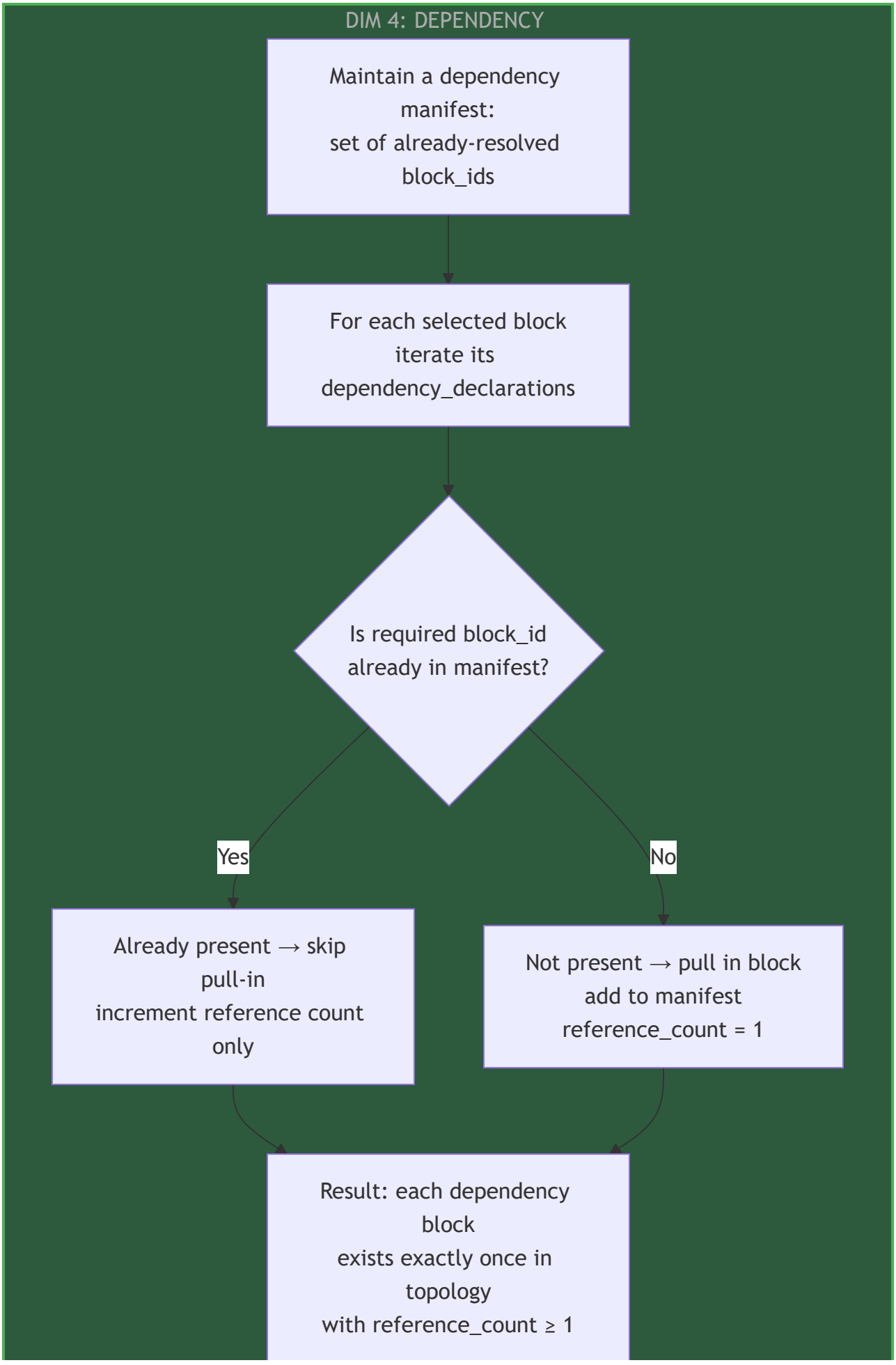
Block Collision 4 — Duplicate Dependency Pull-In

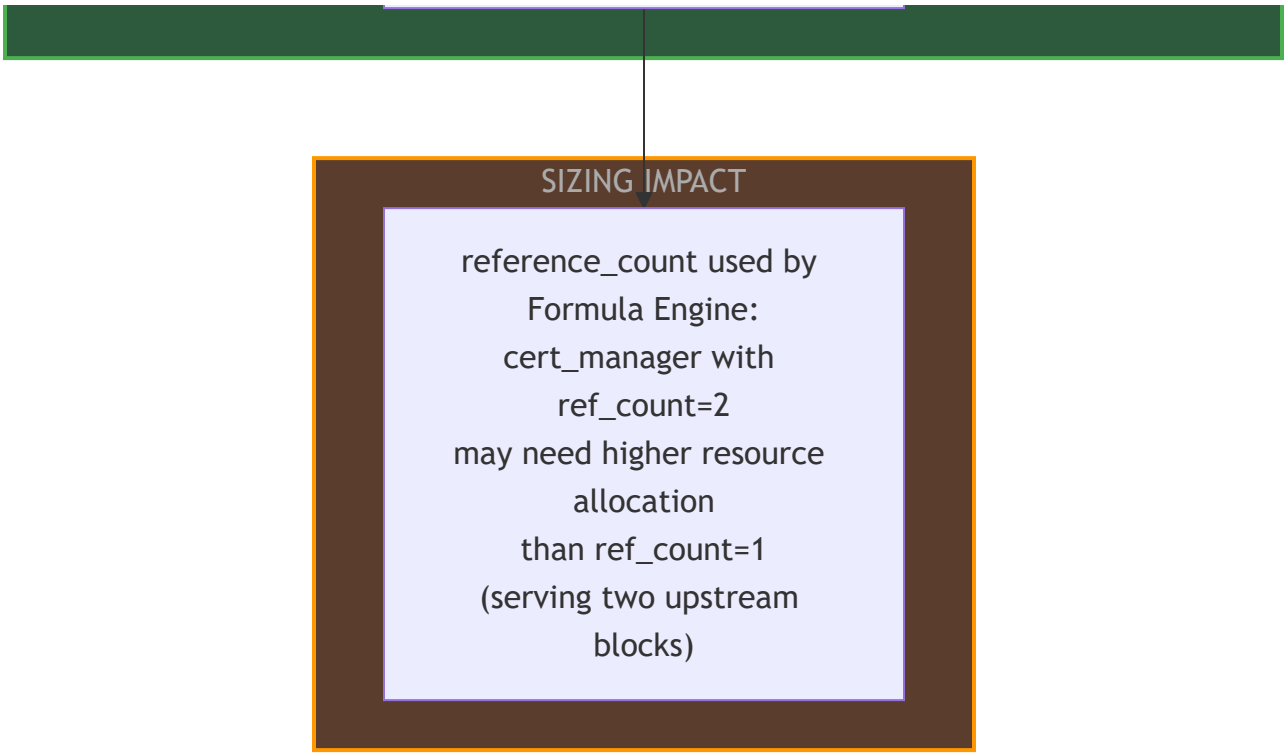
When it happens: Two or more selected blocks both declare the same required dependency. Dim 4 processes each block independently and pulls in the dependency twice.

Example:

```
api_gateway     requires: cert_manager
tls_termination requires: cert_manager
→ cert_manager added twice to topology
```

Resolution Strategy — Deduplication in Dim 4


DIM 4: DEPENDENCY
Maintain a dependency manifest:
set of already-resolved block_ids
For each selected block
iterate its dependency_declarations
Is required block_id already in manifest?
Yes
No
Already present → skip pull-in
increment reference count only
Not present → pull in block
add to manifest
reference_count = 1
Result: each dependency block
exists exactly once in topology
with reference_count ≥ 1



Key addition: `reference_count` is passed to the Formula Engine so shared dependencies can be sized correctly — a `cert_manager` serving 3 blocks may need more resources than one serving 1 block.

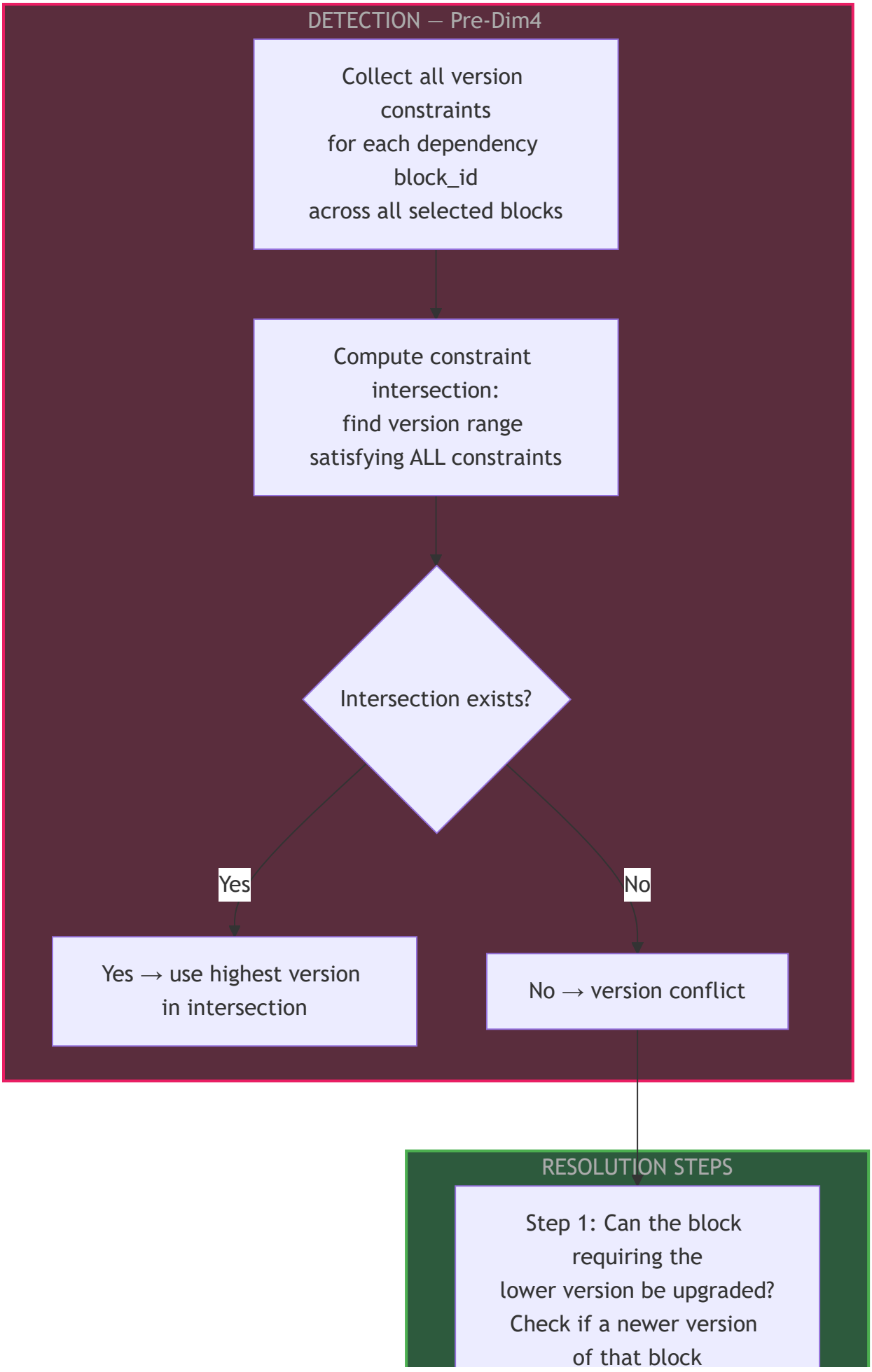
Block Collision 5 — Version Conflict Between Two Blocks' Dependency Requirements

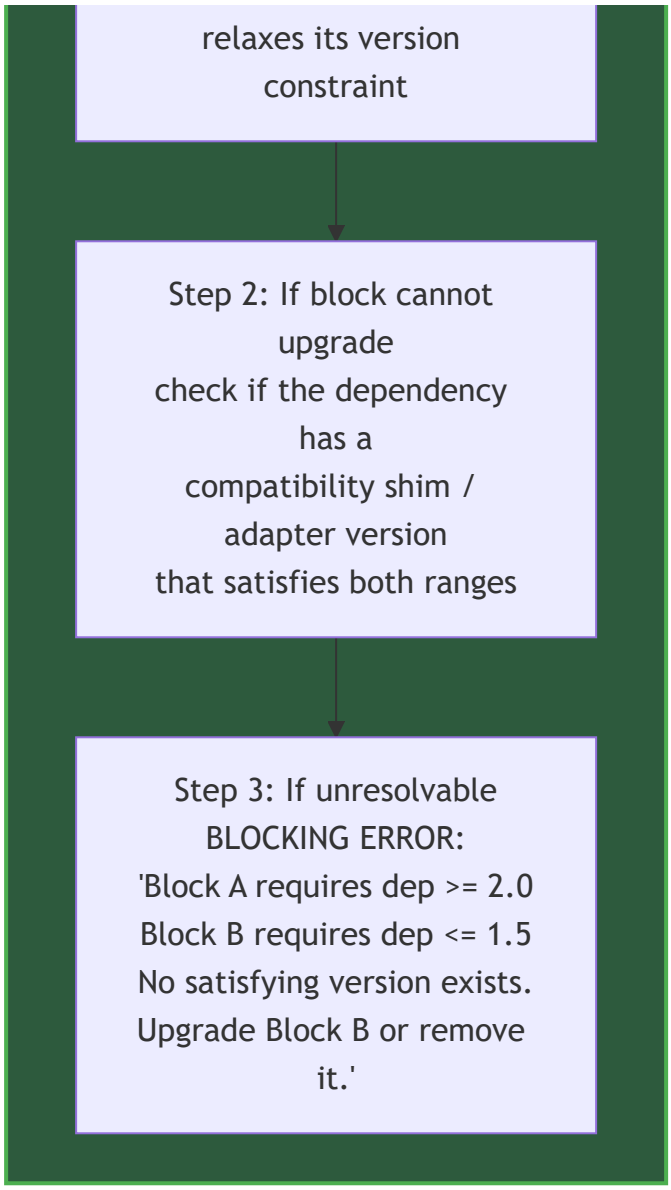
When it happens: Block A and Block B are both selected. Block A requires dependency D at version `>= 2.0`, Block B requires the same dependency D at version `<= 1.5`. No single version satisfies both.

Example:

```
microservice_compute v2.1  requires: ingress_controller >= 2.0
legacy_api_adapter   v1.0  requires: ingress_controller <= 1.5
Both selected → ingress_controller cannot satisfy both version constraints
```

Resolution Strategy — Version Constraint Solver


DETECTION — Pre-Dim4
Collect all version constraints for each dependency block_id across all selected blocks
Compute constraint intersection: find version range satisfying ALL constraints
Intersection exists?
Yes
No
Yes → use highest version in intersection
No → version conflict
RESOLUTION STEPS
Step 1: Can the block requiring the lower version be upgraded? Check if a newer version of that block



Complete Collision Summary (Instance-Level + Block-Level)

#	Collision	Type	Detection Point	Resolution	Escalation
1	Multiple capability profiles match	Instance-level	Product Resolver	Specificity score → priority field	Admin fixes mapping config
2	Instances conflict on shared infra	Instance-level	Shared Infra Lock	Priority hierarchy → re-evaluate	User splits deployment groups

#	Collision	Type	Detection Point	Resolution	Escalation
3	Dependency pull-in vs forbidden pattern	Instance-level	Dim 6 post-pull-in loop	Try alternative → drop optional	Blocking error
4	Mutually exclusive capabilities	Instance-level	Stage 1 Validation	Fail fast from capability_exclusions	User fixes intent
5	Two blocks tie in scoring	Block-level	Dim 5 Scorer	tiebreaker_priority → version → alphabetical	Always deterministic
6	Circular dependency	Block-level	Pre-Dim4 cycle check	Drop optional dep → post_deploy timing	Admin fixes block registry
7	Partial capability overlap / role conflict	Block-level	Post Dim 3 Filter	role_mutex → score winner keeps role	Re-check unmet caps
8	Duplicate dependency pull-in	Block-level	Dim 4 manifest	Deduplication + reference_count	None — always resolved
9	Version conflict on shared dependency	Block-level	Pre-Dim4 version solver	Find intersecting version → upgrade check	Blocking error

Full Block Registry Shape (All Collision-Support Fields)

```
{
  "block_id": "api_gateway",
  "category": "api_ingress",
  "version": "2.1.0",
  "tiebreaker_priority": 10,
  "role_mutex": "traffic_ingress_controller",
  "capabilities_provided": ["request_routing", "rate_limiting", "auth_integration"],
  "capability_exclusions": [
    { "cap_a": "stateless", "cap_b": "statefulset_support",
      "reason": "contradictory state models" }
  ],
  "dependency_declarations": [
    {
      "block_id": "cert_manager",
      "version_constraint": ">= 1.8.0",
      "required": true,
      "optional": false,
      "dependency_timing": "pre_deploy",
      "acceptable_alternatives": ["cert_manager_private", "acme_cert_manager"]
    },
    {
      "block_id": "service_mesh",
      "version_constraint": ">= 1.15.0",
      "required": false,
      "optional": true,
      "dependency_timing": "post_deploy"
    }
  ]
}
```

All fields are **config-driven** — managed through the Block Registry Manager UI (Section 5). Zero code change for any new collision rule.

19. Intent JSON Builder — Chatbot Question Strategy

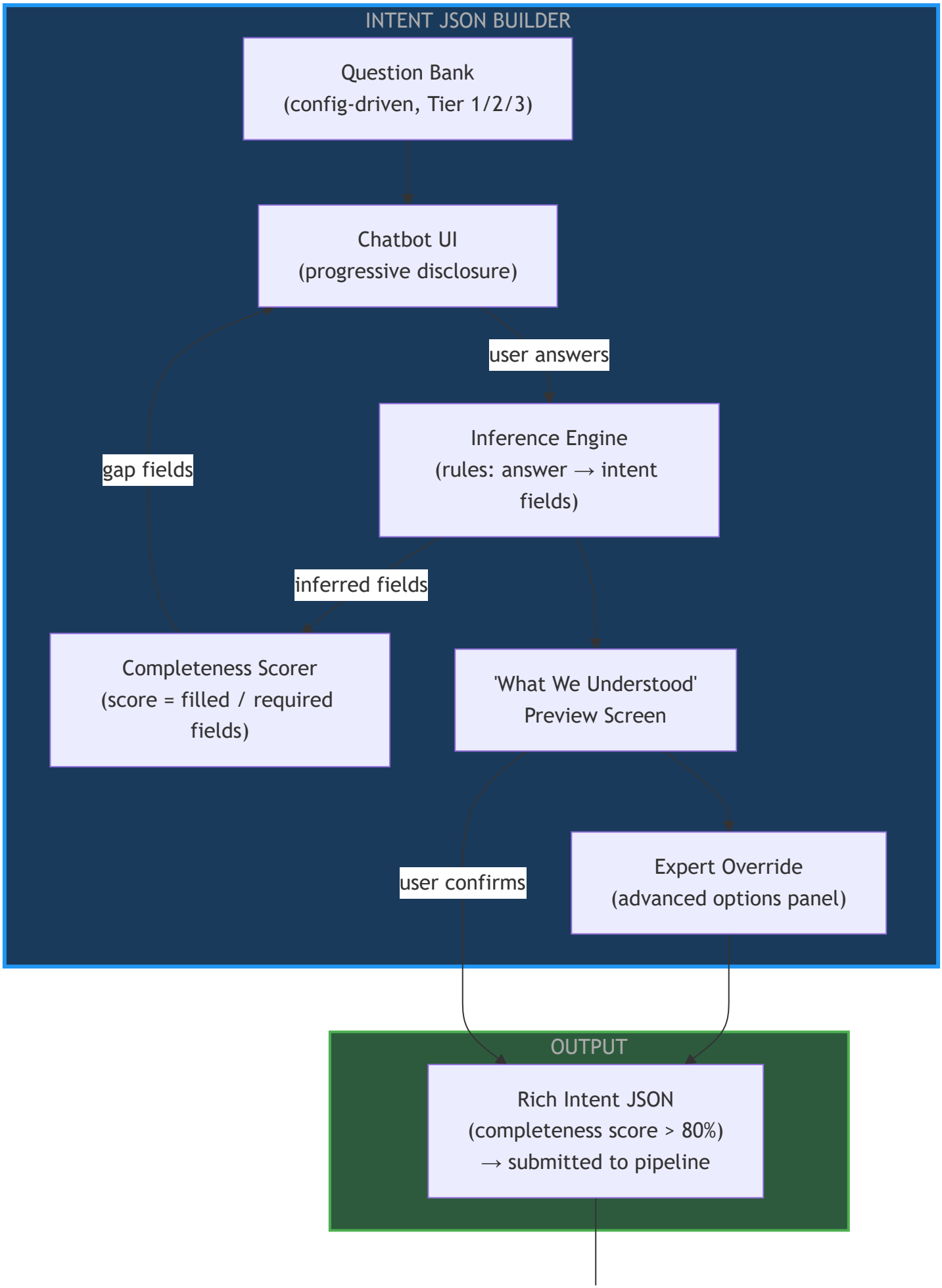
The pipeline starts at "User submits Intent JSON" — but the current doc never describes **how that JSON is built**. In practice, users are not architects who know to fill `expected_rps_peak` ,

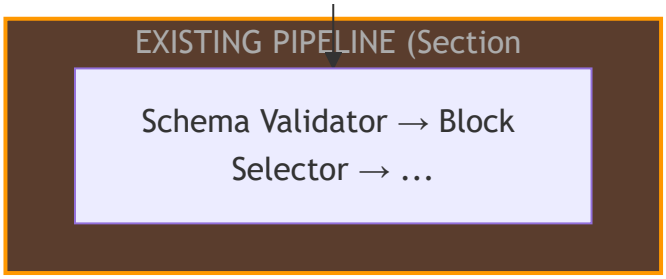
`data_residency` , and `optimization_priorities` . They answer friendly questions in a chatbot. This section defines the strategy for turning 5-10 simple user answers into a complete, pipeline-ready Intent JSON.

The Core Tension

```
User wants:   5-10 simple, friendly questions
System needs: 25-30 technical signal fields for a mature topology

Solution:     Ask simple → Infer complex
              One friendly question fills 4-6 technical fields via inference rules
```

Architecture of the Intent JSON Builder


INTENT JSON BUILDER
Question Bank
(config-driven, Tier 1/2/3)
Chatbot UI
(progressive disclosure)
user answers
Inference Engine
(rules: answer → intent
fields)
gap fields
inferred fields
Completeness Scorer
(score = filled / required
fields)
'What We Understood'
Preview Screen
user confirms
Expert Override
(advanced options panel)
OUTPUT
Rich Intent JSON
(completeness score > 80%)
→ submitted to pipeline



Question Bank — Three Tiers

Questions are **config-driven** — stored in the Question Bank, managed by admins. No code change to add new questions or modify inference rules.

Tier 1 — Always Ask (5-6 questions, mandatory for all users)

These cover ~70% of intent signals. Every user answers these regardless of app type:

#	Question (User-Friendly)	Intent Fields Filled Directly
1	"What kind of app are you building?" *(web app / mobile API / data pipeline / ML system / event-driven system / microservices)*	`application_type` , `interaction_model`
2	"How many users or requests do you expect at your busiest moment?" *(< 100 / 100-1000 / 1000-10000 / 10000+ / not sure)*	`expected_rps_peak` , `horizontal_scaling_required` , `availability_target`
3	"Will users log in? How?" *(no login / username + password / social login / company SSO / all of these)*	`authentication_required` , `auth_type`
4	"What kind of data does your app store?" *(user profiles / financial transactions / health records / general content / no sensitive data)*	`data_types` , `compliance[]`
5	"Where will your users be and where should data be stored?" *(single country / multiple countries / specific region for legal reasons)*	`multi_region_required` , `data_residency`
6	"What matters most to you?" *(drag to rank: save cost / best performance / meet compliance / keep it simple)*	`optimization_priorities` weights

Tier 2 — Conditionally Ask (0-4 questions, triggered by Tier 1 answers)

Only shown when a Tier 1 answer opens a new dimension:

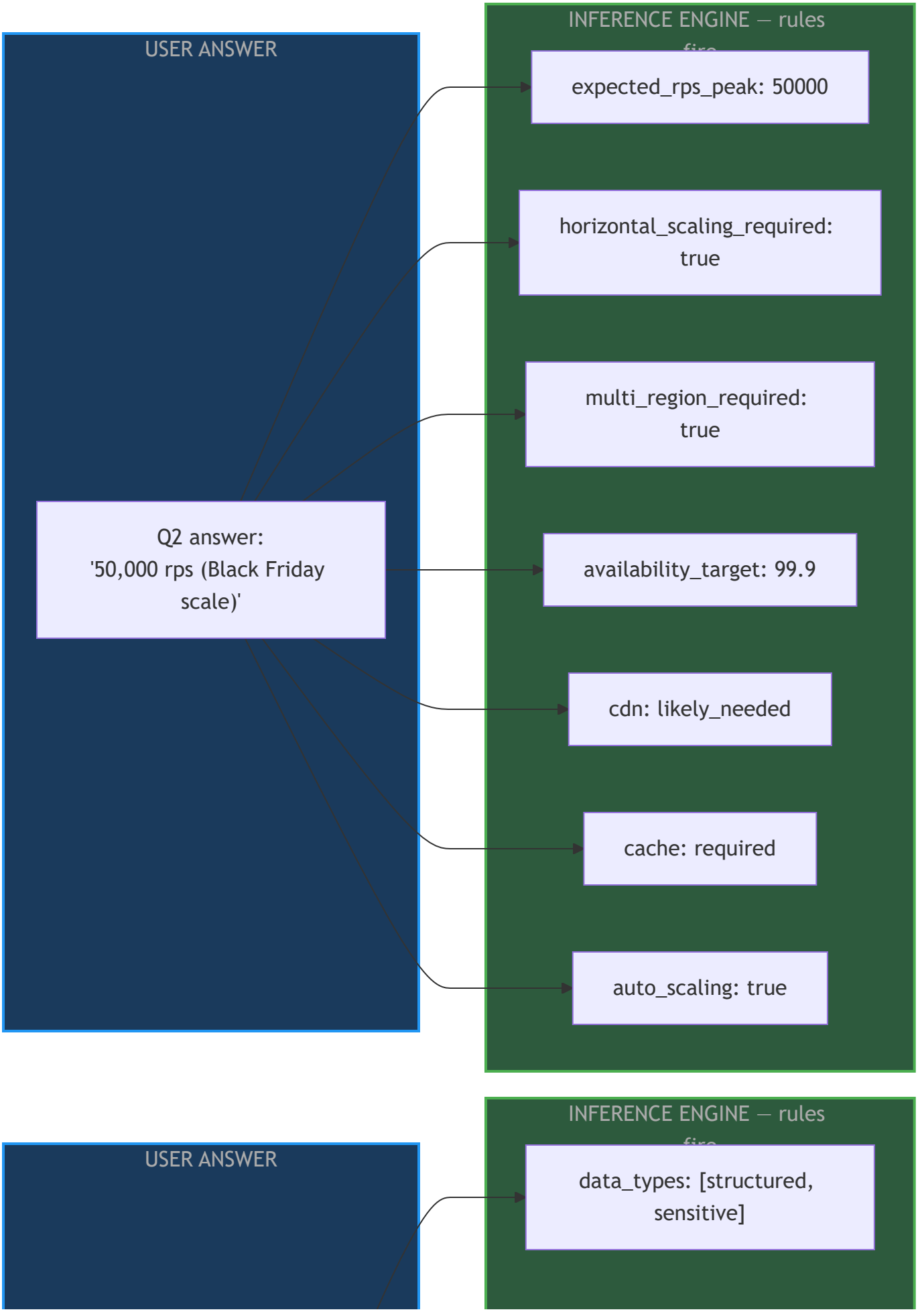
Trigger Condition	Follow-Up Question
`application_type = ml_pipeline`	"Is this for training new models, serving predictions, or both?"
`data_types includes financial`	"Do you need PCI DSS compliance for card payments?"
`data_types includes health`	"Do you need HIPAA compliance for patient data?"
`multi_region_required = true`	"If one region fails, should the app keep running automatically?"
`expected_rps > 10000`	"Does your traffic spike suddenly (like a flash sale) or grow gradually?"
`application_type = microservices`	"How many services are you building? Can you name them briefly?"
`auth_type = enterprise_sso`	"Which identity provider does your company use? (Okta / Azure AD / Google / other)"

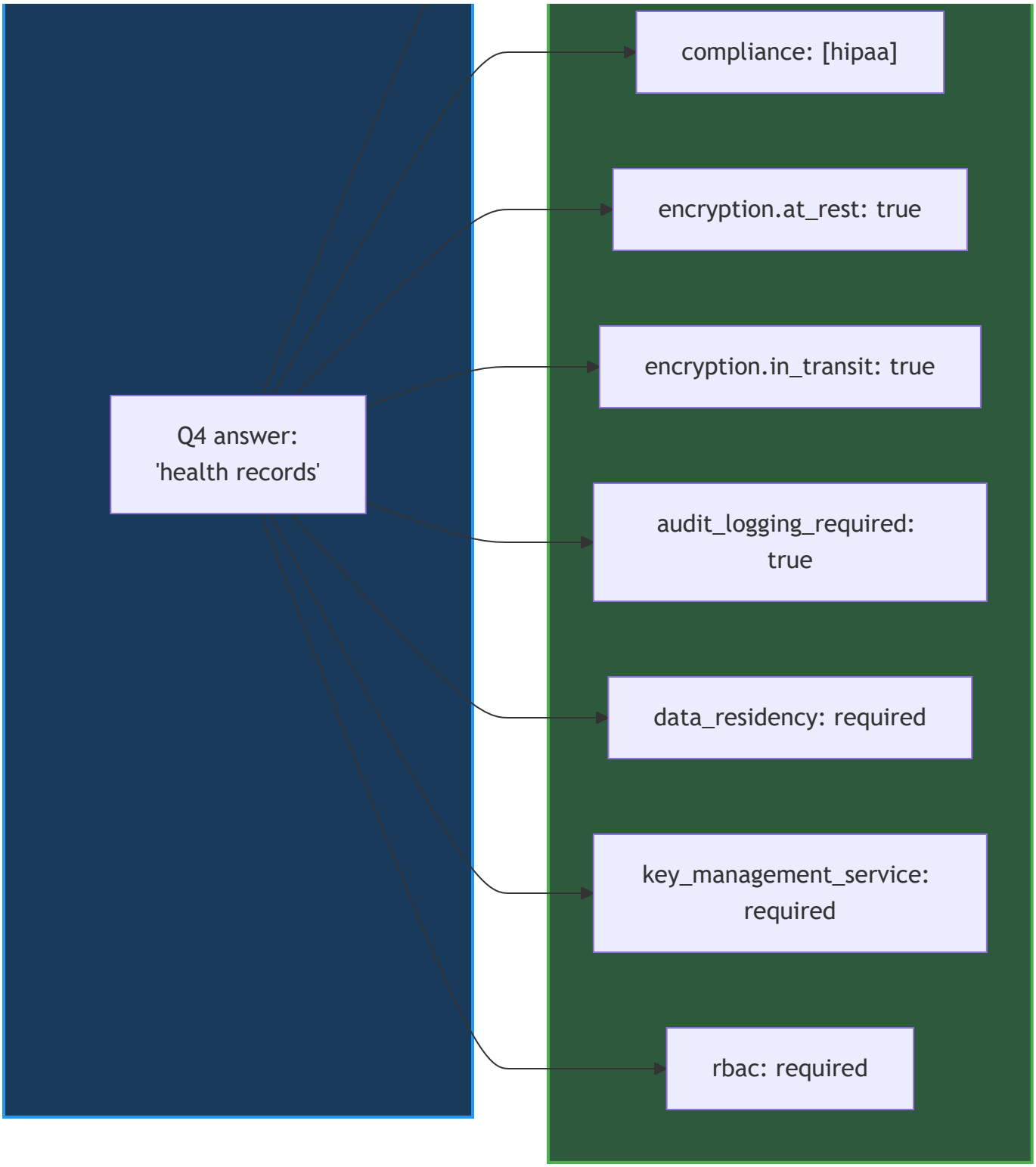
Tier 3 — Expert Override (hidden by default, optional)

An "Advanced Options" panel pre-populated with all inferred values. Power users can adjust. Non-technical users never see it.

Inference Rules Engine

After each Tier 1/2 answer, the Inference Engine fires declarative rules to fill downstream intent fields. Rules are **config-driven** — admins add/modify rules without code changes.


USER ANSWER
Q2 answer:
'50,000 rps (Black Friday
scale)'
INFERENCE ENGINE — rules
fire
expected_rps_peak: 50000
horizontal_scaling_required:
true
multi_region_required:
true
availability_target: 99.9
cdn: likely_needed
cache: required
auto_scaling: true
USER ANSWER
INFERENCE ENGINE — rules
fire
data_types: [structured,
sensitive]



Inference Rule Config Shape:

```
{
  "rule_id": "health_data_implies_hipaa",
  "trigger": { "field": "data_types", "contains": "health_records" },
  "infer": [
    { "field": "compliance", "append": "hipaa" },
    { "field": "encryption.at_rest", "set": true },
    { "field": "encryption.in_transit", "set": true },
    { "field": "audit_logging_required", "set": true },
    { "field": "key_management_service_required", "set": true }
  ],
  "confidence": "certain"
}
```

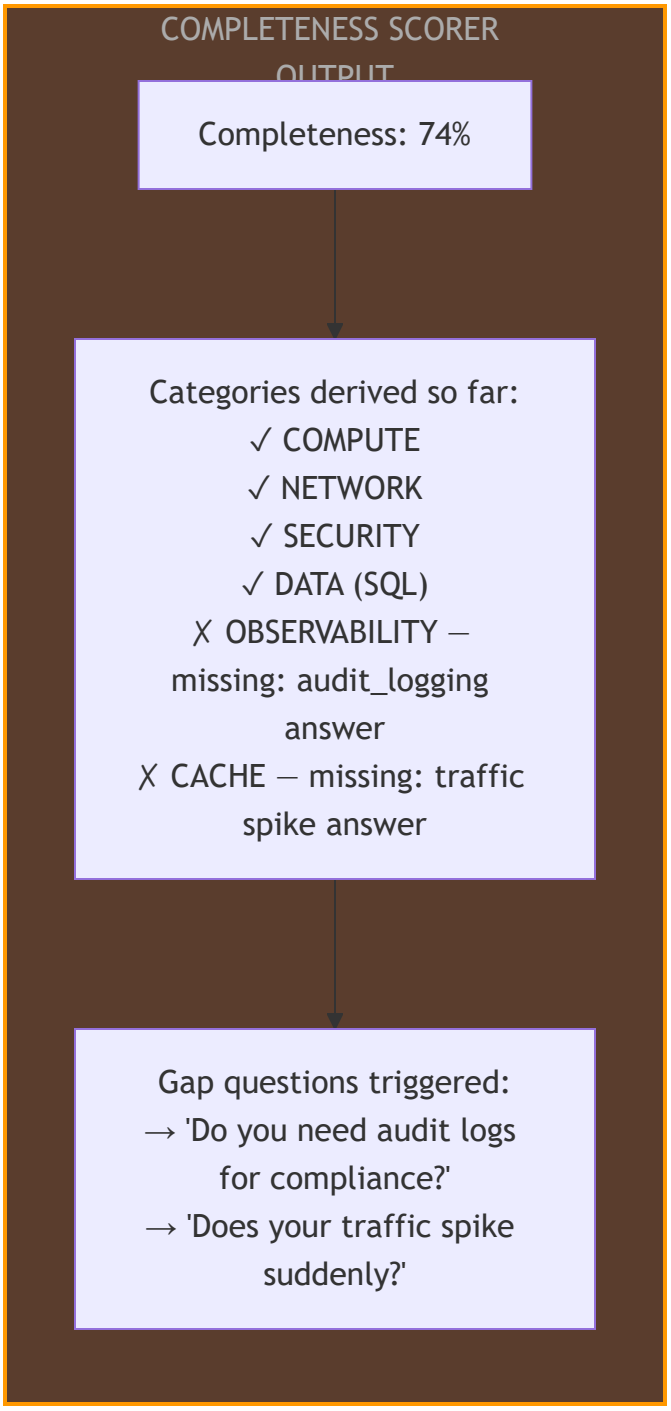
`confidence` field: `certain` (always set) / `likely` (set as default, user can override) / `possible` (suggest in preview, user must confirm).

Completeness Scorer

Runs after every question answer. Calculates what percentage of required intent fields for this `application_type` are filled:

```
Score = (filled_required_fields / total_required_fields_for_app_type) × 100

< 60%  → BLOCK submission — ask gap-filling questions
60-80% → WARN — show gaps, allow submission with warnings
> 80%  → PASS — proceed with high confidence
```

The scorer also shows the user a **live preview of what categories will be derived** — so they see the impact of their answers in real time:



"What We Understood" Preview Screen

After all questions, before submitting to the pipeline, the user sees a plain-English summary of all inferred intent fields — grouped as **confirmed**, **assumed defaults**, and **gaps**:

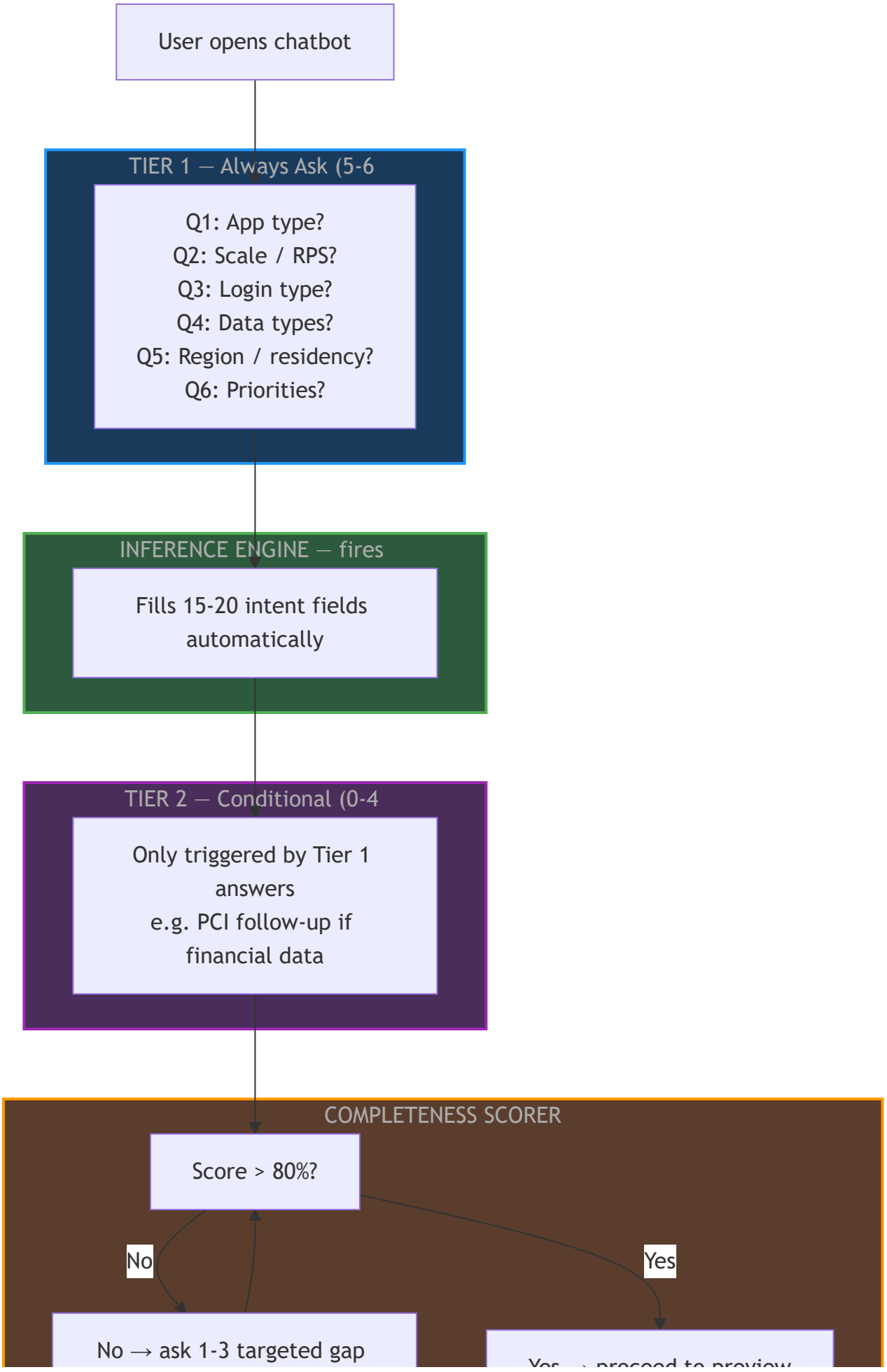
✓ CONFIRMED (from your answers):
  Web app with OAuth2 login
  Financial data → PCI compliance applied
  50,000 peak RPS → auto-scaling, CDN, Redis cache included
  EU users → data residency: eu-west enforced

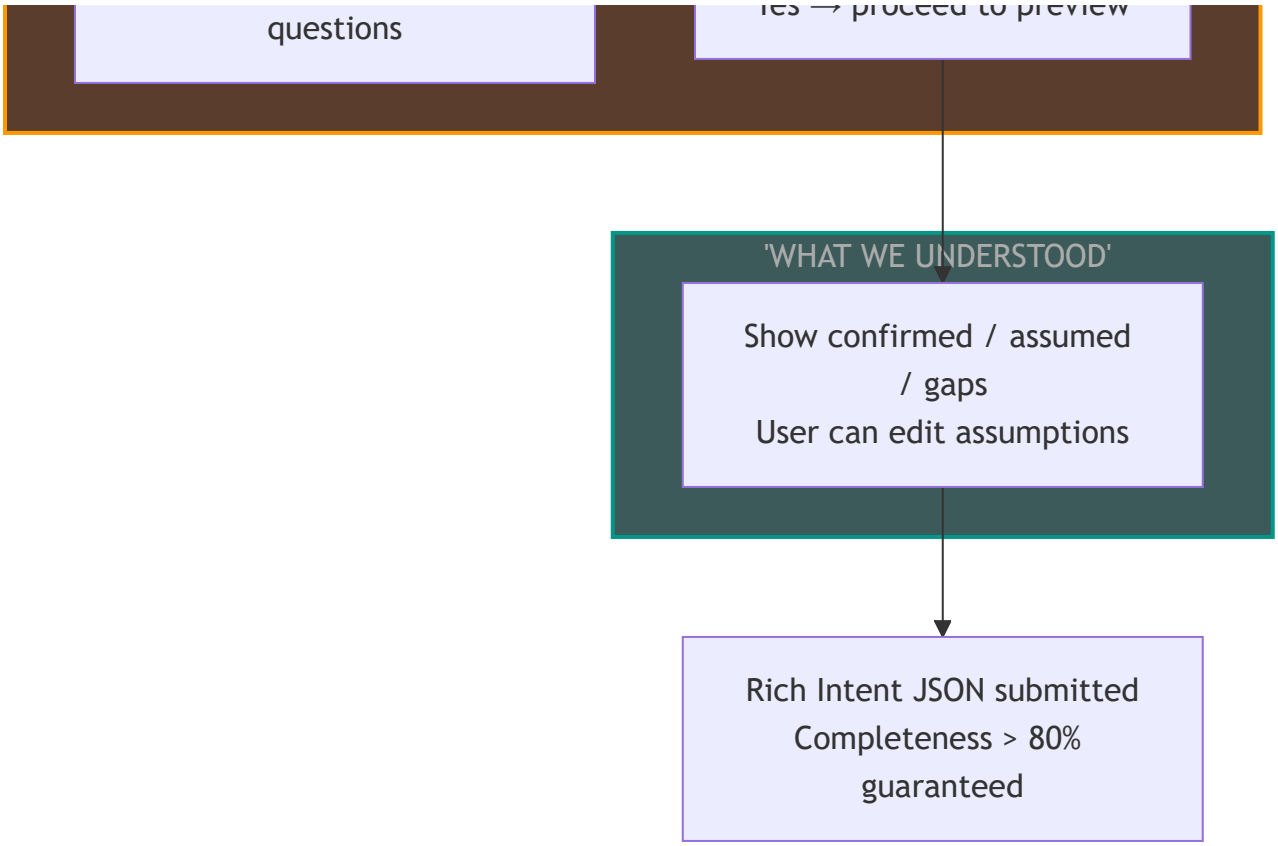
~ ASSUMED DEFAULTS (we filled these for you — you can change them):
  Encryption at rest: ON   (required by PCI)
  Audit logging: ON   (required by PCI)
  High availability: ON   (inferred from 50k RPS)
  Multi-region: ON   (inferred from 50k RPS + HA)

⚠ GAPS (we couldn't infer — using safe defaults):
  Real-time processing: assumed NO
  ML components: assumed NONE

[Confirm & Generate Architecture]   [Edit assumptions]

Complete Intent Builder Flow


User opens chatbot
TIER 1 — Always Ask (5-6
Q1: App type?
Q2: Scale / RPS?
Q3: Login type?
Q4: Data types?
Q5: Region / residency?
Q6: Priorities?
INFERENCE ENGINE — fires
Fills 15-20 intent fields automatically
TIER 2 — Conditional (0-4
Only triggered by Tier 1 answers
e.g. PCI follow-up if financial data
COMPLETENESS SCORER
Score > 80%?
No
Yes
No → ask 1-3 targeted gap



Intent Builder as a Configurator Component

The Question Bank and Inference Rules are managed through the Configurator Layer — same as blocks and policies:

Config Entity	Managed By	What It Controls
`Question Bank`	Admin UI — new "Intent Builder" page	Tier 1/2/3 questions, their wording, answer options
`Inference Rules`	Admin UI — Intent Builder page	answer → intent field mappings, confidence levels
`Completeness Thresholds`	Admin UI	Score thresholds per `application_type`
`Preview Templates`	Admin UI	Plain-English summary templates per app type

Zero code change when a new question is needed or an inference rule is updated.

20. Under-Selection Safety Nets

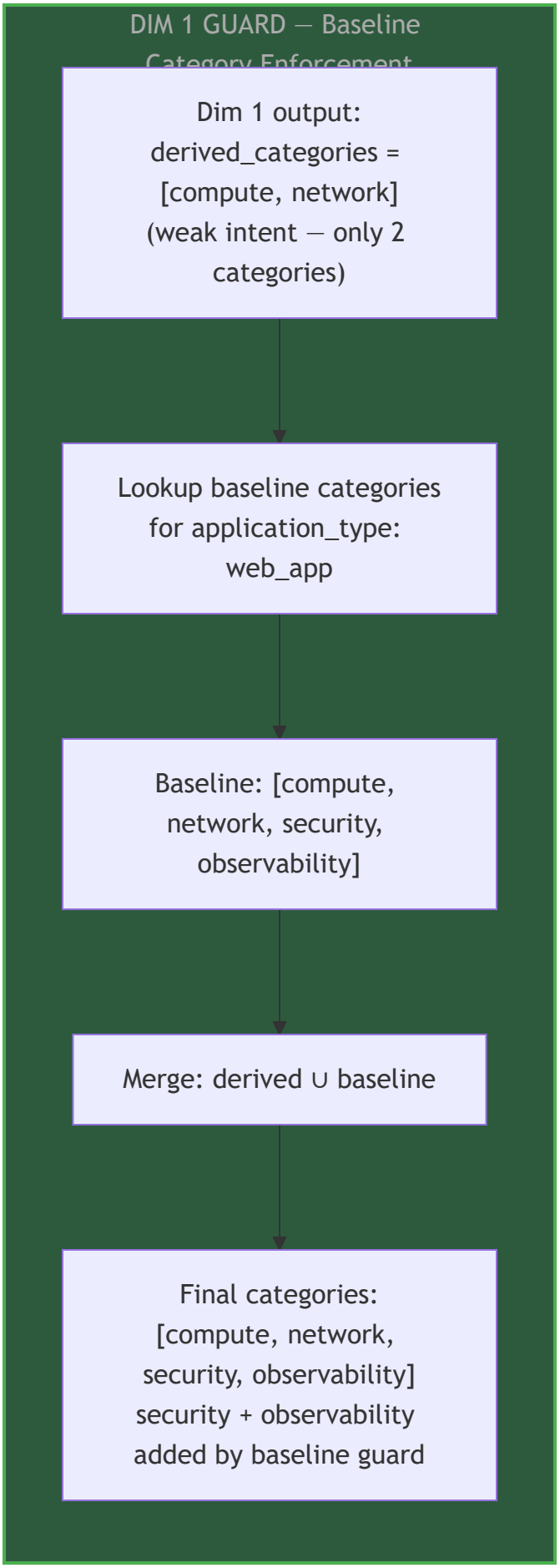
Even with a high-quality intent JSON from Section 19, edge cases remain where the pipeline can produce an immature or empty topology. Three safety nets catch these at different stages.

Safety Net 1 — Baseline Category Defaults (Dim 1 Guard)

Problem: Weak or minimal intent → Dim 1 derives too few categories → critical architectural layers missing entirely.

Solution: After Dim 1 runs, enforce a **minimum category floor** based on `application_type`. These categories are always included regardless of intent signals:

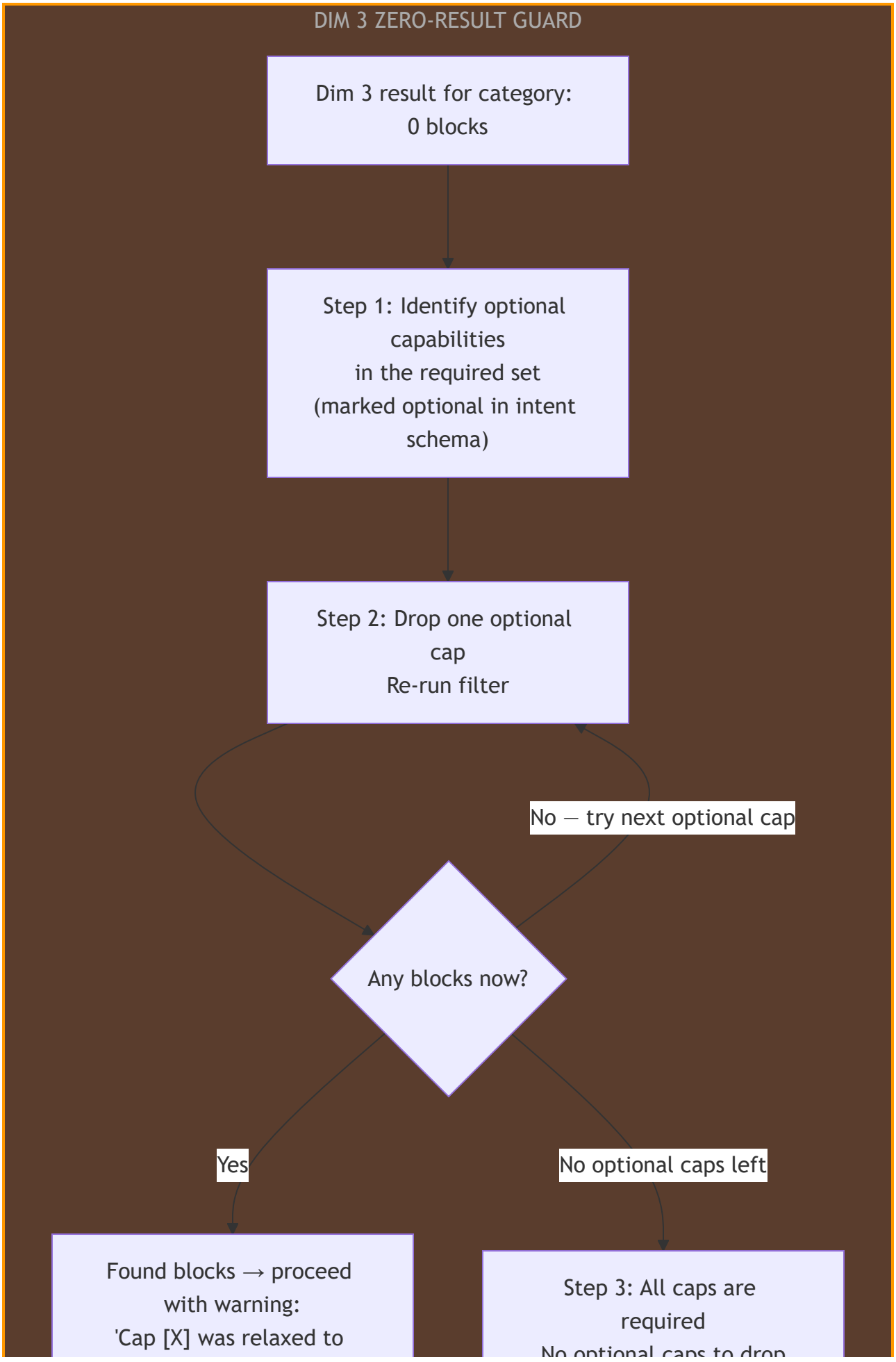
application_type	Mandatory Baseline Categories (always derived)
`web_app`	`compute`, `network`, `security`, `observability`
`api_service`	`compute`, `network`, `security`, `api_ingress`, `observability`
`microservices`	`compute`, `network`, `security`, `api_ingress`, `observability`, `messaging`
`ml_pipeline`	`compute`, `storage`, `observability`
`event_driven`	`compute`, `messaging`, `observability`

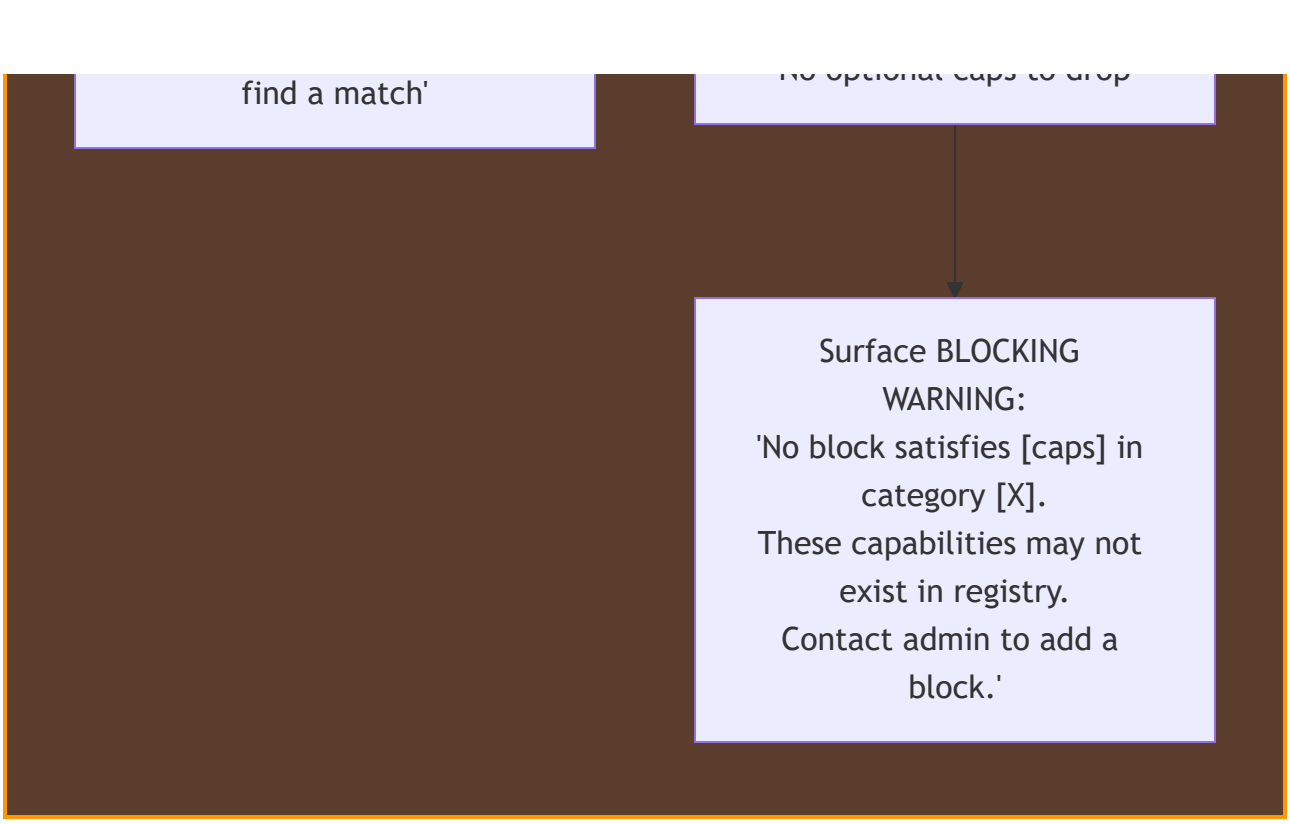

DIM 1 GUARD — Baseline
Dim 1 output: derived_categories = [compute, network] (weak intent — only 2 categories)
Lookup baseline categories for application_type: web_app
Baseline: [compute, network, security, observability]
Merge: derived ∪ baseline
Final categories: [compute, network, security, observability] security + observability added by baseline guard

Safety Net 2 — Zero-Result Guard at Dim 3

Problem: Capability requirements are too strict → no block survives the filter for a required category → that category silently disappears from the topology.

Solution: If Dim 3 returns 0 blocks for any required category, run a **capability relaxation loop**:


DIM 3 ZERO-RESULT GUARD
Dim 3 result for category:
0 blocks
Step 1: Identify optional capabilities
in the required set
(marked optional in intent schema)
Step 2: Drop one optional cap
Re-run filter
No — try next optional cap
Any blocks now?
Yes
No optional caps left
Found blocks → proceed with warning:
'Cap [X] was relaxed to
Step 3: All caps are required
No optional caps to drop

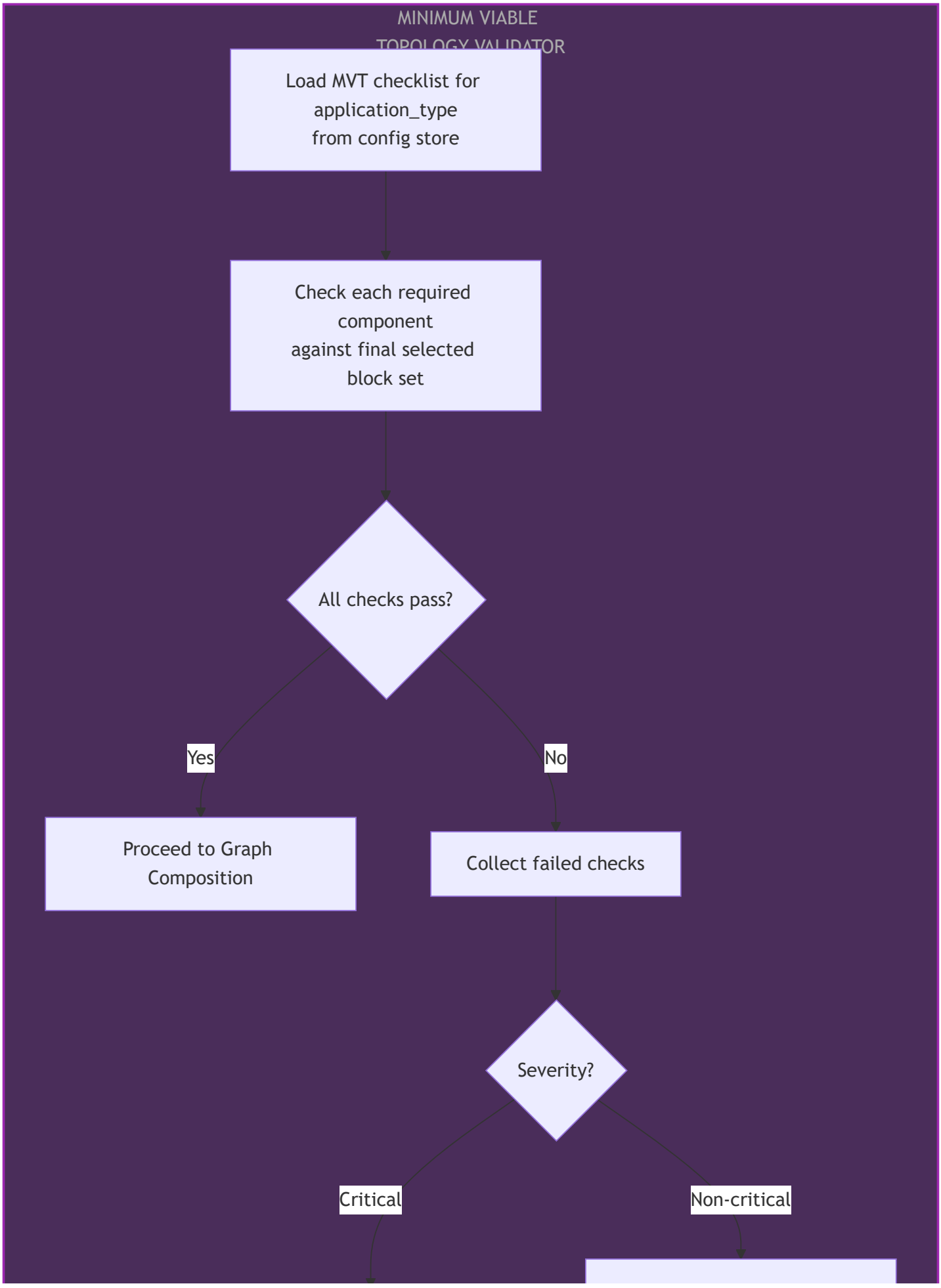


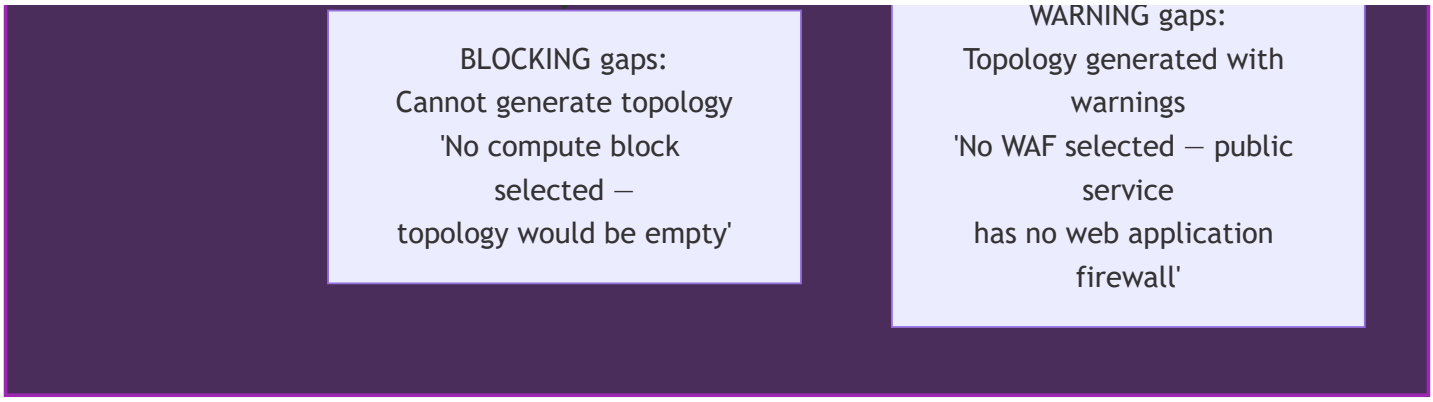
Rule: Never silently continue with a missing category. Always surface a warning or error.

Safety Net 3 — Minimum Viable Topology Validation (Post Dim 6)

Problem: After all 6 dimensions run, the resulting topology may still be architecturally incomplete — missing critical layers that policy enforcement or scoring eliminated.

Solution: Run a final checklist validation after Dim 6, before Graph Composition:

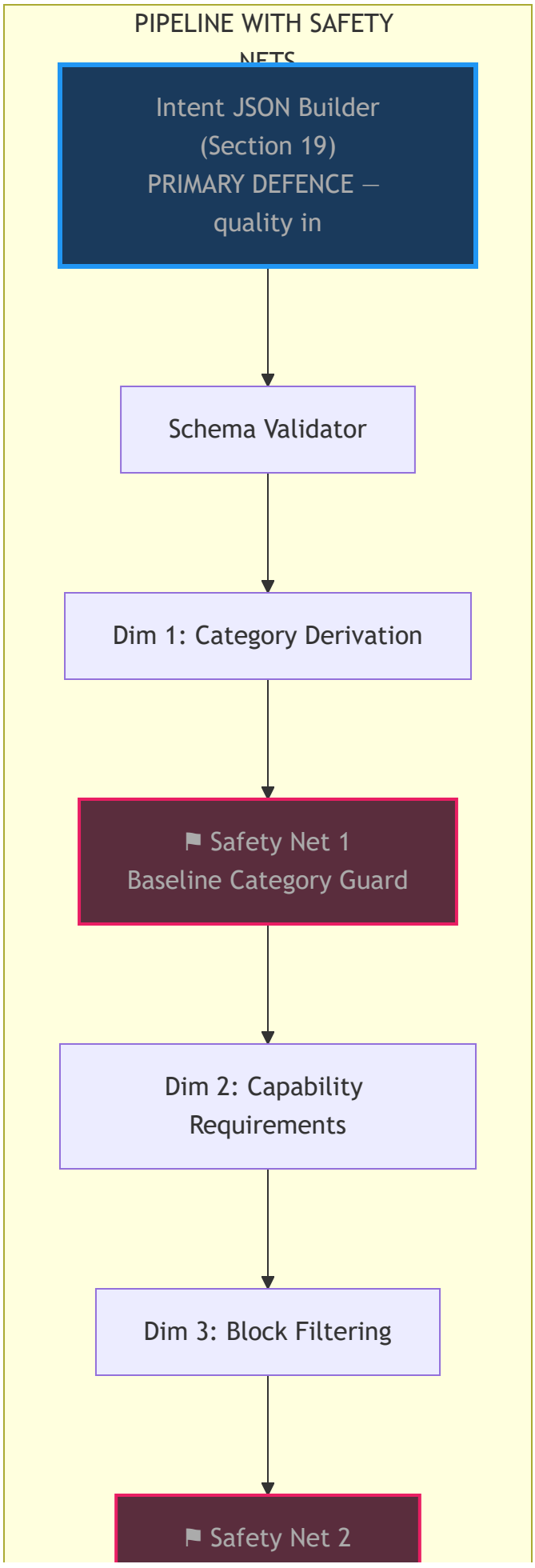

MINIMUM VIABLE
TOPOLOGY VALIDATOR
Load MVT checklist for application_type from config store
Check each required component against final selected block set
All checks pass?
Yes
No
Proceed to Graph Composition
Collect failed checks
Severity?
Critical
Non-critical

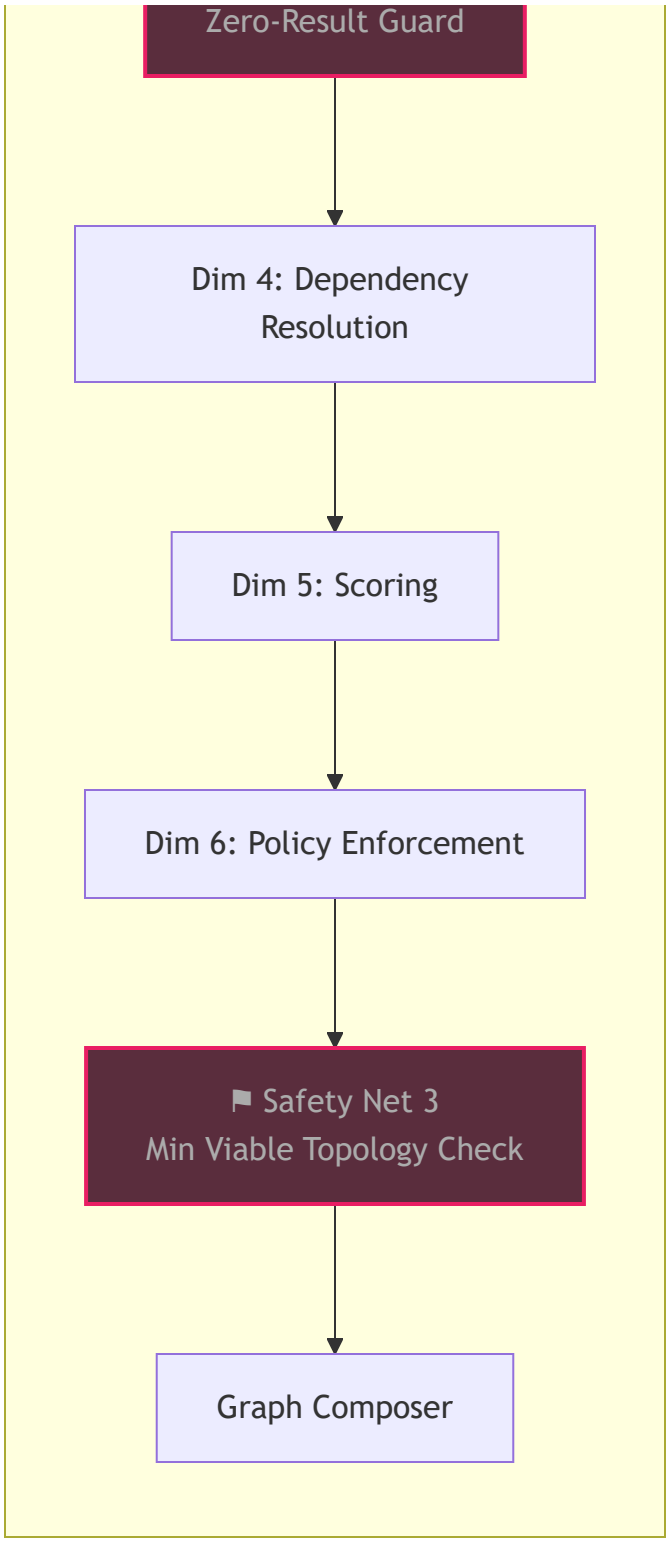


MVT Checklist Config (for `web_app`):

```
[
  { "check": "at_least_one_compute_block",   "severity": "blocking",     "message": "No compute
  { "check": "at_least_one_ingress_block",   "severity": "blocking",     "message": "No ingress
  { "check": "at_least_one_security_block",  "severity": "warning",      "message": "No security
  { "check": "at_least_one_data_block_if_data_types_declared", "severity": "blocking", "message"
  { "check": "at_least_one_observability_block", "severity": "warning",  "message": "No observab
]
```

Three Safety Nets — Where They Fire in the Pipeline


PIPELINE WITH SAFETY
NETS
Intent JSON Builder
(Section 19)
PRIMARY DEFENCE —
quality in
Schema Validator
Dim 1: Category Derivation
⚑ Safety Net 1
Baseline Category Guard
Dim 2: Capability
Requirements
Dim 3: Block Filtering
⚑ Safety Net 2



Design principle: The Intent JSON Builder (Section 19) is the primary defence — it ensures quality input. The three safety nets are the last resort for edge cases that slip through. Safety nets are the seatbelt, not the steering wheel.

Safety Net Config — All Config-Driven

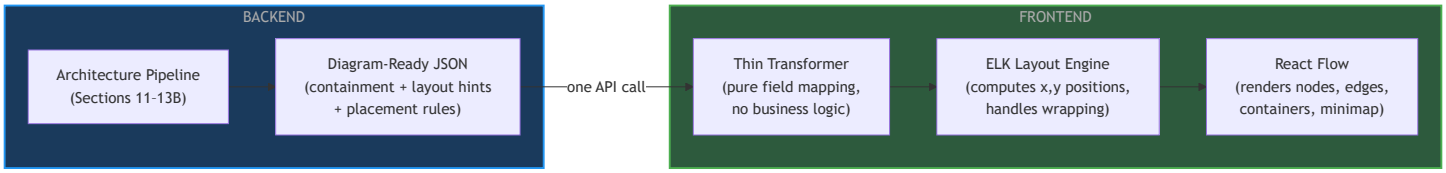
Safety Net	Config Entity	Where Managed
Baseline Category Defaults	`baseline_categories` per `application_type`	Block Registry Manager
Zero-Result Guard	`optional_capability_flags` on intent schema fields	Intent Schema Registry
MVT Checklist	`mvt_checks` per `application_type`	Policy Rule Manager

All three are config-driven. Adding a new `application_type` = adding its baseline categories + MVT checklist in config. Zero code change.

21. Diagram-Ready JSON & Frontend Rendering Strategy

Responsibility Split

The pipeline produces a **Diagram-Ready JSON** — not raw topology and not React-specific JSON. It is a middle format that carries all architectural and layout knowledge. The frontend transformer is thin and dumb: pure field mapping, zero business logic.



What lives where:

Concern	Lives In	Why
Which subnet tier a block belongs to	Block Expansion Registry (backend)	Architectural knowledge
Container hierarchy (VPC → subnet → node)	Backend Topology Builder	Architectural knowledge

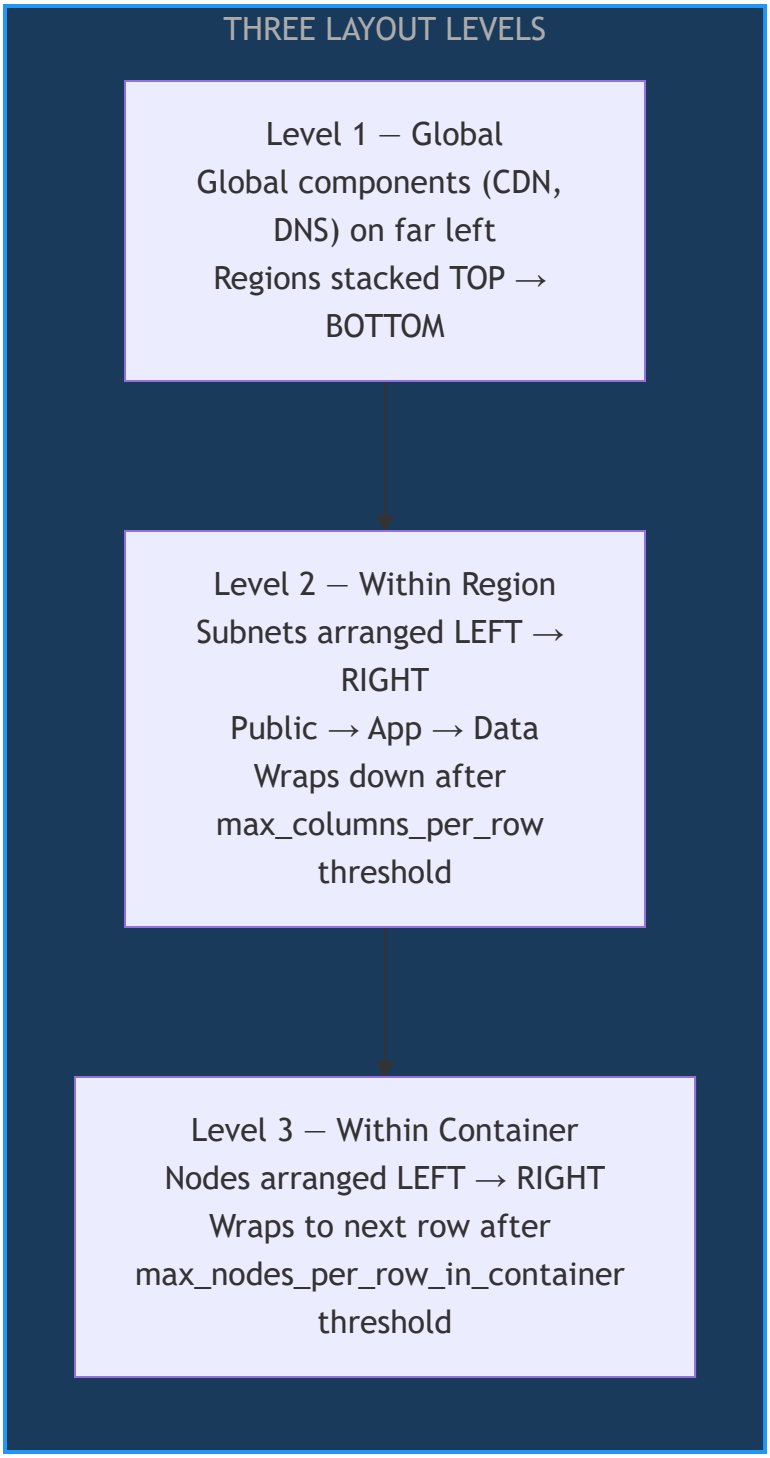
Concern	Lives In	Why
Layout direction + wrapping thresholds	Backend (Diagram-Ready JSON)	Architectural convention
Mapping JSON fields to React Flow fields	Frontend thin transformer	Pure UI mapping
Computing x,y node positions	ELK / Dagre (frontend library)	Layout algorithm
Visual styling of nodes and containers	React custom components	Pure UI concern

Layout Strategy — Left-to-Right Is the Standard

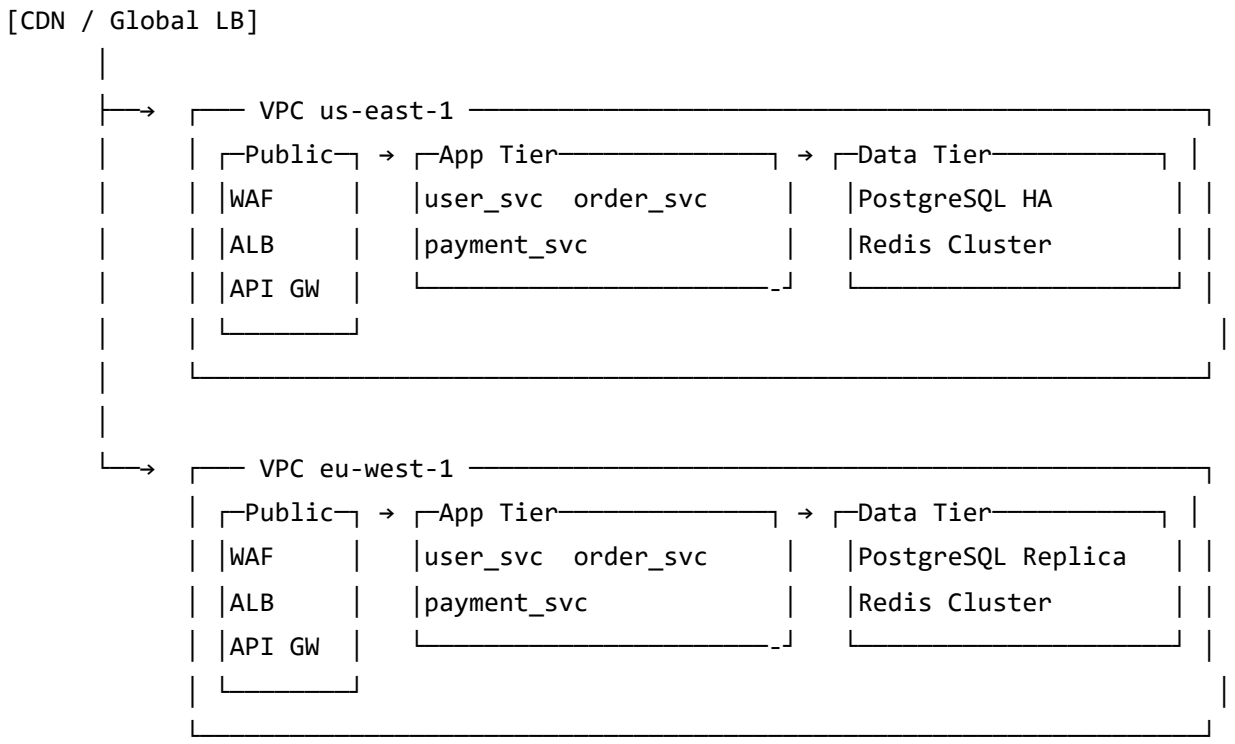
Professional architecture diagrams (AWS, Azure, GCP reference architectures) use **left-to-right** layout. Traffic flows left → right like reading direction — public-facing on the left, sensitive data on the right — matching how security architects think about network zones.

```
Top-to-Bottom (flowchart style — avoid):    Left-to-Right (architecture style — use this):

  [WAF]                                       [Public] → [App] → [Data]
    ↓                                          reads like a story:
  [API GW]                                    "request enters left,
    ↓                                          processed middle,
  [DB]                                         stored right"
```

Three layout levels, each with its own direction:



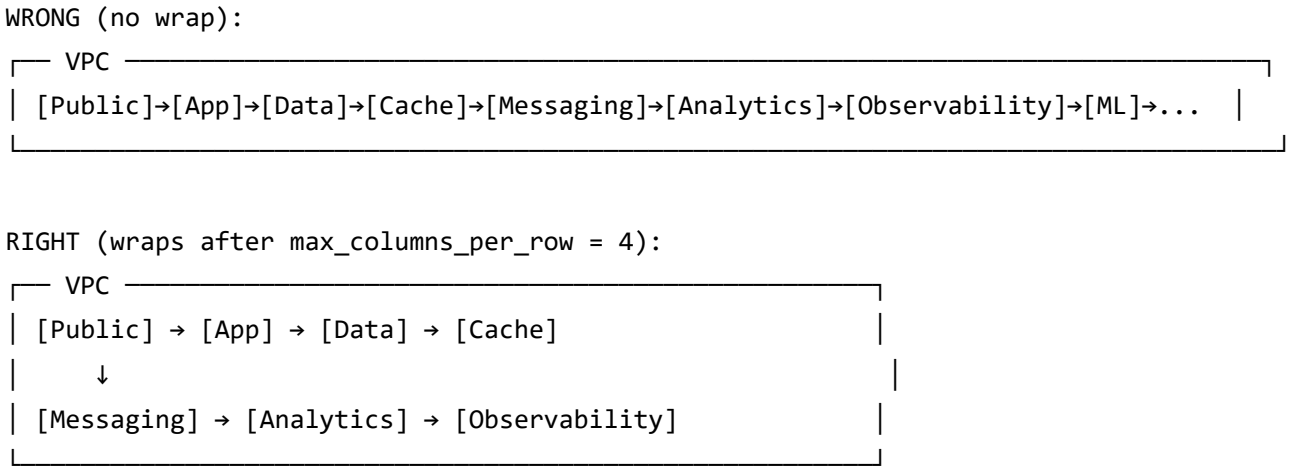
Visual result with multi-region + wrapping:



The Wrapping Problem — Two Thresholds

Without wrapping, boxes grow infinitely wide. The backend declares explicit thresholds to force wrapping at both the container and node level.

Problem 1 — Too many subnets horizontally:



Problem 2 — Too many nodes inside one subnet:

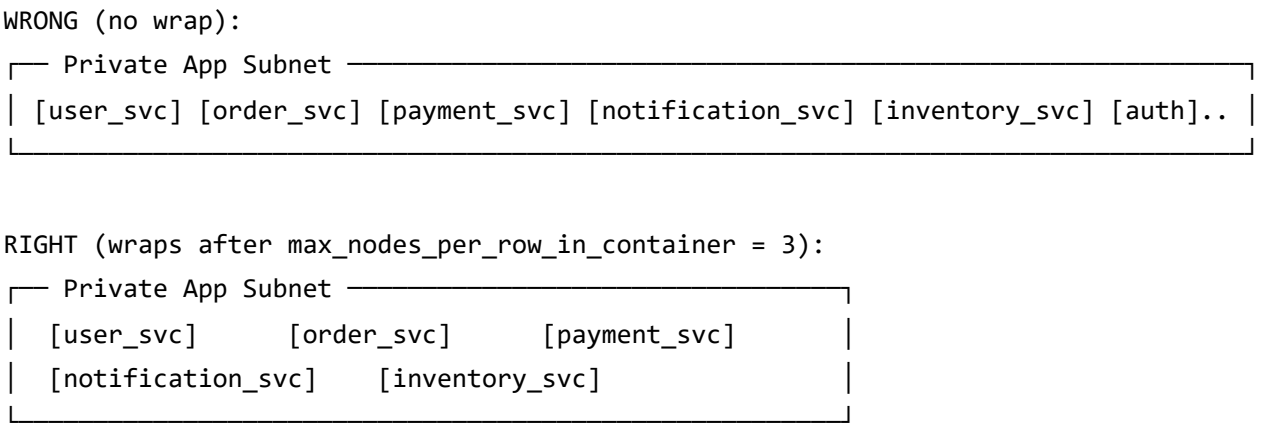


Diagram-Ready JSON — Full Format

This is what the backend pipeline outputs. The frontend consumes it directly.

```
{
  "topology_id": "topo_abc123",
  "generated_at": "2026-03-01T10:00:00Z",

  "layout": {
    "within_region_direction": "LR",
    "region_arrangement": "TB",
    "global_components_position": "left",
    "max_columns_per_row": 4,
    "max_nodes_per_row_in_container": 3,
    "target_aspect_ratio": 1.6,
    "container_max_width_px": 800
  },

  "containers": [
    {
      "container_id": "external",
      "container_type": "external_zone",
      "label": "External / Managed Services",
      "parent_container": null,
      "layer": 0,
      "max_children_per_row": 4,
      "style": "external"
    },
    {
      "container_id": "vpc-us-east",
      "container_type": "vpc",
      "label": "VPC — us-east-1 (10.0.0.0/16)",
      "region": "us-east-1",
      "parent_container": null,
      "layer": null,
      "max_children_per_row": 4,
      "style": "vpc"
    },
    {
      "container_id": "subnet-public-us-east",
      "container_type": "subnet",
      "label": "Public Subnet (10.0.1.0/24)",
      "parent_container": "vpc-us-east",
      "subnet_tier": "public",
      "layer": 1,
      "max_children_per_row": 3,
      "style": "subnet_public"
```

```
    },
    {
      "container_id": "subnet-private-app-us-east",
      "container_type": "subnet",
      "label": "Private Subnet — App Tier (10.0.2.0/24)",
      "parent_container": "vpc-us-east",
      "subnet_tier": "private_app",
      "layer": 2,
      "max_children_per_row": 3,
      "style": "subnet_private"
    },
    {
      "container_id": "subnet-private-data-us-east",
      "container_type": "subnet",
      "label": "Private Subnet — Data Tier (10.0.3.0/24)",
      "parent_container": "vpc-us-east",
      "subnet_tier": "private_data",
      "layer": 3,
      "max_children_per_row": 3,
      "style": "subnet_private"
    }
  ],

  "nodes": [
    {
      "node_id": "cdn-global",
      "block_type": "cdn",
      "product": "AWS CloudFront",
      "category": "network",
      "container_id": "external",
      "layer": 0,
      "instance_name": null,
      "spec": {}
    },
    {
      "node_id": "waf-us-east",
      "block_type": "web_application_firewall",
      "product": "AWS WAF v2",
      "category": "security",
      "container_id": "subnet-public-us-east",
      "layer": 1,
      "instance_name": null,
      "spec": {}
```

```
    },
    {
      "node_id": "user-service-us-east",
      "block_type": "microservice_compute",
      "product": "ECS Fargate",
      "category": "compute",
      "container_id": "subnet-private-app-us-east",
      "layer": 2,
      "instance_name": "user_service",
      "spec": { "cpu": 2, "ram_gb": 4, "instances": 3 }
    },
    {
      "node_id": "pg-us-east",
      "block_type": "relational_database_ha",
      "product": "RDS PostgreSQL Multi-AZ",
      "category": "data",
      "container_id": "subnet-private-data-us-east",
      "layer": 3,
      "instance_name": null,
      "spec": { "cpu": 4, "ram_gb": 32, "disk_gb": 500 }
    }
  ],

  "edges": [
    {
      "edge_id": "e1",
      "source": "cdn-global",
      "target": "waf-us-east",
      "protocol": "HTTPS",
      "port": 443,
      "edge_type": "ingress"
    },
    {
      "edge_id": "e2",
      "source": "waf-us-east",
      "target": "user-service-us-east",
      "protocol": "HTTPS",
      "port": 443,
      "edge_type": "internal"
    },
    {
      "edge_id": "e3",
      "source": "user-service-us-east",
```

```
      "target": "pg-us-east",
      "protocol": "TCP",
      "port": 5432,
      "edge_type": "data"
    }
  ]
}
```

Subnet Placement Rules — Declared in Block Expansion Registry

The rule "WAF goes in public subnet, database goes in private data subnet" lives in the **Block Expansion Registry** (backend), not the frontend. The backend reads this during topology building and sets `container_id` on each node:

Block	`default_subnet_tier`	Reasoning
`web_application_firewall`	`public`	Must receive internet traffic
`regional_load_balancer`	`public`	Must receive internet traffic
`api_gateway`	`public`	Public-facing ingress
`nat_gateway`	`public`	Provides outbound internet for private subnets
`microservice_compute`	`private_app`	Internal app logic, not exposed directly
`ingress_controller`	`private_app`	K8s internal routing
`relational_database_ha`	`private_data`	Never publicly accessible
`in_memory_cache`	`private_data`	Never publicly accessible
`message_queue`	`private_app`	Internal async communication
`cdn`	`external`	Managed service outside VPC
`object_storage`	`external`	Managed service outside VPC
`audit_trail`	`external`	Managed logging service

Frontend Technology Stack

Concern	Library	Why
Diagram renderer	**React Flow**	React-native, JSON-driven, free, supports `parentNode` for nested containers
Auto layout + wrapping	**ELK (Eclipse Layout Kernel)**	Native support for `wrapping.strategy`, aspect ratio, hierarchical graphs — better than Dagre for nested layouts
Custom node cards	React components	Full control over block card appearance per category
Region/subnet containers	React Flow `parentNode` + `extent: parent`	Native containment support
Edge routing	React Flow `smoothstep` or `step`	Clean right-angle routing for architecture diagrams

ELK configuration for this layout:

```
const elkOptions = {
  algorithm: 'layered',
  'elk.direction': 'RIGHT',
  'elk.layered.wrapping.strategy': 'MULTI_EDGE',
  'elk.layered.wrapping.additionalEdgeSpacing': 20,
  'elk.aspectRatio': layout.target_aspect_ratio,
  'elk.spacing.nodeNode': 40,
  'elk.layered.spacing.nodeNodeBetweenLayers': 60,
  'nodePlacement.strategy': 'NETWORK_SIMPLEX'
}
```

Frontend Thin Transformer — Pure Field Mapping

```
function transformToReactFlow(diagramJson) {
  const nodes = []
  const edges = []

  // Map containers → React Flow parent nodes (VPC, subnet, external zone)
  diagramJson.containers.forEach(c => {
    nodes.push({
      id: c.container_id,
      type: c.container_type,              // 'vpc' | 'subnet' | 'external_zone'
      parentNode: c.parent_container || undefined,
      extent: c.parent_container ? 'parent' : undefined,
      position: { x: 0, y: 0 },           // ELK computes this
      data: {
        label: c.label,
        style: c.style,
        layer: c.layer,
        maxChildrenPerRow: c.max_children_per_row
      }
    })
  })

  // Map topology nodes → React Flow nodes
  diagramJson.nodes.forEach(n => {
    nodes.push({
      id: n.node_id,
      type: 'architectureBlock',
      parentNode: n.container_id,         // ← direct field mapping, zero logic
      extent: 'parent',
      position: { x: 0, y: 0 },
      data: { ...n }
    })
  })

  // Map edges → React Flow edges
  diagramJson.edges.forEach(e => {
    edges.push({
      id: e.edge_id,
      source: e.source,
      target: e.target,
      label: `${e.protocol}:${e.port}`,
      type: 'smoothstep',
```

```
      data: { edge_type: e.edge_type }
    })
  })

  // Pass to ELK for layout computation — no position logic here
  return applyELKLayout(nodes, edges, diagramJson.layout)
}
```

No architectural decisions on the frontend. All layout and placement knowledge came pre-declared in the Diagram-Ready JSON from the backend.

22. Diagram Scenarios & Rendering Modes — Complete Coverage

The pipeline generates one topology but the frontend can render it in **multiple views and modes**. All view switching is driven by the backend — the frontend receives different Diagram-Ready JSON payloads per view, never computes what to show.

Scenario 1 — Availability Zone (AZ) Sub-Level Containment

AWS, GCP, and Azure deploy resources across multiple **Availability Zones** within a region for HA. This adds one level to the containment hierarchy between VPC and Subnet.

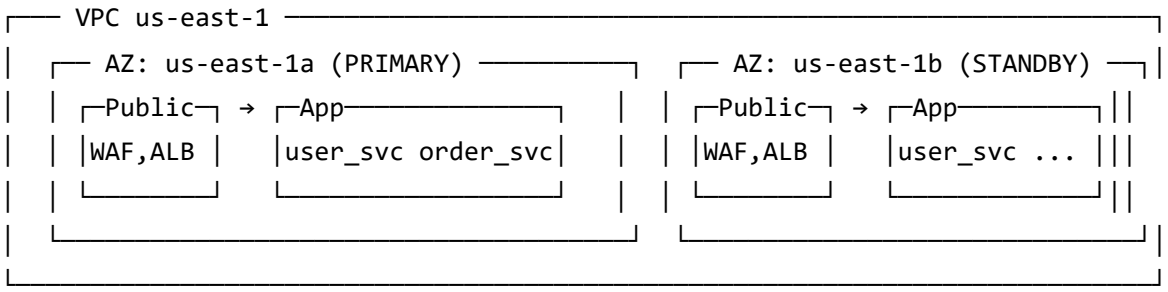
Full containment hierarchy:

```
VPC (region)
  └── Availability Zone A (us-east-1a)
        └── Public Subnet
              └── [WAF, ALB]
        └── Private App Subnet
              └── [user_service, order_service]
  └── Availability Zone B (us-east-1b)
        └── Public Subnet (standby)
              └── [WAF, ALB]
        └── Private App Subnet (replica)
              └── [user_service, order_service]
```

Container type additions to Diagram-Ready JSON:

```
{
  "container_id": "az-us-east-1a",
  "container_type": "availability_zone",
  "label": "AZ: us-east-1a",
  "parent_container": "vpc-us-east",
  "az_role": "primary",
  "max_children_per_row": 3,
  "style": "az_primary"
},
{
  "container_id": "az-us-east-1b",
  "container_type": "availability_zone",
  "label": "AZ: us-east-1b",
  "parent_container": "vpc-us-east",
  "az_role": "standby",
  "max_children_per_row": 3,
  "style": "az_standby"
},
{
  "container_id": "subnet-public-1a",
  "container_type": "subnet",
  "label": "Public Subnet",
  "parent_container": "az-us-east-1a",
  "subnet_tier": "public",
  "layer": 1,
  "style": "subnet_public"
}
```

Visual result:



Subnet placement rule update in Block Expansion Registry — subnet now has both `subnet_tier` and `az_placement`:

Block	subnet_tier	az_placement
`web_application_firewall`	`public`	`all_azs`
`regional_load_balancer`	`public`	`all_azs`
`microservice_compute`	`private_app`	`all_azs`
`relational_database_ha` (primary)	`private_data`	`az_a`
`relational_database_ha` (replica)	`private_data`	`az_b`
`nat_gateway`	`public`	`per_az`

Scenario 2 — Internet Boundary Visual Separator

Every professional architecture diagram shows a visual line separating the public internet from the VPC boundary. This is a special **boundary_line** container type — not a node, just a visual separator.

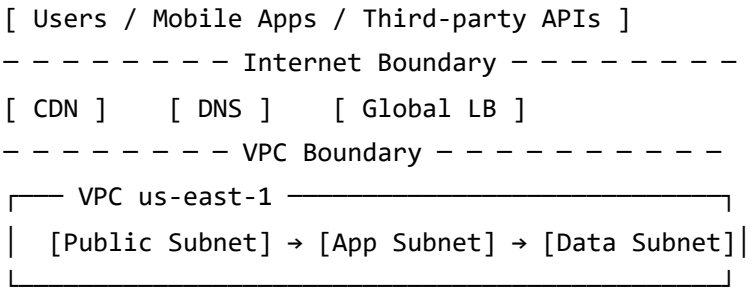
Two boundary lines are standard:

1. **Internet boundary** — between public internet (CDN, DNS, users) and the VPC edge
2. **VPC boundary** — the VPC container itself already represents this

Diagram-Ready JSON addition:

```
{
  "container_id": "internet-zone",
  "container_type": "internet_zone",
  "label": "Public Internet",
  "parent_container": null,
  "layer": -1,
  "max_children_per_row": 4,
  "style": "internet"
},
{
  "boundary_lines": [
    {
      "boundary_id": "internet-boundary",
      "label": "Internet Boundary",
      "separates": ["internet-zone", "external"],
      "style": "dashed_orange"
    },
    {
      "boundary_id": "vpc-boundary",
      "label": "VPC Boundary (Private Network)",
      "separates": ["external", "vpc-us-east"],
      "style": "solid_blue"
    }
  ]
}
```


Visual result:



Scenario 3 — Primary/Replica Edge Styling & HA Badges

Replication edges and HA relationships are architecturally distinct from normal traffic edges. They need separate visual treatment.

Edge types and their visual styles:

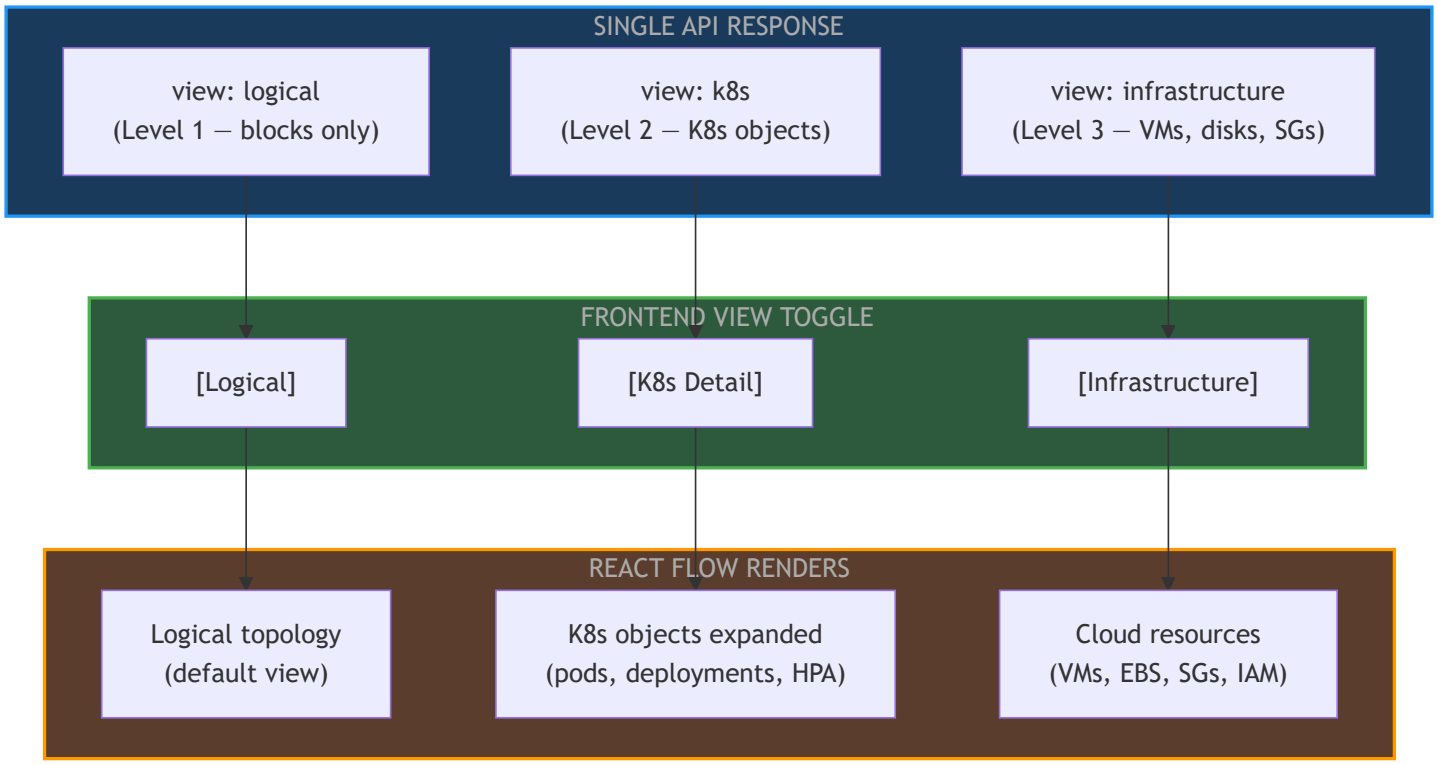
`edge_type`	Line Style	Color	Label Example
`ingress`	Solid, thick	Blue	`HTTPS:443`
`internal`	Solid, normal	Grey	`gRPC:8443`
`data`	Solid, normal	Orange	`TCP:5432`
`replication`	Dashed, animated	Purple	`streaming replication`
`async`	Dashed, static	Grey	`async:amqp`
`cross_region`	Dashed, thick	Red	`cross-region replication`

Node badges declared in the Diagram-Ready JSON:

```
{
  "node_id": "pg-primary-us-east",
  "block_type": "relational_database_ha",
  "badges": [
    { "badge_type": "ha_role", "value": "PRIMARY", "style": "green" }
  ]
},
{
  "node_id": "pg-replica-eu-west",
  "block_type": "relational_database_ha",
  "badges": [
    { "badge_type": "ha_role", "value": "REPLICA", "style": "blue" },
    { "badge_type": "lag", "value": "< 1s", "style": "grey" }
  ]
},
{
  "edge_id": "repl-pg",
  "source": "pg-primary-us-east",
  "target": "pg-replica-eu-west",
  "protocol": "TCP",
  "port": 5432,
  "edge_type": "cross_region_replication",
  "label": "async streaming replication"
}
```


Scenario 4 — Multiple Depth Views (Level Toggle)

Section 13A defines 3 depth levels but the switching mechanism was undefined. The strategy: the backend generates **all depth levels in one API response**. The frontend toggles between them without additional API calls.



API response shape:

```
{
  "topology_id": "topo_abc123",
  "views": {
    "logical": { ...Diagram-Ready JSON for Level 1... },
    "k8s": { ...Diagram-Ready JSON for Level 2... },
    "infrastructure": { ...Diagram-Ready JSON for Level 3... }
  },
  "default_view": "logical"
}
```

Drill-down within logical view — clicking a K8s compute node in the logical view opens an **inline expanded panel** showing its Level 2 K8s detail without switching the whole diagram:

```
{
  "node_id": "user-service-us-east",
  "expandable": true,
  "expanded_view_ref": "k8s.user-service-us-east"
}
```

Scenario 5 — Legend

Every architecture diagram needs a legend. The backend declares it — the frontend renders it as a fixed overlay panel.

Diagram-Ready JSON addition:

```
{
  "legend": {
    "categories": [
      { "category": "compute",      "color": "#4caf50", "label": "Compute" },
      { "category": "network",      "color": "#2196f3", "label": "Network" },
      { "category": "security",     "color": "#e91e63", "label": "Security" },
      { "category": "data",         "color": "#ff9800", "label": "Data" },
      { "category": "cache",        "color": "#cddc39", "label": "Cache" },
      { "category": "storage",      "color": "#9c27b0", "label": "Storage" },
      { "category": "messaging",    "color": "#00bcd4", "label": "Messaging" },
      { "category": "observability","color": "#607d8b", "label": "Observability" }
    ],
    "edge_types": [
      { "edge_type": "internal",              "style": "solid",  "color": "#888",    "label": "I
      { "edge_type": "ingress",               "style": "solid",  "color": "#2196f3", "label": "I
      { "edge_type": "replication",           "style": "dashed", "color": "#9c27b0", "label": "R
      { "edge_type": "cross_region_replication","style":"dashed","color": "#e91e63", "label": "C
    ],
    "badges": [
      { "badge_type": "ha_role", "value": "PRIMARY",  "label": "HA Primary node" },
      { "badge_type": "ha_role", "value": "REPLICA",  "label": "HA Replica node" }
    ],
    "containers": [
      { "container_type": "vpc",               "style": "solid_blue",   "label": "VPC boundary"
      { "container_type": "availability_zone", "style": "dashed_grey",  "label": "Availability Z
      { "container_type": "subnet",            "style": "solid_grey",   "label": "Subnet" },
      { "container_type": "internet_zone",     "style": "dashed_orange","label": "Public Interne
    ]
  }
}
```

Scenario 6 — Compliance / Security Overlay View

Same topology, different rendering mode. Highlights compliance scope and security coverage without changing the diagram structure. Toggled by a frontend mode button — no new API call needed.

Diagram-Ready JSON — compliance annotations on nodes:

```
{
  "node_id": "pg-primary-us-east",
  "compliance": {
    "in_scope_for": ["pci_dss", "soc2"],
    "encryption_at_rest": true,
    "audit_logged": true,
    "security_group_id": "sg-db-private"
  }
}
```

Rendering modes declared in the JSON:

```
{
  "rendering_modes": [
    {
      "mode_id": "default",
      "label": "Architecture View",
      "node_coloring": "by_category",
      "edge_coloring": "by_type",
      "badges_shown": ["ha_role"]
    },
    {
      "mode_id": "compliance",
      "label": "Compliance View",
      "node_coloring": "by_compliance_scope",
      "edge_coloring": "by_encryption",
      "badges_shown": ["compliance_scope", "encryption_at_rest", "audit_logged"],
      "highlight_rule": "in_scope_for contains pci_dss → green border, else grey"
    },
    {
      "mode_id": "cost",
      "label": "Cost View",
      "node_coloring": "by_cost_tier",
      "node_sizing": "by_monthly_cost",
      "badges_shown": ["monthly_cost_usd"],
      "show_total_cost": true
    },
    {
      "mode_id": "diff",
      "label": "Change View",
      "node_coloring": "by_change_status",
      "badges_shown": ["change_type"],
      "highlight_rule": "added → green, removed → red, modified → yellow, unchanged → grey"
    }
  ],
  "default_mode": "default"
}
```


Cost annotations on nodes:

```
{
  "node_id": "pg-primary-us-east",
  "cost": {
    "monthly_estimate_usd": 340,
    "cost_tier": "high",
    "cost_breakdown": {
      "compute": 180,
      "storage": 120,
      "io": 40
    }
  }
}
```

Diff annotations on nodes (for change view):

```
{
  "node_id": "payment-service-us-east",
  "diff": {
    "change_type": "added",
    "previous_spec": null,
    "current_spec": { "cpu": 2, "ram_gb": 4 }
  }
},
{
  "node_id": "pg-primary-us-east",
  "diff": {
    "change_type": "modified",
    "previous_spec": { "cpu": 2, "ram_gb": 16 },
    "current_spec":  { "cpu": 4, "ram_gb": 32 }
  }
}
```

Scenario 7 — Request Flow / Sequence View

A fundamentally different diagram type showing **how a single user request travels through the system** — temporal sequence, not spatial arrangement. Generated from the same topology's edges but rendered as a swimlane diagram.

Flow paths declared in Diagram-Ready JSON:

```
{
  "request_flows": [
    {
      "flow_id": "user_login_flow",
      "label": "User Login Request",
      "steps": [
        { "step": 1, "from": "user",              "to": "cdn-global",          "action": "HTTPS
        { "step": 2, "from": "cdn-global",        "to": "waf-us-east",         "action": "forwa
        { "step": 3, "from": "waf-us-east",       "to": "apigw-us-east",       "action": "forwa
        { "step": 4, "from": "apigw-us-east",     "to": "user-service-us-east", "action": "gRPC
        { "step": 5, "from": "user-service-us-east","to": "redis-us-east",     "action": "cache
        { "step": 6, "from": "user-service-us-east","to": "pg-us-east",        "action": "SELEC
        { "step": 7, "from": "pg-us-east",        "to": "user-service-us-east", "action": "retur
        { "step": 8, "from": "user-service-us-east","to": "redis-us-east",     "action": "cache
        { "step": 9, "from": "user-service-us-east","to": "apigw-us-east",     "action": "retur
        { "step": 10,"from": "apigw-us-east",     "to": "user",                "action": "200 O
      ]
    }
  ]
}
```

The frontend renders this as a **swimlane diagram** (separate from the topology view) using the same node IDs — consistent naming between views.

Complete Diagram-Ready JSON — Extended Structure

The full structure combining all scenarios:

```
{
  "topology_id": "topo_abc123",
  "generated_at": "2026-03-01T10:00:00Z",

  "views": {
    "logical":         { ...Level 1 Diagram-Ready JSON... },
    "k8s":             { ...Level 2 Diagram-Ready JSON... },
    "infrastructure":  { ...Level 3 Diagram-Ready JSON... }
  },
  "default_view": "logical",

  "rendering_modes": [ ...default, compliance, cost, diff... ],
  "default_mode": "default",

  "legend": { ...categories, edge_types, badges, containers... },

  "request_flows": [ ...flow paths for sequence view... ],

  "layout": {
    "within_region_direction": "LR",
    "region_arrangement": "TB",
    "global_components_position": "left",
    "max_columns_per_row": 4,
    "max_nodes_per_row_in_container": 3,
    "target_aspect_ratio": 1.6,
    "container_max_width_px": 800
  }
}
```

All Diagram Scenarios — Coverage Summary

Scenario	Driven By	In Doc Now?
Basic topology (nodes + edges)	Section 12, 13	Yes
Multi-region (regions side by side)	Section 13	Yes
VPC → Subnet → Resource containment	Section 21	Yes
LR layout with wrapping	Section 21	Yes

Scenario	Driven By	In Doc Now?
External managed services zone	Section 21	Yes
AZ-level containment	Section 22 (this)	Yes
Internet boundary visual	Section 22 (this)	Yes
Primary/Replica badges + edge styles	Section 22 (this)	Yes
Multiple depth views (Level 1/2/3 toggle)	Section 22 (this)	Yes
Legend	Section 22 (this)	Yes
Compliance overlay view	Section 22 (this)	Yes
Cost view	Section 22 (this)	Yes
Diff / change view	Section 22 (this)	Yes
Request flow / sequence view	Section 22 (this)	Yes

> **End of Configurator Layer Architecture Addendum**